

情報工学実験Ⅲ・実験報告書

実験テーマ

実験題目

C 後半

ディープラーニングの実験

Experiment of deep learning

班番号

水-12

■予習自己点検（予習で理解してきた項目に自身で○を付ける）

はじめに・全用語・目的とねらい・実験手順・結果と考察事項

学生番号	4096	氏名	松本 進秀
------	------	----	-------

共同実験者（実験データを共有する者）

実験日（1） 年 月 日（ ）	学生番号	学生番号	学生番号	学生番号
	氏名	氏名	氏名	氏名
実験日（2） 年 月 日（ ）	学生番号	学生番号	学生番号	学生番号
	氏名	氏名	氏名	氏名
実験日（3） 年 月 日（ ）	学生番号	学生番号	学生番号	学生番号
	氏名	氏名	氏名	氏名

予習レポート 年 月 日（ ）	本レポート 初回 年 月 日（ ）	本レポート 2回目 年 月 日（ ）	本レポート 3回目 年 月 日（ ）
■禁止 簡易な図のコピー カラー印刷・文章丸写し 未完成 公的機関以外の参照 ■不足[全般] レイアウト・フォント 見出し・文章 図・表・式 項番・ページ番号 ■誤り又は過不足[項目] 表紙・目的・原理・方法 予習課題・参考文献	■禁止 簡易な図のコピー カラー印刷・文章丸写し 未修正・未完成 公的機関以外の参照 指摘なし ■不足[全般] レイアウト・フォント 見出し・文章 図・表・式 項番・ページ番号 指摘なし ■誤り又は過不足[項目] 表紙・目的・原理・方法 予習課題・使用器具 実験結果・考察課題 実験考察・参考文献 指摘なし	■禁止 簡易な図のコピー カラー印刷・文章丸写し 未修正・未完成 公的機関以外の参照 指摘なし ■不足[全般] レイアウト・フォント 見出し・文章 図・表・式 項番・ページ番号 指摘なし ■誤り又は過不足[項目] 表紙・目的・原理・方法 予習課題・使用器具 実験結果・考察課題 実験考察・参考文献 指摘なし	

1. 目的

Python とディープラーニングライブラリを用いてディープラーニングによる画像認識モデルの構築および構築したモデルの改良を行い、ディープラーニングに関する基本的な知識や理論を実践的に身に付けていくことが全体の目的である。C3 実験では、PyTorch ライブラリを用いてディープニューラルネットワークの構築の基本を習得することを目的とする。C4 実験では、MNIST データを例としたディープニューラルネットワークのパラメータの変更や、CNN の最適化等の技術を習得することを目的とする。C5 実験では、自身で学習するデータを収集し、CNN の学習を行う方法を習得することを目的とする。

2. 原理

2.1 Colaboratory のコードスニペット

Colaboratory には使用頻度の高いコードなどをスニペットとして保存する機能が備わっている。保存したスニペットは「コードスニペット」から呼び出すことができる。

標準で用意されているスニペットが多数あり、Colab ノートブックと GoogleDrive との連携のためのスニペットや、web カメラとの連携のためのスニペットなどが利用できる。Colab ノートブックを開き、メニューバーの「挿入」から「コードスニペット」を選択することで、画面左のペインに利用可能なスニペットが表示される。一覧から利用するスニペットを選択し「挿入」から編集中のノートブック内でスニペットを利用できる。

2.2 Colaboratory と Google Drive の連携

Colaboratory と Google Drive の連携は図 1 のコードで実現可能である。このコードはコードスニペットの「Mounting Google Drive in your VM」を使用してノートブックに挿入できる。

```
1 from google.colab import drive
2 drive.mount('/gdrive')
```

図 1 Google Drive と連携するためのコード

連携後は/gdrive/Mydrive/から毎ドライブのディレクトリ、ファイルにアクセスすることができる。

2.3 pyplot による画像の表示

matplotlib ライブラリに pyplot モジュールには、画面表示に必要なメソッド imshow() が用意されており、また、画像の読み込みでは、matplotlib ライブラリの image モジュールから、imread() メソッドを利用することができる。

2.4 畳み込みニューラルネットワーク (Convolutional Neural Network:CNN)

CNN は脳の視覚野の情報処理機能から発案されたニューラルネットワークである。構造として画像認識に適しており、主に畳み込み層とプーリング層で構成される。分類問題を解く場合、出力を分類クラスの数へ落とし込む際に全結合層が利用されることが多い。畳み込み演算により同一の局所フィルタを画像全体に適用することで、全体の結合の値が共有される重み共有が行われることが特徴の一つである。これにより、ネットワークを高次元化するパラメータ数が小さくなる点や、空間位置に依存しない特徴抽出が可能である。畳み込み層の演算は画像処理によるフィルタ処理と類似しており、畳み込み層は画像から何らかの特徴の抽出に用いられる。畳み込み層を経た出力は画像と同様の空間的構造を保ち、一般に特徴マップと呼ばれる。畳み込み層にはフィルタのサイズやストライド、チャンネル数などのパラメータが存在する。カーネルサイズとは画像や特徴マップにおける特徴抽出のピクセルの範囲の指定を行う。AlexNet などでは 7×7 や 11×11 などのカーネルサイズが用いられていたが、VGGNet 以降は 3×3 などの小さいフィルタの積み重ねにより、パラメータの削減と同時に近似的に同様の範囲から特徴の抽出が実現でき、主流となった。チャンネル数は畳み込み層で抽出する特徴の数といえるため、多いほど様々な特徴が利用可能であり、モデルの表現力が向上する。しかし、必要以上に多い場合学習するパラメータ数が増大し、学習データに対する過剰な適合の恐れがある。

畳み込み層で抽出する特徴マップは、画像中に映る物体の位置の変化に敏感であるが、画像分類問題においては映る位置に関係なく同様の特徴を抽出する必要がある。このような位置の普遍性を獲得するための方法としてはあげられるものは、特徴マップの空間解像度を下げることである。プーリング層はこれを目的としたものであり、基本的に 2×2 の空間内の代表値を利用して近傍の出力を集約させる。この処理により、特徴マップのサイズは 4 分の 1 になるため位置ずれの感度の低下が期待できる。

畳み込みやプーリングを繰り返すことで、後の特徴マップほど特徴抽出に関与する入力画像の範囲が増大する。CNN の設計においては、各総出の特徴マップのサイズや受容野が重要である。

2.5 ディープラーニングライブラリ

ディープニューラルネットワークのモデル構築や学習においては、ディープラーニングライブラリの利用が一般的となっている。広く知られているのは、Google が中心となり開発している TensorFlow と、Meta が中心となり開発している PyTorch である。また、TensorFlow はその高レベル API である Keras としても知られている。

それぞれのライブラリにはメリット、デメリットが存在している。一般に、企業などにおける製品開発においては TensorFlow に、大学などにおける研究の領域では PyTorch に利点があるといわれている。本実験では主に PyTorch を用いる。

2.6 PyTorch

PyTorch とは、2017FAIR から発表されたオープンソースのディープラーニングライブラリである。ディープラーニングライブラリにおける重要な概念として、静的計算グラフと動的計算グラフがあげられるが、PyTorch は静的計算グラフを採用している。

まず、ニューラルネットワークの学習における重要な要素である誤差逆伝播は、勾配降下法を用いた最適化に必要な勾配の計算を、合成関数の微分公式を基に効率的に求めるものである。よって、誤差伝播学習法お用いる限りはニューラルネットワークの学習には微分計算が不可欠である。ディープラーニングライブラリは基本的に、ニューラルネットワークを計算グラフとして連鎖率を用いた自動微分機能を持つ。計算グラフが静的であるとは、事前にモデルを定義しコンパイル後に処理を行うことを指す。静的計算グラフの場合、事前に計算グラフの形を固定することでコード最適化が容易であるが、ネットワーク構造への制約が強くなるほか標準的な Python 体系とは異なる特有のコーディング知識が必要となる。動的であるとは、データを処理しながら計算グラフの構築を行うことを指す。動的計算グラフの場合、モデルに入力されたデータに応じて構造の変更が可能であり、デバッグが容易である。

2.7 PyTorch によるディープニューラルネットワークの学習プログラム

PyTorch を用いてディープニューラルネットワークを学習するプログラムは、基本的に 6 つのパートに分けることで理解が容易となる。以下に①～⑥のパートとして分けて示す。

① ライブラリの読み込み

図 2 に、PyTorch を用いてディープニューラルネットワークの学習を行う標準的なインポート分を示す。PyTorch は `torch` という名前でインポートできる。ニューラルネットワークを構築する全結合層や畳み込み層などの部品の多くは `torch.nn` にまとめられている。関数類などは `nn` モジュール内の `functional` にまとめられており、別途インポートすると利便性が高い。SGD や Adam など最適化に関するものは `torch.optim` モジュールにまとめられている。`Torch.utils.data` モジュールは、ミニバッチ学習の実装において重要となるデータローダの作成に用いる。`Torchvision` ライブラリはインポートすることで、コンピュータビジョンにおいて有名なデータセットやモデル、一般的な画像処理などが利用可能である。`Torchsummary` は、モデルの構造などのサマ리를詳細に表示するためのライブラリである。

```
1 import torch
2 from torch import nn
3 from torch.nn import functional as F
4 from torch import optim
5 from torch.utils import data
6 import torchvision
7 from torchvision import datasets
8 from torchvision import transforms as T
9 from torchsummary import summary
```

図 2 インポート文の例

② データ準備・設定

本実験では主に画像データを対象とするが、Python で画像データを扱うためのライブラリとして、Pillow や OpenCV がある。これらのライブラリを用いて読み込んだ画像データは 0~255 の値域の NumPy 配列であり、形状は Height、Width、Channels となる。PyTorch においては畳み込み層などでの扱いやすさから、C,H,W の形状で扱われる。また、PyTorch を用いたディープラーニングでは NumPy ではなく Tensor を用いる。自動微分や GPU デバイス上での演算などの機能を備えた専用のデータ型であり、NumPy 配列を PyTorch 内で扱うためには Tensor に変換する必要がある。これらの前処理を一括で行うものが `transforms.ToTensor()` である。

画像を処理する際、全結合層はユニット数が画像の各画素×チャネル分必要となるため、画像スケールが変わると処理ができない。簡単な対策としては、ネットワークへの入力前に一定のサイズに画像をリサイズして利用することである。これを行うものが `transforms.Resize` である。比率変化により特徴がゆがめられるのを防ぐために、`transforms.CenterCrop()` や `transforms.RandomCrop()` などの切り抜き処理とも組み合わせられる。

図 3 に画像データに対する前処理の例を示す。複数の前処理を記述したい場合、`transforms.Compose` にリストとして渡すことで実装が可能である。

```
1 image_size = 32 #任意の画像サイズ
2 transform = T.Compose(
3     [T.CenterCrop((256, 256)), #任意の切り抜きサイズ
4     T.RandomCrop((224, 224)), #任意の切り抜きサイズ
5     T.Resize((image_size, image_size)),
6     T.ToTensor()]
7 )
```

図 3 画像データに対する前処理の記述例

③ ディープニューラルネットワークモデルの定義

学習するモデルは `nn.Module` クラスを継承する形で定義する必要があり、`__init__()` メソッドと `forward()` メソッドの定義が必須である。図 4 にモデル定義の例を示す。

```
1 class MyModel(nn.Module):
2     def __init__(self):
3         super(MyModel, self).__init__()
4         self.conv = nn.Conv2d(1, 128, 3)
5         self.flat = nn.Flatten()
6         self.fc = nn.Linear(86528, 10)
7     def forward(self, x):
8         h = self.conv(x)
9         h = F.relu(h)
10        h = self.flat(h)
11        y = self.fc(h)
12        return y
```

図 4 ディープニューラルネットワークモデル定義の例

`__init__()`メソッドでは主に、全結合層や畳み込み層などの定義を行う。`Forward()`メソッドでは、伝番の処理の記述を行う。

CNN などにおいて、画像データなどの空間的構造を持つデータを全結合層に入力するには、平坦化が必要がある。PyTorch では `nn.Flatten()` クラスから容易に実装できる。ここで、CNN の設計では畳み込み層の出力を平坦化し、全結合層における入力ユニット数の設定が重要となる。適切な CNN の設計を行うには、入力画像の空間解像度やチャンネル数が特徴抽出の過程でどのように変化していくかを把握しておく必要がある。

すべての層を直列に連ねるネットワーク構造の場合、`nn.Sequential` クラスを利用すると便利に記述できるが、その一方エラー時のデバックに難点がある。図 5 に、図 4 を `nn.Sequential` クラスにより書き換えたコードを示す。`nn.Sequential` クラスのインスタンスには、`add_module()`メソッドを利用することで層の追加が可能である。

```
1 class MyModel(nn.Module):
2     def __init__(self):
3         super(MyModel, self).__init__()
4         self.net = nn.Sequential(
5             nn.Conv2d(1, 128, 3),
6             nn.ReLU(),
7             nn.Flatten())
8         self.net.add_module(
9             "out_linear", nn.Linear(86528, 10))
10    )
11    def forward(self, x):
12        y = self.net(x)
13        return y
```

図 5 `nn.Sequential` を利用したモデル定義の例

④ ディープニューラルネットワークモデルの生成と最適化方法の設定

ディープニューラルネットワークモデルの生成と最適化方法の設定の例を図 7 に示す。GPU で処理するには、モデルのインスタンスを GPU デバイスに事前に送る必要がある。`Model.to("cuda")` のように記述することで可能である。図 6 の例においては、GPU が使用できる場合は `"cuda"` を、使用できない場合は `"cpu"` を自動的に `device` 変数に格納することで実装している。

損失関数は学習したいタスクに応じて適切に設定する必要がある。多クラス分類に用いられる損失関数の項さエントロピーは、PyTorch を用いた場合、`nn.CrossEntropyLoss` クラスにより実装可能である。PyTorch の `nn.CrossEntropyLoss` クラスでは内部で `softmax` 関数が実装されている。そのため、モデルの出力に `softmax` 関数を適用すべきでないことに注意が必要である。

最適化方法を設定するには、学習するモデルのパラメータを与える必要がある。

```
1 device = 'cuda' if torch.cuda.is_available() else 'cpu'
2 model = MyModel() #モデルのインスタンス生成
3 model.to(device)
4 criterion = nn.CrossEntropyLoss() #損失関数(誤差関数)の選択
5 lr = 0.01 #任意の学習率
6 optimizer = optim.SGD(model.parameters(), lr=lr) #最適化方法の選択
```

図 6 ディープニューラルネットワークモデルの生成と最適化方法の設定の例

⑤ ディープニューラルネットワークモデルの学習

ディープニューラルネットワークの学習ループのコード例を図 7 に示す。なお、図 6 に示したモデルの生成をやり直さない限り、このコードによる学習は追加学習となるため注意が必要である。以降は、ディープニューラルネットワークモデルのインスタンスを `model` として作成した場合の説明である。

Python のディープニューラルネットワークモデルには学習用のモードとテスト用のモードが存在する。これは、学習時とテスト時で挙動が異なる定義である層に対応する。`Model.train()`とすることで学習用モードとなり、`model.eval()`とすることでテスト用モードとなる。

順伝播処理である `forward()`メソッドを呼び出すためには `model.forward(・)`と記述すべきだが、特殊なメソッドとして実装されているため単に `model(・)`と記述することでも自動的に `forward()`メソッドが呼び出される仕様である。モデルが GPU デバイス上にある場合、モデルへの入力データも GPU デバイスへ送る必要がある。そのために、`image=images.to("cuda")`のように記述する。

モデルの出力を得た後、出力と教師データを定義した損失関数を定義した `criterion` に入力し、損失を算出する。損失が求まったら `backward()`メソッドにより誤差逆伝播を実行し、ネットワークの勾配を算出する。勾配が求まったら、選択した最適化方法のインスタンスである `optimizer` を用いて `step()`を実行しパラメータの更新、すなわち学習を行うことができる。なお、`backward()`メソッドによる勾配の算出前に、`optimizer` インスタンスの `zero_grad()`により勾配を初期化しておく必要がある。

```
1 n_epochs = 100 #任意の学習回数
2 for epoch in range(n_epochs):
3     model.train()
4     for image, labels in dataloader:
5         images = images.to(device)
6         labels = labels.to(device)
7         outputs = model(images) #ネットワークの forward 処理
8         loss = criterion(outputs, labels) #損失(誤差)計算
9         optimizer.zero_grad() #ネットワークの勾配初期化
10        loss.backward() #逆伝搬してネットワークの勾配を計算
11        optimizer.step() #勾配を基にネットワークのパラメータ更新
```

図 7 ディープニューラルネットワークの学習ループのコードの例

⑥ 学習結果の出力

モデルの学習状況や、テストデータに対する精度を確認する。エポックごとの損失や精度のグラフ化などが一般的に行われる。

2.8 ドロップアウト

ドロップアウトとは、ディープニューラルネットワークの正則化の一つに分類される技術であり、学習時にユニットをランダムに消去することで過学習を抑えられる。PyTorch では `nn.Dropout` クラスや `nn.Dropout2d` クラスなどから利用できる。学習後の推論時にはすべてのユニットを用いるため、学習用モードとテスト用モードで推論の挙動が異なる。

2.9 ResNet

ResNet とは 2015 年に開発された CNN のアーキテクチャである。ディープニューラルネットワークにおいて、層を深くしすぎると浅いネットワークよりも性能が劣る問題を解決するため、残差接続あるいはスキップ接続と呼ばれる ResNet の構造を用いて 100 層を超えるディープニューラルネットワークの学習を可能にした。深いネットワークにおいては勾配消失問題などにより、ネットワーク全体を十分に学習することが困難となるが、スキップ接続を用いることで上層の出力側の誤差を下層の入力側にそのまま伝えられるため、学習が効率的かつ容易となった。ResNet は様々なディープラーニングライブラリに実装されており、PyTorch では torchvision ライブラリを通して、`torchvision.models.resnet50` クラスなどから利用できる。学習済みの ResNet を用いる際には、インスタンス生成時の `weight` 引数を適切に指定する必要がある。

3. 方法

3.1 C3: ライブラリを用いたモデルの構築と学習

- (1) PyTorch を用いた MNIST データの数字分類を行った。
 - ① 必要なライブラリのインポートをした。
 - ② Google Drive のマウントを行った。
 - ③ シードの固定を行うための関数定義を行った。
 - ④ MNIST データを取得した。
 - ⑤ データローダの作成を行った。
 - ⑥ 学習データのミニバッチ一つ分の可視化を行った。
 - ⑦ 学習データの一枚目の可視化を行った。
 - ⑧ 学習のロギング機能および制度検証機能の実装を行った。
 - ⑨ 学習モデルの定義を行った。
 - ⑩ モデルのインスタンス生成、サマリの表示および損失関数、最適化手法の設定を行った。学習をし直す場合には、設定を再度行った。
 - ⑪ モデルの学習をした。
 - ⑫ モデルを保存した。
 - ⑬ 学習の結果を可視化した。
 - ⑭ 学習したモデルで学習データの一枚に対して推論を行った。
 - ⑮ 保存したモデルを読み込み、読み込んだ学習済みモデルで学習データの一枚に対して推論し、結果が変わらないことを確認した。
 - ⑯ MNIST のテストデータの 4096 番目の画像を表示し、その画像に対する推論結果を求めた。
- (2) CNN (ResNet50) を用いた画像分類実験を行う。
 - ① 追加で必要なライブラリのインポートをした。
 - ② CNN (ResNet50) モデルの生成と前処理の設定をした。
 - ③ モデルのサマリを出力した。
 - ④ Google Drive を Colaboratory から参照するためマウントした。
 - ⑤ Moodle 内の犬の画像を Google Drive に保存し、パスを代入した。
 - ⑥ 犬の画像を Tensor に格納し、形状と型を確認した。
 - ⑦ 実装した imshow 関数を用いて犬の画像を表示した。
 - ⑧ 画像の前処理及びバッチ化を行う。バッチ化した後のデータの形状と型を確認した。
 - ⑨ CNN (ResNet50) モデルにより、犬の画像を推論した。推論結果の形状と型を確認した。
 - ⑩ 推論結果を分かりやすいようにデコードした。犬の種類の推論結果を確認した。
 - ⑪ 使用した犬以外の様々な種類の画像に対して推論を行った。結果をまとめ、考察を行った。

3.2 C4:MNIST モデルの最適化

C3 実験で構築した MNIST データの手書き認識モデルのためのモデルを、CNN を用いて学習した。CNN により MNIST データの分類を行い、学習用のデータ量を 100 分の 1 にして MNIST データの分類を行った。さらに、学習データ量を 100 分の 1 の状態で、テストデータに対する精度向上のための最適化を試みた。以下の手順で行った。

- ① 必要なライブラリのインポートを行った。
- ② シードの固定を行うための関数の定義を行った。
- ③ MNIST データを取得した。
- ④ MNIST データの学習データとテストデータにおけるクラスごとのバランスを可視化した。
- ⑤ データローダの作成を行った。
- ⑥ 学習のロギング機能および精度検証機能の実装を行った。
- ⑦ CNN を使った学習モデルの定義を行った。
- ⑧ CNN のインスタンス生成、サマリの表示および損失関数、最適化手法の設定を行った。学習をし直す場合は、この設定を再度実行した。
- ⑨ モデルの学習をする。最終エポックにおける accuracy と loss の値、学習に要した時間を記録した。
- ⑩ モデルを保存した。記録した accuracy と loss の値、学習に要した時間と対応付けられるようにした。
- ⑪ 学習結果を可視化した。保存したモデルと対応付けて保存した。
- ⑫ Colaboratory のランタイムのタイプを GPU に変更し、ランタイムを再起動して①からやり直し、実行時間を計測した。また、GPU の種類を調べた。
- ⑬ ⑧のモデルのインスタンス生成を行う際に、最適化方法を SGD から Adam に変更した。また、⑨~⑪を再度実施して精度や学習結果のグラフの変化を確認した。
- ⑭ 学習に用いるデータを 100 分の 1 にしたサブデータセットを作成した。
- ⑮ サブデータセット用のデータローダを作成した。
- ⑯ サブデータセット用のデータローダを用いて⑦~⑫と同様に CNN モデルの構築、インスタンス生成、サマリの表示および損失関数、最適化手法の設定、学習を行い、精度の変化や学習結果のグラフの変化を確認した。最適化手法の設定は SGD を用いた。
- ⑰ 学習に用いるデータ数を 100 分の 1 にした状態で、テストデータに対する精度を向上させる方法を検討し、CNN モデルの構築、学習を繰り返した。結果をすべて記録し、検討した内容と精度との関係等をまとめ、考察を行った。

3.3 C5:画像認識コンテスト

本実験では、Web カメラなどで取得した画像を利用してじゃんけんの手を三値分類するための学習データセットを作成し、CNN モデルを学習した。実験テキスト通りのベースライン CNN を利用して学習とテストデータに対する分類精度の検証を行った。次に、テストデータに対する分類精度の向上のためハイパーパラメータの調整、学習データセットの増強や CNN モデルの変更を検討した。以下の手順で行った。

- ① 必要なライブラリのインポートを行った。
- ② Google Drive のマウントを行った。
- ③ 実験用ディレクトリ階層 0_グー、1_パー、2_チョキの構築を行った。
- ④ TensorFlow Datasets で利用可能なデータセットの一つである「rock_paper_scissors」をテスト用データとして用いた。
- ⑤ Rock_paper_scissors データセットを PyTorch で扱うためのカスタムデータセットクラスを定義した。
- ⑥ カスタムデータセットクラスを用いて「rock_paper_scissors」テスト用データセットを作成した。
- ⑦ グー、チョキ、パーを学習するためのデータをカメラで撮影した。撮影した画像を Google Drive 内のグー、チョキ、パーそれぞれのディレクトリに格納した。
- ⑧ Datasets.ImageFolder を利用して学習用データを読み込んだ。
- ⑨ 作成した学習用及びテスト用データセットのバランスを可視化した。
- ⑩ 学習用及びテスト用データローダを作成した。batch_size は調整可能なハイパーパラメータであった。
- ⑪ 学習用およびテスト用のデータを確認した。
- ⑫ 学習のロギング機能および精度検証機能の実装を行った。
- ⑬ ベースライン CNN を実装した。
- ⑭ 構築したベースライン CNN モデルのインスタンス生成や損失関数、最適化手法の設定を行った。学習をし直す場合、この設定を再度実行した。
- ⑮ CNN モデルの学習をした。最終エポックにおける accuracy と loss の値、学習に要した時間を記録した。
- ⑯ 学習したモデルは Google Drive 上に保存した。
- ⑰ 学習の結果を可視化した。保存したモデルと対応付けられるように保存した。
- ⑱ テストデータに対する精度を向上させる方法を検討し、手順を繰り返した。
- ⑲ 精度が向上した改善案、低下した改善案について整理し、考察を行った。

4. 予習課題

4.1 C3 実験

4.1.1 確率的勾配降下法および学習率について調べ、説明する。

確率的勾配降下法は、学習データ全体から一部をランダムに選択したミニバッチを用いて損失関数の勾配を計算し、パラメータを更新する方法である。勾配とは損失を最小化するために、パラメータを更

新すべき方向を示す。通常の勾配降下法の場合、勾配計算に時間がかかってしまう他、極大値や極小値のように局所的な最適解に陥りやすい。確率的勾配降下法の場合、確率によるノイズが入ることで局所最適解から抜け出しやすくなっている。

学習率は、ニューラルネットワークにおいてパラメータがどの程度更新されたかを示す値であり、一度の学習でどれだけ学習を得られたかの効果を示す。

4.1.2 pyplot モジュールによる画像の表示を、任意の画像を用いて Python で実行する。

Matplotlib の pyplot モジュールを用いた画像表示のコードおよび実行結果を図 8 に示す。

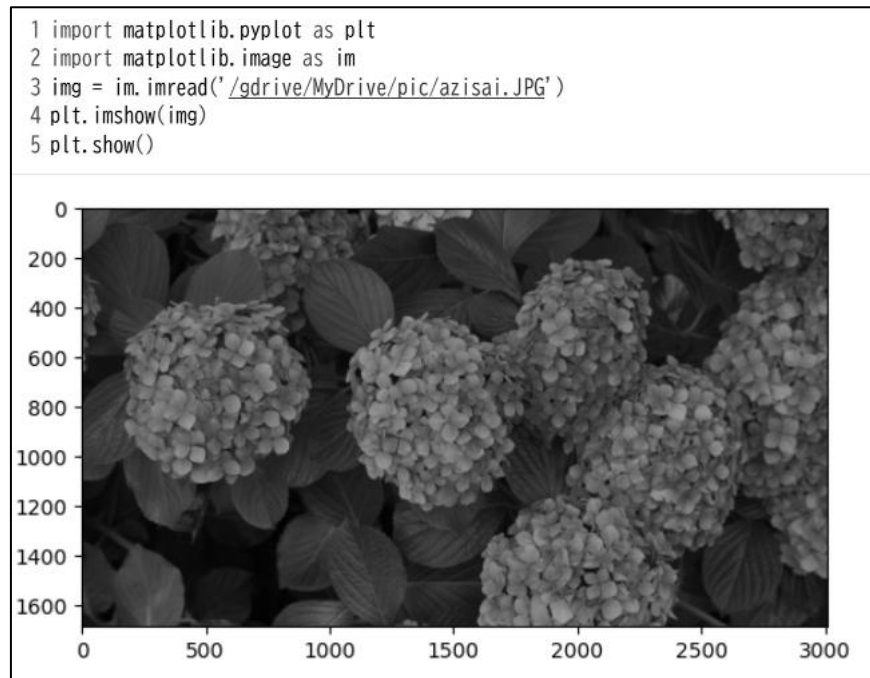


図 8 画像表示のコードおよび実行結果

4.1.3 ResNet50、ImageNet について調べ、説明する。

ResNet50 は、50 層からなる CNN であり、スキップ接続を持つ。スキップ接続により、畳み込み層をスキップして伝播が行われるため、勾配消失の問題が解決された。

ImageNet は、1000 万枚以上の画像データからなる大規模データセットである。画像は 1000 種類のカテゴリに分類されており、各画像には正解ラベルが紐づいている。そのため、モデルの画像識別のための教師付き学習の学習データとして有用である。

4.1.4 PyTorchにおける全結合層の実装の仕方について調べ、説明する。

PyTorchにおける全結合層は、`nn.Linear` クラスを用いて実装される。全結合層はすべてのノードのベクトルと重みを行列演算し、バイアスを加算して出力する。`nn.Linear` は入力数と出力数を定めることで、重みとベクトルを用いた線形変換を行う。`torch.nn` をインポートして `nn.Linear(入力数,出力数)` の形で記述することで、全結合層を実装することができる。モデルの構築では、`__init__` 内で `nn.Linear` を定義し順伝播を行う。

4.1.5 ディープニューラルネットワークで扱うデータを 0~1 に正規化することの利点について調査検討し、説明する。

データの値を 0~1 に収めることで、単位の異なる入力データを用いる場合でも比較が容易である。また、各入力変数の重みを同じバランスで、効率的に更新した学習が行える。正規化せずに学習をする場合、取りえる値の大きい特徴を持つ入力変数がより影響力を持ってしまい、学習が特定のデータへ依存してしまう。入力値が大きい場合勾配の変動幅が大きくなり、数値のオーバーフローや丸め誤差などにより学習が不安定となる。そのため、正規化することで学習が安定し、パラメータも収束しやすくなる。

4.2 C4 実験

4.2.1 6×6 の 2 次元データ入力、3×3 のフィルタに対して畳み込み演算を行った時の出力を求めよ。

ストライドを 1、パディングをなしとして畳み込み演算を行った。入力行列はコード内の 6×6 行列であり、フィルタ行列はコード内の 3×3 行列である。畳み込み演算を行ったコードと実行結果を図 9 に示す。

```
1 import numpy as np
2 inp=np.array([[#6×6の入力行列
3   [100,100,100,0,0,0],
4   [100,100,100,0,0,0],
5   [100,100,100,0,0,0],
6   [100,100,100,0,0,0],
7   [100,100,100,0,0,0],
8   [100,100,100,0,0,0]
9 ]])
10
11 f=np.array([[#3×3のフィルタ行列
12   [0.3,0,-0.3],
13   [0.3,0,-0.3],
14   [0.3,0,-0.3]
15 ]])
16 fh=f.shape[0]
17 fw=f.shape[1]
18 # 出力サイズの計算
19 oh=inp.shape[0]-fh+1
20 ow=inp.shape[1]-fw+1
21 o= np.zeros((oh, ow))
22 # 畳み込み演算
23 for i in range(oh):
24     for j in range(ow):
25         r=inp[i:i+fh, j:j+fw]
26         o[i, j] = np.sum(r * f)
27
28 print(o)
```

```
[[ 0. 90. 90.  0.]
 [ 0. 90. 90.  0.]
 [ 0. 90. 90.  0.]
 [ 0. 90. 90.  0.]
```

図 9 畳み込み演算のコードおよび実行結果

- 4.2.2 図 10 に示す 4×4 の 2 次元データ入力に対して、フィルタの大きさを 2×2 にした場合の Max プーリングの出力を求めよ。

1	8	2	1
4	2	1	1
1	3	1	1
1	2	1	1

図 10 Max プーリングの入力

Max プーリングは指定範囲の最大値を返すフィルタである。今回のフィルタは 2×2 であるため、範囲内の最大値が抽出され、図 12 のような出力となる。

8	2
3	1

図 11 Max プーリングの出力

4.2.3 PyTorch における畳み込み層及びプーリング層の実装の仕方を調べ、説明する。

畳み込み層は、`nn.Conv2d` クラスを用いて実装することができる。`nn.Conv2d(入力数,出力数,フィルタサイズ,ストライド,パディング)` という構造になっている。画像学習の場合、入力数は色の種類によって決まる。白黒のように 2 種類するとき 1、色の諧調が増えると入力数も増えていく。出力数は畳み込み演算後に特徴を抽出した結果の出力の数を決める。出力が多いほど学習できる特徴の数が増える。

Max プーリング層は、`nn.MaxPool2d` クラスもしくは `nn.AvgPool2d` を用いて実装することができる。`nn.MaxPool2d(フィルタサイズ,ストライド,パディング,ダイレーション)` という構造になっている。

4.2.4 CNN の性能や効率を向上させるための学習上の工夫や、CNN モデルの構成について 4 種類以上調べ、説明する。

CNN の性能や効率を向上させるための手法を以下に示す。

(1) Batch Normalization (バッチ正規化)

各層の出力をミニバッチごとに正規化する手法である。正規化により学習の安定化や高速化につながり、過学習の抑制に効果がある。

(2) DataAugmentation (データオーグメンテーション)

入力データに対してランダムな回転などの変換を行い、学習データを拡張する手法である。訓練データの多様性を増やすことで汎化性能が上がり、過学習の抑制に効果がある。

(3) Transfer Learning (転移学習)

大規模なデータセットの事前学習が完了しているモデルを使用し、特定データの追加学習を行う手法である。学習データが少なくとも、高い精度のモデルを実装することができる。

(4) Adam W

Adam の改良版の最適化手法である。重み減衰の防止など効率的なパラメータ更新を行い、過学習の抑制に効果がある。

4.3 C5 実験

4.3.1 汎化性能および過学習について調べ、説明する。

汎化性能とは、学習データに含まれない未知のデータに対して、モデルがどの程度正しく推論、分類ができるかを示す能力である。

過学習とは、訓練データのみで過剰に適合してしまい、未知のデータに対する正しい推論が行えなくなってしまうことである。学習データに対する損失は減少するが、それ以外のデータに対しては損失が上昇していく。

4.3.2 機械学習におけるドメインシフトについて調べ、説明する。

ドメインシフトとは、学習用データとテスト用データの分布が異なる際にモデルの推論の性能が低下してしまうことである。入力の特徴が異なることで、モデルが学習した特徴抽出がテスト用データに対して適合しないために生じる。

5. 使用器具

表 1 に、本実験の使用器具を示す。

表 1 使用器具

名称	製造会社	型式	製造番号	備品番号
PC	ASUS	F94J	TAN0CV06V67842B	—

6. 実験結果

6.1 C3 : ライブラリを用いたモデルの構築と学習

6.1.1 PyTorch を用いた MNIST データの数字分類

必要なライブラリのインポートを行った。コードおよび実行結果を図 12 に示す。

```
1 import torch
2 from torch import nn
3 from torch.nn import functional as F
4 from torch import optim
5 from torch.utils import data
6 import torchvision
7 from torchvision import datasets
8 from torchvision import transforms as T
9 from torchsummary import summary
10 import numpy as np
11 import pandas as pd
12 import matplotlib as mpl
13 from matplotlib import pyplot as plt
14 from tqdm.notebook import tqdm
15 import time
16 import os
17 import math
18 print('pytorch', torch.__version__)
19 print('torchvision', torchvision.__version__)

pytorch 2.10.0+cpu
torchvision 0.25.0+cpu
```

図 12 ライブラリのインポートとインポートしたライブラリのバージョン

GoogleDrive をマウントした。コードと実行結果を図 13 に示す。

<pre> 1 #google Driveのマウント 2 from google.colab import drive #driveモジュールのインポート 3 drive.mount('/gdrive') #/gdrive に接続 ドライブ内のファイルのアクセスが可能になる </pre>
<pre> Drive already mounted at /gdrive; to attempt to forcibly remount, call drive.mount("/gdrive", force_remount=True). </pre>
<pre> 1 #「マウント成功」と表示されたらマウント成功 2 with open('/gdrive/My Drive/foo.txt', 'w') as f: 3 #withを使用するとブロックの最後に自動的にファイルが閉じられる 4 #as f はこのブロック内で、foo.txtをfとして扱えるようにしている 5 f.write('マウント成功') #foo.txt内に書き込み 6 !cat '/gdrive/My Drive/foo.txt' #システムコマンドは!をつける catはファイルの内容を表示したりするコマンド </pre>
マウント成功

図 13 Google Drive のマウント

シードの固定を行うための関数の定義を行った。コードを図 14 に示す。

```

1 def init_seed(seed=0):
2     np.random.seed(seed) #npライブラリシード値の固定
3     torch.manual_seed(seed) #Pytorchのシード値の固定
4     torch.cuda.manual_seed(seed) #PytorchのGPU上のシード値の固定
5     torch.backends.cudnn.deterministic = True #cuDNNのアルゴリズムのランダム性を無くす
6     seed = 3407 #seed値を3407に設定

```

図 14 シード固定の関数

MNIST データの取得を行った。コードと実行結果を図 15 に示す。

<pre> 1 image_size = 28 2 transform = T.Compose(3 [T.Resize((image_size, image_size)), #画像サイズを28x28にリサイズ 4 T.ToTensor()] #Pytorchのテンソルに変換し、値を0.0~1.0にスケールリング 5) 6 7 #MNISTから学習データ、テストデータを取得 8 train_dataset = datasets.MNIST(root='./data', train=True, 9 transform=transform, download = True) 10 test_dataset = datasets.MNIST(root='./data', train=False, 11 transform=transform, download = True) 12 train_samples = len(train_dataset) #学習データのサンプル数 13 classes = [key for key in test_dataset.class_to_idx] #テストデータセットから正解ラベルとクラス名を取り出す 14 print("train dataset:%n",train_dataset,"%n") 15 print("test dataset:%n",test_dataset,"%n") 16 print("dataset classes:%n",classes) </pre>	
<pre> 100% ██████████ 9.91M/9.91M [00:02<00:00, 4.79MB/s] 100% ██████████ 28.9k/28.9k [00:00<00:00, 127kB/s] 100% ██████████ 1.65M/1.65M [00:01<00:00, 1.21MB/s] 100% ██████████ 4.54k/4.54k [00:00<00:00, 11.6MB/s]train dataset:%n Dataset MNIST Number of datapoints: 60000 Root location: ./data Split: Train StandardTransform Transform: Compose(Resize(size=(28, 28), interpolation=bilinear, max_size=None, antialias=True) ToTensor()) %n test dataset:%n Dataset MNIST Number of datapoints: 10000 Root location: ./data Split: Test StandardTransform Transform: Compose(Resize(size=(28, 28), interpolation=bilinear, max_size=None, antialias=True) ToTensor()) %n dataset classes:%n ['0 - zero', '1 - one', '2 - two', '3 - three', '4 - four', '5 - five', '6 - six', '7 - seven', '8 - eight', '9 - nine'] </pre>	

図 15 MNIST データの取得

データローダの作成を行った。コードを図 16 に示す。

```
1 batch_size = 128#1度に処理するデータサイズ
2 init_seed(seed)#シード固定の関数呼び出し
3 #学習データ、テストデータの処理設定
4 train_loader = data.DataLoader(train_dataset, batch_size = batch_size,
5                                 shuffle=True, num_workers = 2)
6 test_loader = data.DataLoader(test_dataset, batch_size=batch_size,
7                                shuffle=False, num_workers = 2)
```

図 16 データローダの作成

学習データのミニバッチ一つ分の可視化を行った。コードを図 17 に、実行結果を図 18 に示す。

```
1 def imshow(img):
2     plt.axis('off')
3     npimg = img.numpy()
4     plt.imshow(np.transpose(npimg, (1, 2, 0)))#matplotlibのためCHWをHWCの配列に置換
5     plt.show()
6
7 dataiter = iter(train_loader)#train_loaderからミニバッチ1回分の学習データを取得
8 images, labels = next(dataiter)
9
10 imshow(torchvision.utils.make_grid(images))
```

図 17 ミニバッチの可視化



図 18 可視化の結果

学習データの一枚目の可視化を行った。コードと実行結果を図 19 に示す。

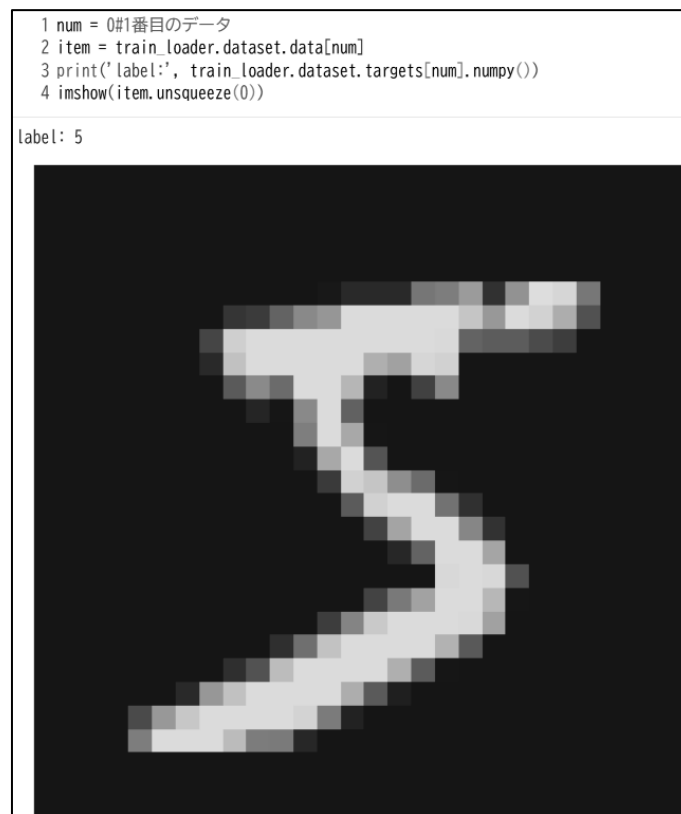


図 19 画像の可視化

学習のロギング機能および精度検証機能の実装を行った。コードを図 20,21 に示す。

```
1 #学習データの推論結果のロギング
2 class Logger():
3     def __init__(self, print_iteration, train_samples, batch_size):
4         self.print_iteration = print_iteration
5         self.iterations = math.ceil(train_samples / batch_size)#学習にかかるループ回数
6         self.log = {'loss': [], 'acc': [], 'epochs': []}
7     def clear_run(self):
8         self.run_loss = 0.0
9         self.run_acc = 0.0
10        self.run_count = 0
11        self.s_time = time.time()
12    def logging(self, outputs, loss, labels, epoch, i):
13        b_size = outputs.size(0)
14        self.run_loss += loss.item()
15        _, predicted = torch.max(outputs.data, 1)
16        correct = (predicted == labels).sum()
17        accuracy = 100 * correct / b_size
18        self.run_acc += accuracy.item()
19        self.run_count += 1
20        if i % self.print_iteration == 0 or i == self.iterations:
21            e_time = (time.time() - self.s_time) / self.run_count
22            progress = round(epoch + i / self.iterations, 2)
23            self.log['loss'].append(self.run_loss / self.run_count)
24            self.log['acc'].append(self.run_acc / self.run_count)
25            self.log['epochs'].append(progress)
26            print('%5d, %9d) loss:%.3f, accuracy:%.2f%%, speed:%.3fs/iter' % (epoch, i, self.log['loss'][-1], self.log['acc'][-1], e_time))
27            self.clear_run()
28    def save(self, path):
29        dict_ = dict(epochs = self.log['epochs'], loss = self.log['loss'], accuracy = self.log['acc'])
30        df = pd.DataFrame(dict_)
31        df.to_csv(path)
```

図 20 学習データの推論のロギング機能

```

33 #テストデータの推論結果のロギング
34 class Evaluator():
35     def __init__(self, dataloader, criterion, device):
36         self.log = {'loss': [], 'acc': [], 'epochs': []}
37         self.loader = dataloader
38         self.device = device
39         self.n_iters = len(self.loader)
40         self.criterion = criterion
41     def evaluation(self, model, epoch):
42         model.eval()
43         correct = 0
44         val_loss = 0.0
45         val_accuracy = 0.0
46         test_samples = 0
47         eval_start = time.time()
48         for images, labels in self.loader:
49             images = images.to(self.device)
50             labels = labels.to(self.device)
51             b_size = images.size(0)
52             test_samples += b_size
53             with torch.no_grad():
54                 outputs = model(images)
55                 loss = self.criterion(outputs, labels)
56                 val_loss += loss.item()
57                 _, predicted = torch.max(outputs.data, 1)
58                 correct += (predicted == labels).sum().item()
59         eval_time = (time.time() - eval_start) / self.n_iters
60         val_accuracy = 100 * correct / test_samples
61         self.log['loss'].append(val_loss / self.n_iters)
62         self.log['acc'].append(val_accuracy)
63         self.log['epochs'].append(epoch)
64         print('[%5d]test loss: %.3f, accuracy: %.2f%%, speed: %.3fs/iter' % (epoch, self.log['loss'][-1], self.log['acc'][-1], eval_time))
65     def save(self, path):
66         dict_ = dict(epochs = self.log['epochs'], loss = self.log['loss'], accuracy = self.log['acc'])
67         df = pd.DataFrame(dict_)
68         df.to_csv(path)

```

図 21 テストデータの推論のロギング機能

学習モデルの定義を行った。コードを図 22 に示す。

```

1 class MyModel(nn.Module):
2     def __init__(self, units):
3         super(MyModel, self).__init__()
4         self.flat = nn.Flatten()
5         self.fc1 = nn.Linear(units[0], units[1])
6         self.fc2 = nn.Linear(units[1], units[2])
7         self.fc3 = nn.Linear(units[2], units[3])
8         self.relu = nn.ReLU()
9     def forward(self, x):
10         h = self.flat(x)#入力データの平坦化
11         h = self.relu(self.fc1(h))#全結合層の後ReLU関数により活性化
12         h = self.relu(self.fc2(h))#全結合層の後ReLU関数により活性化
13         y = self.fc3(h)#全結合層
14         return y
15     def predict(self, x):
16         y = self.forward(x)
17         return F.softmax(y, dim=1)#softmax関数を通して推論結果を出力

```

図 22 学習モデル

モデルのインスタンス生成、サマリの表示、サマリの表示および損失関数、最適化手法の設定を行った。コードと実行結果を図 23 に示す。

```

1 learning_rate = 0.01#学習率
2 init_seed(seed)#seed固定
3 input_size = image_size*image_size
4 units = [input_size, 64, 32, 10]#層の構成 入力層→全結合層→全結合層→出力
5 device = 'cude' if torch.cuda.is_available() else 'cpu'
6 print('use device:', device)
7 model = MyModel(units).to(device)
8 summary(model, (input_size,),device=device)
9 criterion = nn.CrossEntropyLoss()
10 optimizer = optim.SGD(model.parameters(), lr=learning_rate)#最適化手法にSGDを利用
11 print('optimizer',optimizer)

```

use device: cpu

Layer (type)	Output Shape	Param #
Flatten-1	[-1, 784]	0
Linear-2	[-1, 64]	50,240
ReLU-3	[-1, 64]	0
Linear-4	[-1, 32]	2,080
ReLU-5	[-1, 32]	0
Linear-6	[-1, 10]	330

Total params: 52,650
 Trainable params: 52,650
 Non-trainable params: 0

Input size (MB): 0.00
 Forward/backward pass size (MB): 0.01
 Params size (MB): 0.20
 Estimated Total Size (MB): 0.21

```

optimizer SGD (
Parameter Group 0
  dampening: 0
  differentiable: False
  foreach: None
  fused: None
  lr: 0.01
  maximize: False
  momentum: 0
  nesterov: False
  weight_decay: 0
)

```

図 23 モデルのインスタンス生成および出力結果

モデルの学習を行った。コードを図 24 に、実行結果を図 25,26 に示す。ss

```

1 n_epoch=10#エポック数
2 print_iteration=100#任意の値 学習時のロギング間隔
3 logger=Logger(print_iteration,train_samples,batch_size)
4 eval=Evaluator(test_loader,criterion,device)
5 start=time.time()
6 for epoch in tqdm(range(n_epoch)):#エポック数だけ学習を行う
7   eval.evaluation(model,epoch)
8   model.train()
9   logger.clear_run()
10  print_time=time.time()
11  for i,(images,labels) in enumerate(train_loader,0):
12    images=images.to(device)
13    labels=labels.to(device)
14    outputs=model(images)
15    optimizer.zero_grad()
16    loss=criterion(outputs,labels)
17    loss.backward()
18    optimizer.step()
19    logger.logging(outputs,loss,labels,epoch,i+1)
20  eval.evaluation(model,n_epoch)
21  elapsed_time=time.time()-start
22  print('training total',elapsed_time,'sec.')#合計の処理時間

```

図 24 モデルの学習

```

100% ██████████ 10/10 [01:50<00:00, 10.57s/it]
[ 0]test loss: 2.305, accuracy: 13.08%, speed:0.017s/iter
[ 0, 100] loss:2.292, accuracy:15.52%, speed:0.029s/iter
[ 0, 200] loss:2.254, accuracy:20.62%, speed:0.029s/iter
[ 0, 300] loss:2.194, accuracy:28.64%, speed:0.017s/iter
[ 0, 400] loss:2.093, accuracy:37.23%, speed:0.017s/iter
[ 0, 469] loss:1.969, accuracy:40.89%, speed:0.016s/iter
[ 1]test loss: 1.907, accuracy: 40.73%, speed:0.016s/iter
[ 1, 100] loss:1.821, accuracy:44.42%, speed:0.017s/iter
[ 1, 200] loss:1.607, accuracy:57.03%, speed:0.016s/iter
[ 1, 300] loss:1.358, accuracy:67.45%, speed:0.022s/iter
[ 1, 400] loss:1.116, accuracy:72.15%, speed:0.028s/iter
[ 1, 469] loss:0.972, accuracy:75.12%, speed:0.029s/iter
[ 2]test loss: 0.901, accuracy: 77.17%, speed:0.016s/iter
[ 2, 100] loss:0.870, accuracy:76.86%, speed:0.018s/iter
[ 2, 200] loss:0.786, accuracy:78.45%, speed:0.024s/iter
[ 2, 300] loss:0.718, accuracy:80.12%, speed:0.026s/iter
[ 2, 400] loss:0.665, accuracy:81.69%, speed:0.017s/iter
[ 2, 469] loss:0.620, accuracy:83.61%, speed:0.024s/iter
[ 3]test loss: 0.588, accuracy: 83.72%, speed:0.027s/iter
[ 3, 100] loss:0.606, accuracy:83.27%, speed:0.027s/iter
[ 3, 200] loss:0.547, accuracy:84.94%, speed:0.017s/iter
[ 3, 300] loss:0.529, accuracy:85.72%, speed:0.017s/iter
[ 3, 400] loss:0.522, accuracy:85.53%, speed:0.017s/iter
[ 3, 469] loss:0.480, accuracy:86.68%, speed:0.017s/iter
[ 4]test loss: 0.462, accuracy: 87.21%, speed:0.016s/iter
[ 4, 100] loss:0.476, accuracy:87.03%, speed:0.018s/iter
[ 4, 200] loss:0.450, accuracy:87.43%, speed:0.023s/iter
[ 4, 300] loss:0.442, accuracy:88.16%, speed:0.028s/iter
[ 4, 400] loss:0.439, accuracy:87.84%, speed:0.020s/iter
[ 4, 469] loss:0.431, accuracy:88.32%, speed:0.017s/iter
[ 5]test loss: 0.400, accuracy: 88.70%, speed:0.017s/iter
[ 5, 100] loss:0.399, accuracy:89.02%, speed:0.017s/iter
[ 5, 200] loss:0.413, accuracy:88.53%, speed:0.017s/iter
[ 5, 300] loss:0.394, accuracy:89.06%, speed:0.017s/iter
[ 5, 400] loss:0.396, accuracy:88.98%, speed:0.020s/iter
[ 5, 469] loss:0.406, accuracy:88.23%, speed:0.027s/iter

```

図 25 モデルの学習の実行結果前半部分

```

[ 6]test loss: 0.365, accuracy: 89.39%, speed:0.027s/iter
[ 6, 100] loss:0.377, accuracy:89.70%, speed:0.019s/iter
[ 6, 200] loss:0.374, accuracy:89.06%, speed:0.017s/iter
[ 6, 300] loss:0.380, accuracy:89.16%, speed:0.017s/iter
[ 6, 400] loss:0.365, accuracy:89.74%, speed:0.017s/iter
[ 6, 469] loss:0.365, accuracy:89.81%, speed:0.017s/iter
[ 7]test loss: 0.344, accuracy: 90.07%, speed:0.016s/iter
[ 7, 100] loss:0.367, accuracy:89.64%, speed:0.023s/iter
[ 7, 200] loss:0.358, accuracy:89.77%, speed:0.028s/iter
[ 7, 300] loss:0.352, accuracy:89.92%, speed:0.020s/iter
[ 7, 400] loss:0.347, accuracy:90.03%, speed:0.017s/iter
[ 7, 469] loss:0.337, accuracy:90.52%, speed:0.017s/iter
[ 8]test loss: 0.328, accuracy: 90.46%, speed:0.017s/iter
[ 8, 100] loss:0.346, accuracy:90.02%, speed:0.018s/iter
[ 8, 200] loss:0.338, accuracy:90.52%, speed:0.017s/iter
[ 8, 300] loss:0.331, accuracy:90.18%, speed:0.020s/iter
[ 8, 400] loss:0.333, accuracy:90.69%, speed:0.028s/iter
[ 8, 469] loss:0.342, accuracy:90.17%, speed:0.026s/iter
[ 9]test loss: 0.316, accuracy: 90.75%, speed:0.016s/iter
[ 9, 100] loss:0.338, accuracy:90.33%, speed:0.018s/iter
[ 9, 200] loss:0.342, accuracy:90.25%, speed:0.017s/iter
[ 9, 300] loss:0.319, accuracy:90.69%, speed:0.017s/iter
[ 9, 400] loss:0.306, accuracy:91.25%, speed:0.017s/iter
[ 9, 469] loss:0.319, accuracy:90.83%, speed:0.017s/iter
[ 10]test loss: 0.305, accuracy: 91.08%, speed:0.024s/iter
training total 112.0058205127716 sec.

```

図 26 モデルの学習の実行結果後半部分

作成したモデルの保存を行った。コードを図 27 に示す。

```
1 base_fol = '/gdrive/MyDrive/実験' #MyDrive内のモデルを保存したいところにパスを指定する
2 os.makedirs(base_fol, exist_ok = True)
3 torch.save(model.state_dict(), os.path.join(base_fol, 'mnist.pth')) #モデル保存
4 logger.save(os.path.join(base_fol, 'train_loss_acc.csv')) #学習のロギング
5 eval.save(os.path.join(base_fol, 'test_loss_acc.csv')) #推論のロギング
6 with open( os.path.join(base_fol, 'architecture.txt'), mode = "w") as f:
7     f.write('model\n')
8     f.write(str(model))
9     f.write('optimizer\n')
10    f.write(str(optimizer))
```

図 27 モデルの保存

学習結果の可視化をした。コードを図 28 に示す。

```
2 mpl.rcParams['figure.dpi'] = 150
3 #グラフのプロット
4 plt.plot(logger.log['epochs'], logger.log['acc'], marker="o")
5 plt.plot(eval.log['epochs'], eval.log['acc'], marker="o")
6 plt.title('Model accuracy')
7 plt.ylabel('Accuracy[%]')
8 plt.xlabel('Epoch')
9 plt.legend(['Train', 'Test'])
10 plt.show()
11
12 plt.plot(logger.log['epochs'], logger.log['loss'], marker="o")
13 plt.plot(eval.log['epochs'], eval.log['loss'], marker="o")
14 plt.title('Model loss')
15 plt.ylabel('Loss')
16 plt.xlabel('Epoch')
17 plt.legend(['Train', 'Test'])
18 plt.show()
```

図 28 学習結果の可視化

実行結果を図 29,30 に示す。

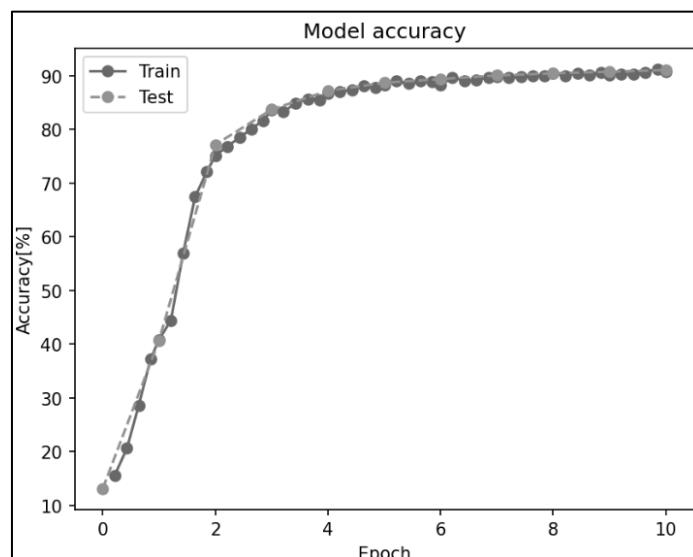


図 29 モデルの精度

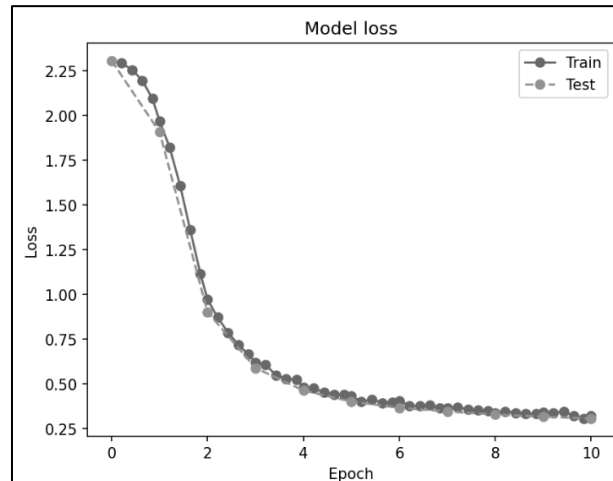


図 30 モデルの損失

学習したモデルで学習データの一枚に対して推論をした。コードおよび実行結果を図 31 に示す。

```

1 # 14
2 num = 1
3 item = train_loader.dataset.data[num].unsqueeze(0)#バッチサイズ1として、学習用データセットからnum+1番目の画像を取り出す
4 print('label:', train_loader.dataset.targets[num].numpy())
5 model.eval()
6 #推論
7 with torch.no_grad():
8     yhat = model.predict(item / 255.)
9 print('predicts:%n', yhat)

```

label: 0
predicts:
tensor([[9.9708e-01, 1.3295e-09, 7.8507e-05, 4.1560e-05, 9.3431e-08, 2.7746e-03,
4.2943e-06, 9.4721e-07, 1.2557e-05, 7.3196e-06]])

図 31 推論および実行結果

保存したモデルを読み込み、読み込んだ学習済みモデルで学習データの一枚に対して推論をした。コードおよび実行結果を図 32 に示す。

```

1 #保存したモデルのロード
2 load_model = MyModel(units)
3 load_model.load_state_dict(
4     torch.load(
5         os.path.join(base_fol, 'mnist.pth')
6     )
7 )
8
9 num = 1
10 item = train_loader.dataset.data[num].unsqueeze(0)
11 print('label:', train_loader.dataset.targets[num].numpy())
12 model.eval()
13 with torch.no_grad():
14     yhat = model.predict(item / 255.)
15 print('predicts:%n', yhat)

```

label: 0
predicts:
tensor([[9.9708e-01, 1.3295e-09, 7.8507e-05, 4.1560e-05, 9.3431e-08, 2.7746e-03,
4.2943e-06, 9.4721e-07, 1.2557e-05, 7.3196e-06]])

図 32 モデルの読み込みと推論および実行結果

図 31,32 より、結果が変わらないことを確認した。

4096 番目の画像を表示した。コードと実行結果を図 33,34 に示す。

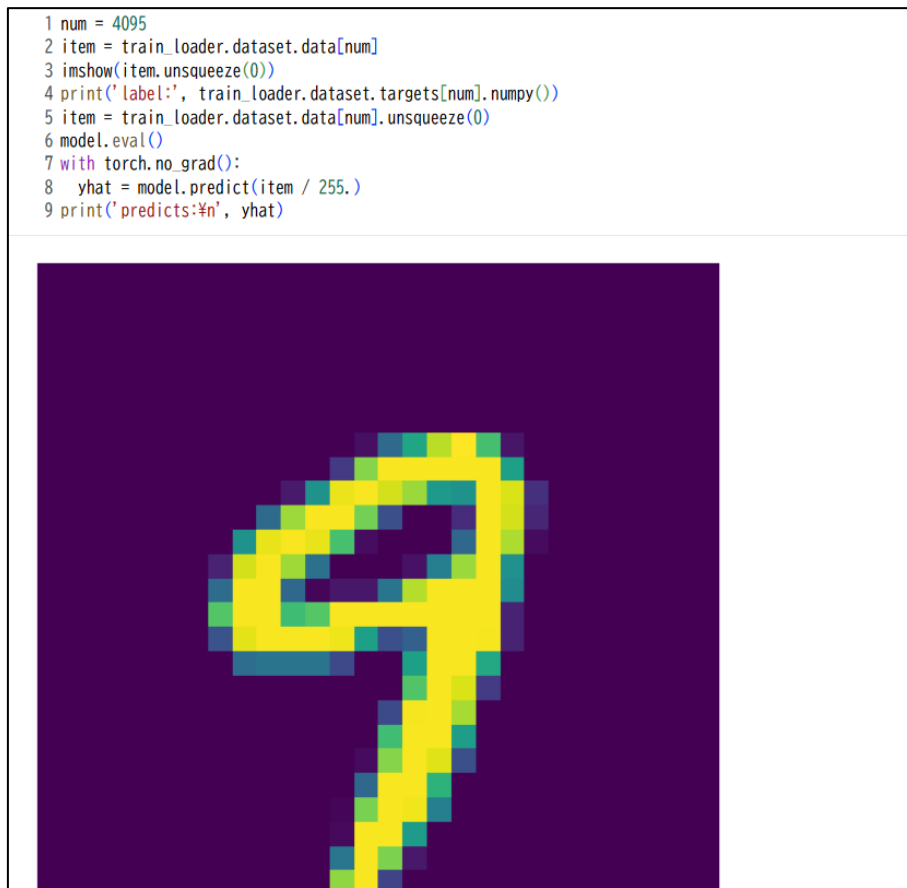


図 33 学生番号目の画像の表示および推論

```
label: 9
predicts:
tensor([[5.2691e-05, 7.8243e-07, 1.0396e-05, 3.4928e-04, 8.4129e-02, 1.7537e-03,
        9.3459e-06, 1.2046e-02, 1.2320e-03, 9.0042e-01]])
```

図 34 画像の推論結果

6.1.2 CNN (ResNet50) を用いた画像の分類実験

追加の必要なライブラリをインポートした。コードを図 35 に示す。

```
21 from torchvision.io import read_image
22 from torchvision.models import resnet50, ResNet50_Weights
```

図 35 追加のライブラリのインポート

ResNet50 のモデルの生成を行った。コードおよび実行結果を図 36 に示す。

```

1 #ResNet50の学習済みの重みを使用
2 weights = ResNet50_Weights.IMAGENET1K_V2
3 resnet = resnet50(weights=weights)
4 resnet.eval()
5 preprocess = weights.transforms()#入力データの前処理
6 print('preprocess:¥n',preprocess)#前処理の中身を出力

preprocess:
ImageClassification(
  crop_size=[224]
  resize_size=[232]
  mean=[0.485, 0.456, 0.406]
  std=[0.229, 0.224, 0.225]
  interpolation=InterpolationMode.BILINEAR
)

```

図 36 モデルの生成および実行結果

モデルのサマリの出力を行った。コードおよび実行結果を図 37,38,39,40,41 に示す。

1 summary(resnet, (3,224,224))		
Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 112, 112]	9,408
BatchNorm2d-2	[-1, 64, 112, 112]	128
ReLU-3	[-1, 64, 112, 112]	0
MaxPool2d-4	[-1, 64, 56, 56]	0
Conv2d-5	[-1, 64, 56, 56]	4,096
BatchNorm2d-6	[-1, 64, 56, 56]	128
ReLU-7	[-1, 64, 56, 56]	0
Conv2d-8	[-1, 64, 56, 56]	36,864
BatchNorm2d-9	[-1, 64, 56, 56]	128
ReLU-10	[-1, 64, 56, 56]	0
Conv2d-11	[-1, 256, 56, 56]	16,384
BatchNorm2d-12	[-1, 256, 56, 56]	512
Conv2d-13	[-1, 256, 56, 56]	16,384
BatchNorm2d-14	[-1, 256, 56, 56]	512
ReLU-15	[-1, 256, 56, 56]	0
Bottleneck-16	[-1, 256, 56, 56]	0
Conv2d-17	[-1, 64, 56, 56]	16,384
BatchNorm2d-18	[-1, 64, 56, 56]	128
ReLU-19	[-1, 64, 56, 56]	0
Conv2d-20	[-1, 64, 56, 56]	36,864
BatchNorm2d-21	[-1, 64, 56, 56]	128
ReLU-22	[-1, 64, 56, 56]	0
Conv2d-23	[-1, 256, 56, 56]	16,384
BatchNorm2d-24	[-1, 256, 56, 56]	512
ReLU-25	[-1, 256, 56, 56]	0
Bottleneck-26	[-1, 256, 56, 56]	0
Conv2d-27	[-1, 64, 56, 56]	16,384
BatchNorm2d-28	[-1, 64, 56, 56]	128
ReLU-29	[-1, 64, 56, 56]	0
Conv2d-30	[-1, 64, 56, 56]	36,864
BatchNorm2d-31	[-1, 64, 56, 56]	128
ReLU-32	[-1, 64, 56, 56]	0
Conv2d-33	[-1, 256, 56, 56]	16,384
BatchNorm2d-34	[-1, 256, 56, 56]	512

図 37 モデルのサマリの出力および実行結果 1

ReLU-25	[-1, 256, 56, 56]	0
Bottleneck-26	[-1, 256, 56, 56]	0
Conv2d-27	[-1, 64, 56, 56]	16,384
BatchNorm2d-28	[-1, 64, 56, 56]	128
ReLU-29	[-1, 64, 56, 56]	0
Conv2d-30	[-1, 64, 56, 56]	36,864
BatchNorm2d-31	[-1, 64, 56, 56]	128
ReLU-32	[-1, 64, 56, 56]	0
Conv2d-33	[-1, 256, 56, 56]	16,384
BatchNorm2d-34	[-1, 256, 56, 56]	512
ReLU-35	[-1, 256, 56, 56]	0
Bottleneck-36	[-1, 256, 56, 56]	0
Conv2d-37	[-1, 128, 56, 56]	32,768
BatchNorm2d-38	[-1, 128, 56, 56]	256
ReLU-39	[-1, 128, 56, 56]	0
Conv2d-40	[-1, 128, 28, 28]	147,456
BatchNorm2d-41	[-1, 128, 28, 28]	256
ReLU-42	[-1, 128, 28, 28]	0
Conv2d-43	[-1, 512, 28, 28]	65,536
BatchNorm2d-44	[-1, 512, 28, 28]	1,024
Conv2d-45	[-1, 512, 28, 28]	131,072
BatchNorm2d-46	[-1, 512, 28, 28]	1,024
ReLU-47	[-1, 512, 28, 28]	0
Bottleneck-48	[-1, 512, 28, 28]	0
Conv2d-49	[-1, 128, 28, 28]	65,536
BatchNorm2d-50	[-1, 128, 28, 28]	256
ReLU-51	[-1, 128, 28, 28]	0
Conv2d-52	[-1, 128, 28, 28]	147,456
BatchNorm2d-53	[-1, 128, 28, 28]	256
ReLU-54	[-1, 128, 28, 28]	0
Conv2d-55	[-1, 512, 28, 28]	65,536
BatchNorm2d-56	[-1, 512, 28, 28]	1,024
ReLU-57	[-1, 512, 28, 28]	0
Bottleneck-58	[-1, 512, 28, 28]	0
Conv2d-59	[-1, 128, 28, 28]	65,536
BatchNorm2d-60	[-1, 128, 28, 28]	256
ReLU-61	[-1, 128, 28, 28]	0
Conv2d-62	[-1, 128, 28, 28]	147,456

図 38 実行結果 2

BatchNorm2d-73	[-1, 128, 28, 28]	256
ReLU-74	[-1, 128, 28, 28]	0
Conv2d-75	[-1, 512, 28, 28]	65,536
BatchNorm2d-76	[-1, 512, 28, 28]	1,024
ReLU-77	[-1, 512, 28, 28]	0
Bottleneck-78	[-1, 512, 28, 28]	0
Conv2d-79	[-1, 256, 28, 28]	131,072
BatchNorm2d-80	[-1, 256, 28, 28]	512
ReLU-81	[-1, 256, 28, 28]	0
Conv2d-82	[-1, 256, 14, 14]	589,824
BatchNorm2d-83	[-1, 256, 14, 14]	512
ReLU-84	[-1, 256, 14, 14]	0
Conv2d-85	[-1, 1024, 14, 14]	262,144
BatchNorm2d-86	[-1, 1024, 14, 14]	2,048
Conv2d-87	[-1, 1024, 14, 14]	524,288
BatchNorm2d-88	[-1, 1024, 14, 14]	2,048
ReLU-89	[-1, 1024, 14, 14]	0
Bottleneck-90	[-1, 1024, 14, 14]	0
Conv2d-91	[-1, 256, 14, 14]	262,144
BatchNorm2d-92	[-1, 256, 14, 14]	512
ReLU-93	[-1, 256, 14, 14]	0
Conv2d-94	[-1, 256, 14, 14]	589,824
BatchNorm2d-95	[-1, 256, 14, 14]	512
ReLU-96	[-1, 256, 14, 14]	0
Conv2d-97	[-1, 1024, 14, 14]	262,144
BatchNorm2d-98	[-1, 1024, 14, 14]	2,048
ReLU-99	[-1, 1024, 14, 14]	0
Bottleneck-100	[-1, 1024, 14, 14]	0
Conv2d-101	[-1, 256, 14, 14]	262,144
BatchNorm2d-102	[-1, 256, 14, 14]	512
ReLU-103	[-1, 256, 14, 14]	0
Conv2d-104	[-1, 256, 14, 14]	589,824
BatchNorm2d-105	[-1, 256, 14, 14]	512
ReLU-106	[-1, 256, 14, 14]	0
Conv2d-107	[-1, 1024, 14, 14]	262,144
BatchNorm2d-108	[-1, 1024, 14, 14]	2,048
ReLU-109	[-1, 1024, 14, 14]	0
Bottleneck-110	[-1, 1024, 14, 14]	0
Conv2d-111	[-1, 256, 14, 14]	262,144

図 39 実行結果 3

BatchNorm2d-108	[-1, 1024, 14, 14]	2,048
ReLU-109	[-1, 1024, 14, 14]	0
Bottleneck-110	[-1, 1024, 14, 14]	0
Conv2d-111	[-1, 256, 14, 14]	262,144
BatchNorm2d-112	[-1, 256, 14, 14]	512
ReLU-113	[-1, 256, 14, 14]	0
Conv2d-114	[-1, 256, 14, 14]	589,824
BatchNorm2d-115	[-1, 256, 14, 14]	512
ReLU-116	[-1, 256, 14, 14]	0
Conv2d-117	[-1, 1024, 14, 14]	262,144
BatchNorm2d-118	[-1, 1024, 14, 14]	2,048
ReLU-119	[-1, 1024, 14, 14]	0
Bottleneck-120	[-1, 1024, 14, 14]	0
Conv2d-121	[-1, 256, 14, 14]	262,144
BatchNorm2d-122	[-1, 256, 14, 14]	512
ReLU-123	[-1, 256, 14, 14]	0
Conv2d-124	[-1, 256, 14, 14]	589,824
BatchNorm2d-125	[-1, 256, 14, 14]	512
ReLU-126	[-1, 256, 14, 14]	0
Conv2d-127	[-1, 1024, 14, 14]	262,144
BatchNorm2d-128	[-1, 1024, 14, 14]	2,048
ReLU-129	[-1, 1024, 14, 14]	0
Bottleneck-130	[-1, 1024, 14, 14]	0
Conv2d-131	[-1, 256, 14, 14]	262,144
BatchNorm2d-132	[-1, 256, 14, 14]	512
ReLU-133	[-1, 256, 14, 14]	0
Conv2d-134	[-1, 256, 14, 14]	589,824
BatchNorm2d-135	[-1, 256, 14, 14]	512
ReLU-136	[-1, 256, 14, 14]	0
Conv2d-137	[-1, 1024, 14, 14]	262,144
BatchNorm2d-138	[-1, 1024, 14, 14]	2,048
ReLU-139	[-1, 1024, 14, 14]	0
Bottleneck-140	[-1, 1024, 14, 14]	0
Conv2d-141	[-1, 512, 14, 14]	524,288
BatchNorm2d-142	[-1, 512, 14, 14]	1,024
ReLU-143	[-1, 512, 14, 14]	0
Conv2d-144	[-1, 512, 7, 7]	2,359,296
BatchNorm2d-145	[-1, 512, 7, 7]	1,024
ReLU-146	[-1, 512, 7, 7]	0

図 40 実行結果 4

Conv2d-147	[-1, 2048, 7, 7]	1,048,576
BatchNorm2d-148	[-1, 2048, 7, 7]	4,096
Conv2d-149	[-1, 2048, 7, 7]	2,097,152
BatchNorm2d-150	[-1, 2048, 7, 7]	4,096
ReLU-151	[-1, 2048, 7, 7]	0
Bottleneck-152	[-1, 2048, 7, 7]	0
Conv2d-153	[-1, 512, 7, 7]	1,048,576
BatchNorm2d-154	[-1, 512, 7, 7]	1,024
ReLU-155	[-1, 512, 7, 7]	0
Conv2d-156	[-1, 512, 7, 7]	2,359,296
BatchNorm2d-157	[-1, 512, 7, 7]	1,024
ReLU-158	[-1, 512, 7, 7]	0
Conv2d-159	[-1, 2048, 7, 7]	1,048,576
BatchNorm2d-160	[-1, 2048, 7, 7]	4,096
ReLU-161	[-1, 2048, 7, 7]	0
Bottleneck-162	[-1, 2048, 7, 7]	0
Conv2d-163	[-1, 512, 7, 7]	1,048,576
BatchNorm2d-164	[-1, 512, 7, 7]	1,024
ReLU-165	[-1, 512, 7, 7]	0
Conv2d-166	[-1, 512, 7, 7]	2,359,296
BatchNorm2d-167	[-1, 512, 7, 7]	1,024
ReLU-168	[-1, 512, 7, 7]	0
Conv2d-169	[-1, 2048, 7, 7]	1,048,576
BatchNorm2d-170	[-1, 2048, 7, 7]	4,096
ReLU-171	[-1, 2048, 7, 7]	0
Bottleneck-172	[-1, 2048, 7, 7]	0
AdaptiveAvgPool2d-173	[-1, 2048, 1, 1]	0
Linear-174	[-1, 1000]	2,049,000
=====		
Total params: 25,557,032		
Trainable params: 25,557,032		
Non-trainable params: 0		

Input size (MB): 0.57		
Forward/backward pass size (MB): 286.56		
Params size (MB): 97.49		
Estimated Total Size (MB): 384.62		

図 41 実行結果 5

犬の画像を GoogleDrive に保存し、image_path に代入した。コードを図 42 に示す。

```
1 image_path= '/gdrive/MyDrive/pic/chihuahua1.jpg' #保存した犬の画像のパス
```

図 42 犬の画像の保存

犬の画像を Tensor に格納し、形状と型を確認した。コードと実行結果を図 43 に示す。

```
1 image = read_image(image_path)
2 print(image.shape)
3 print(' image type', type(image))

torch.Size([3, 3456, 5184])
image type <class 'torch.Tensor'>
```

図 43 Tensor への格納

Imshow 関数を定義した。コードを図 44 に示す。

```
1 #画像表示の関数定義
2 def imshow(img):
3     plt.axis('off')
4     npimg = img.numpy()
5     plt.imshow(np.transpose(npimg, (1, 2, 0)))
6     plt.show()
```

図 44 画像表示の関数

図 44 で定義した imshow 関数を用いて犬の画像を表示した。コードと実行結果を図 45 に示す。



図 45 画像の表示

88

画像の前処理およびバッチ化を行い、バッチ化した後のデータの形状と型を確認した。コードおよび実行結果を図 46 に示す。

```

1 batch = preprocess(image).unsqueeze(0)
2 print(batch.shape)
3 print('batch.type', type(batch))

torch.Size([1, 3, 224, 224])
batch.type <class 'torch.Tensor'>

```

図 46 前処理およびバッチ化

ResNet の CNN モデルにより、犬の画像を推論し、推論結果の形状と型を確認した。コードおよび実行結果を図 47 に示す。

```

1 #勾配を計算せず推論を行う
2 with torch.no_grad():
3     prediction = resnet(batch).squeeze(0).softmax(0)
4     print(prediction.shape)
5     print('output type', type(prediction))

torch.Size([1000])
output type <class 'torch.Tensor'>

```

図 47 ResNet50 による画像の推論

推論結果をデコードした。コードと実行結果を図 48 に示す。

```

1 k = 5
2 topK = torch.topk(prediction, k) #上位k個の推論結果を表示
3 print('predict:')
4 for i in range(k):
5     print(f'{weights.meta['categories'][topK.indices[i]]}:{100 * topK.values[i]:.1f}%')

predict:
Chihuahua:29.2%
Pomeranian:13.7%
papillon:1.6%
toy terrier:0.6%
keeshond:0.5%

```

図 48 k=5 とした推論結果のデコード

図 48 より、推論結果はチワワとなっていた。

図 48 の k の値を 1 にしてデコードした。コードと実行結果を図 49 に示す。

```

1 k = 1
2 topK = torch.topk(prediction, k)
3 print('predict:')
4 for i in range(k):
5     print(f'{weights.meta['categories'][topK.indices[i]]}:{100 * topK.values[i]:.1f}%')

predict:
Chihuahua:29.2%

```

図 49 k=1 とした推論結果のデコード

k の値を 10 にしてデコードした。コードと実行結果を図 50 に示す。

```
1 k = 10
2 topK = torch.topk(prediction, k)
3 print('predict:')
4 for i in range(k):
5     print(f"{weights.meta['categories'][topK.indices[i]]}:{100 * topK.values[i]:.1f}%")
```

```
predict:
Chihuahua:29.2%
Pomeranian:13.7%
papillon:1.6%
toy terrier:0.6%
keeshond:0.5%
Japanese spaniel:0.4%
toy poodle:0.2%
tennis ball:0.2%
dishwasher:0.2%
Norwich terrier:0.2%
```

図 50 k=10 とした推論結果のデコード

使用した犬以外のさまざまな種類の画像に対して推論を行った。一つ目の画像を図 51 に示す。



図 51 推論に使用した画像 1

推論結果を図 52 に示す。

<pre> 1 k = 5 2 topK = torch.topk(prediction, k)#上位k個の推論結果を表示 3 print('predict:') 4 for i in range(k): 5 print(f"{weights.meta['categories'][topK.indices[i]]}:{100 * topK.values[i]:.1f}%") </pre>	
<pre> predict: rapeseed:37.3% hay:4.8% lakeside:2.0% golf ball:1.8% barn:1.6% </pre>	

図 52 画像 1 の推論結果

二つ目の画像を図 53 に示す。



図 53 推論に使用した画像 2

推論結果を図 54 に示す。

<pre>1 k = 5 2 topK = torch.topk(prediction, k)#上位k個の推論結果を表示 3 print('predict:') 4 for i in range(k): 5 print(f"{weights.meta['categories'][topK.indices[i]]}:{100 * topK.values[i]:.1f}%")</pre>	
<pre>predict: Eskimo dog:35.4% Siberian husky:4.8% Norwegian elkhound:2.1% dingo:1.4% basenji:0.6%</pre>	

図 54 画像 2 の推論結果

三つ目の画像を図 55 に示す。



図 55 使用した画像 3

推論結果を図 56 に示す。

<pre>1 k = 5 2 topK = torch.topk(prediction, k)#上位k個の推論結果を表示 3 print('predict:') 4 for i in range(k): 5 print(f"{weights.meta['categories'][topK.indices[i]]}:{100 * topK.values[i]:.1f}%")</pre>	
<pre>predict: Pomeranian:34.4% keeshond:0.6% Pekinese:0.3% papillon:0.2% toy poodle:0.2%</pre>	

図 56 画像 3 の推論結果

6.2 C4:MNIST モデルの最適化

必要なライブラリのインポート、シード固定関数の定義、MNIST データの取得を行った。

その後、MNIST データの学習データとテストデータにおけるクラスごとのバランスを可視化した。

コードおよび実行結果を図 57,58,59 に示す。

```
1 train_df = pd.DataFrame(train_dataset.targets)
2 balance = train_df.value_counts()#学習用データセットの正解ラベル数の集計
3 display('train', balance)
4 test_df = pd.DataFrame(test_dataset.targets)
5 balance = test_df.value_counts()#テスト用データセットの正解ラベル数の集計
6 display('test', balance)
```

図 57 MNIST データのバランスの可視化

```
'train'
count
0
1 6742
7 6265
3 6131
2 5958
9 5949
0 5923
6 5918
8 5851
4 5842
5 5421
dtype: int64
```

図 58 学習用データセットの可視化の出力結果

```
'test'
count
0
1 1135
2 1032
7 1028
3 1010
9 1009
4 982
0 980
8 974
6 958
5 892
dtype: int64
```

図 59 テスト用データセットの可視化の出力結果

データローダの作成、学習のロギング機能および精度検証機能の実装を行った。

その後、CNN を使った学習モデルの定義を行った。コードと実行結果を図 58 に示す。

```
1 class MyCNN(nn.Module):
2     def __init__(self):
3         super(MyCNN, self).__init__()
4         self.conv1 = nn.Conv2d(1, 50, (5, 5))
5         self.conv2 = nn.Conv2d(50, 100, (3, 3), padding=1)
6         self.fc1 = nn.Linear(6*6*100, 100)
7         #self.fc1 = nn.Linear(12*12*50, 100)
8         self.fc2 = nn.Linear(100, 10)
9         self.pool = nn.MaxPool2d((2,2))
10        self.flat = nn.Flatten()
11        self.drop = nn.Dropout(0.5)
12        self.relu = nn.ReLU()
13    def forward(self, x):
14        h = self.relu(self.conv1(x))#畳み込み層の後ReLU関数により活性化
15        h = self.pool(h)#Maxプーリング層
16        h = self.relu(self.conv2(h))#畳み込み層の後ReLU関数により活性化3.2 ⑦精度向上のために追加
17        h = self.pool(h)#Maxプーリング層 3.2 ⑦精度向上のために追加
18        h = self.flat(h)#平坦化
19        h = self.relu(self.fc1(h))#全結合層の後ReLU関数により活性化
20        h = self.drop(h)#ドロップアウト 3.2 ⑦精度向上のために追加
21        y = self.fc2(h)#全結合層の後ReLU関数により活性化
22        return y
23    def predict(self, x):
24        y = self.forward(x)
25        return F.softmax(y, dim=1)#softmax関数を通して推論結果を出力
```

図 60 CNN モデルの定義

CNN のインスタンス生成、サマリの表示および損失関数、最適化手法の設定をした。コードおよび実行結果を図 61,62 に示す。

```
1 learning_rate = 0.01
2 init_seed(seed) #seedの値は任意に設定する
3 device = 'cuda' if torch.cuda.is_available() else 'cpu'
4 print('use device:', device)
5 model = MyCNN().to(device)
6 # image_size変数をtransform作成時に予め定義しておく
7 summary(model, (1, image_size, image_size),device=device)
8 criterion = nn.CrossEntropyLoss()交差エントロピー誤差の計算
9 optimizer = optim.SGD(model.parameters(), lr=learning_rate) #最適化手法SDG 3.2 ③の時はコメントアウトしておく
10 #optimizer = optim.Adam(model.parameters(), lr=learning_rate) #最適化手法Adam 3.2 ③の時だけ使用する
11 print('optimizer',optimizer)
```

図 61 CNN インスタンス生成、サマリの表示および損失関数、最適化手法の設定

```
use device: cpu
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 50, 24, 24]	1,300
ReLU-2	[-1, 50, 24, 24]	0
MaxPool2d-3	[-1, 50, 12, 12]	0
Flatten-4	[-1, 7200]	0
Linear-5	[-1, 100]	720,100
ReLU-6	[-1, 100]	0
Linear-7	[-1, 10]	1,010

```

Total params: 722,410
Trainable params: 722,410
Non-trainable params: 0

Input size (MB): 0.00
Forward/backward pass size (MB): 0.55
Params size (MB): 2.76
Estimated Total Size (MB): 3.31

optimizer SGD (
Parameter Group 0
  dampening: 0
  differentiable: False
  foreach: None
  fused: None
  lr: 0.01
  maximize: False
  momentum: 0
  nesterov: False
  weight_decay: 0
)

```

図 62 モデルのサマリの出力結果

モデルの学習を行った。コードおよび実行結果を図 63,64,65 に示す。

```

1 n_epoch=10#エポック数
2 print_iteration=100#学習時のロギング間隔
3 logger=Logger(print_iteration,train_samples,batch_size)
4 eval=Evaluator(test_loader,criterion,device)
5 start=time.time()
6 for epoch in tqdm(range(n_epoch)):#エポック数だけ学習を行う。
7     eval.evaluation(model,epoch)
8     model.train()
9     logger.clear_run()
10    print_time=time.time()
11    for i,(images,labels) in enumerate(train_loader,0):
12        images=images.to(device)
13        labels=labels.to(device)
14        outputs=model(images)
15        optimizer.zero_grad()
16        loss=criterion(outputs,labels)
17        loss.backward()
18        optimizer.step()
19        logger.logging(outputs,loss,labels,epoch,i+1)
20 #3.2 ⑦精度向上のために追加
21 """
22 if epoch == 10:
23     learning_rate = 0.001
24 if epoch == 15:
25     learning_rate = 0.0001
26 """
27 eval.evaluation(model,n_epoch)
28 elapsed_time=time.time()-start
29 print('training total',elapsed_time,'sec.')
```

#合計の処理時間

```

30
31 #データの記録
32 finalaccuracy = eval.log["acc"][-1] #accuracy
33 finalloss = eval.log["loss"][-1] # loss

```

図 63 CNN モデルの学習


```

100% ██████████ 10/10 [10:49<00:00, 64.34s/it]
[ 0]test loss: 2.316, accuracy: 5.43%, speed:0.054s/iter
[ 0, 100] loss:1.913, accuracy:59.59%, speed:0.130s/iter
[ 0, 200] loss:1.035, accuracy:80.72%, speed:0.128s/iter
[ 0, 300] loss:0.617, accuracy:85.61%, speed:0.127s/iter
[ 0, 400] loss:0.476, accuracy:87.88%, speed:0.128s/iter
[ 0, 469] loss:0.417, accuracy:88.59%, speed:0.131s/iter
[ 1]test loss: 0.384, accuracy: 89.61%, speed:0.050s/iter
[ 1, 100] loss:0.383, accuracy:89.30%, speed:0.131s/iter
[ 1, 200] loss:0.355, accuracy:89.80%, speed:0.159s/iter
[ 1, 300] loss:0.345, accuracy:90.24%, speed:0.126s/iter
[ 1, 400] loss:0.335, accuracy:90.48%, speed:0.126s/iter
[ 1, 469] loss:0.336, accuracy:90.26%, speed:0.131s/iter
[ 2]test loss: 0.295, accuracy: 91.47%, speed:0.050s/iter
[ 2, 100] loss:0.314, accuracy:90.91%, speed:0.128s/iter
[ 2, 200] loss:0.304, accuracy:90.85%, speed:0.127s/iter
[ 2, 300] loss:0.288, accuracy:91.80%, speed:0.127s/iter
[ 2, 400] loss:0.290, accuracy:91.63%, speed:0.134s/iter
[ 2, 469] loss:0.286, accuracy:91.89%, speed:0.138s/iter
[ 3]test loss: 0.262, accuracy: 92.34%, speed:0.053s/iter
[ 3, 100] loss:0.285, accuracy:91.82%, speed:0.133s/iter
[ 3, 200] loss:0.265, accuracy:92.16%, speed:0.132s/iter
[ 3, 300] loss:0.265, accuracy:92.55%, speed:0.134s/iter
[ 3, 400] loss:0.255, accuracy:92.56%, speed:0.132s/iter
[ 3, 469] loss:0.240, accuracy:92.78%, speed:0.135s/iter
[ 4]test loss: 0.234, accuracy: 93.30%, speed:0.051s/iter
[ 4, 100] loss:0.241, accuracy:92.88%, speed:0.130s/iter
[ 4, 200] loss:0.237, accuracy:93.19%, speed:0.129s/iter
[ 4, 300] loss:0.243, accuracy:92.94%, speed:0.129s/iter
[ 4, 400] loss:0.230, accuracy:93.55%, speed:0.128s/iter
[ 4, 469] loss:0.224, accuracy:93.33%, speed:0.131s/iter
[ 5]test loss: 0.212, accuracy: 93.88%, speed:0.051s/iter
[ 5, 100] loss:0.224, accuracy:93.55%, speed:0.129s/iter
[ 5, 200] loss:0.216, accuracy:93.78%, speed:0.128s/iter
[ 5, 300] loss:0.215, accuracy:93.97%, speed:0.128s/iter
[ 5, 400] loss:0.200, accuracy:94.19%, speed:0.129s/iter
[ 5, 469] loss:0.204, accuracy:94.24%, speed:0.132s/iter

```

図 64 CNN モデルの学習の出力結果前半部分

```

[ 6]test loss: 0.188, accuracy: 94.46%, speed:0.050s/iter
[ 6, 100] loss:0.193, accuracy:94.41%, speed:0.128s/iter
[ 6, 200] loss:0.200, accuracy:94.22%, speed:0.128s/iter
[ 6, 300] loss:0.198, accuracy:94.34%, speed:0.128s/iter
[ 6, 400] loss:0.184, accuracy:94.54%, speed:0.127s/iter
[ 6, 469] loss:0.178, accuracy:94.93%, speed:0.131s/iter
[ 7]test loss: 0.173, accuracy: 94.84%, speed:0.050s/iter
[ 7, 100] loss:0.189, accuracy:94.52%, speed:0.129s/iter
[ 7, 200] loss:0.171, accuracy:95.28%, speed:0.127s/iter
[ 7, 300] loss:0.165, accuracy:95.16%, speed:0.128s/iter
[ 7, 400] loss:0.170, accuracy:94.95%, speed:0.129s/iter
[ 7, 469] loss:0.172, accuracy:94.98%, speed:0.132s/iter
[ 8]test loss: 0.158, accuracy: 95.48%, speed:0.054s/iter
[ 8, 100] loss:0.165, accuracy:95.09%, speed:0.133s/iter
[ 8, 200] loss:0.164, accuracy:95.41%, speed:0.129s/iter
[ 8, 300] loss:0.148, accuracy:95.90%, speed:0.128s/iter
[ 8, 400] loss:0.159, accuracy:95.32%, speed:0.128s/iter
[ 8, 469] loss:0.149, accuracy:95.72%, speed:0.132s/iter
[ 9]test loss: 0.143, accuracy: 95.84%, speed:0.050s/iter
[ 9, 100] loss:0.143, accuracy:95.88%, speed:0.129s/iter
[ 9, 200] loss:0.145, accuracy:95.71%, speed:0.122s/iter
[ 9, 300] loss:0.140, accuracy:96.14%, speed:0.128s/iter
[ 9, 400] loss:0.153, accuracy:95.62%, speed:0.127s/iter
[ 9, 469] loss:0.139, accuracy:96.19%, speed:0.126s/iter
[10]test loss: 0.131, accuracy: 96.32%, speed:0.055s/iter
training total 654.1547565460205 sec.

```

図 65 CNN モデルの学習の出力結果後半部分

モデルを保存した。コードを図 66 に示す。

```
1 base_fol_prefix = '/content/drive/MyDrive/C-4' #MyDrive内のモデルを保存したいところにパスを指定する
2 folder_name = f'Acc{finalaccuracy:.2f}_Loss{finalloss:.3f}_Time{elapsed_time:.0f}s' #accuracy、loss、時間の名前を付けてフォルダを作成
3 base_fol = os.path.join(base_fol_prefix, folder_name) #作成したフォルダ内にモデルを収納
4 os.makedirs(base_fol, exist_ok = True)
5 torch.save(model.state_dict(), os.path.join(base_fol, 'mnist.pth'))
6 logger.save(os.path.join(base_fol, 'train_loss_acc.csv'))
7 eval.save(os.path.join(base_fol, 'test_loss_acc.csv'))
8 #テキストファイルにモデルと最適化手法を書き込んで保存
9 with open( os.path.join(base_fol, 'architecture.txt'), mode = "w") as f:
10     f.write('model\n')
11     f.write(str(model))
12     f.write('optimizer\n')
13     f.write(str(optimizer))
```

図 66 CNN モデルの保存

学習の結果を可視化した。コードおよび実行結果を図 67,68,69 に示す。

```
1 mpl.rcParams['figure.dpi'] = 150
2 #モデルのエポックと精度のグラフのプロット
3 plt.plot(logger.log['epochs'], logger.log['acc'], marker="o", linestyle='--')
4 plt.plot(eval.log['epochs'], eval.log['acc'], marker="o")
5 plt.title('Model accuracy')
6 plt.ylabel('Accuracy[%]')
7 plt.xlabel('Epoch')
8 plt.legend(['Train', 'Test'])
9 plt.show()
10 plt.savefig(os.path.join(base_fol, 'accuracy_plot.png'))
11 #モデルのエポックと損失のグラフのプロット
12 plt.plot(logger.log['epochs'], logger.log['loss'], marker="o", linestyle='--')
13 plt.plot(eval.log['epochs'], eval.log['loss'], marker="o")
14 plt.title('Model loss')
15 plt.ylabel('Loss')
16 plt.xlabel('Epoch')
17 plt.legend(['Train', 'Test'])
18 plt.show()
19 plt.savefig(os.path.join(base_fol, 'loss_plot.png'))#グラフの保存
```

図 67 学習結果の可視化

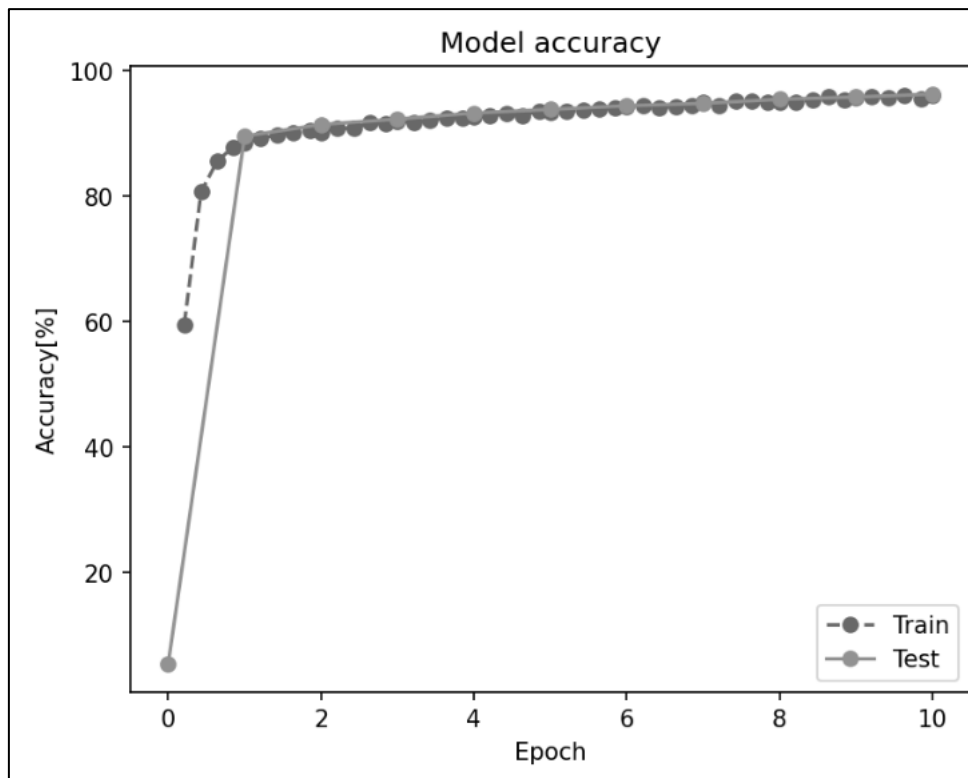


図 68 モデルのエポックと精度のグラフ

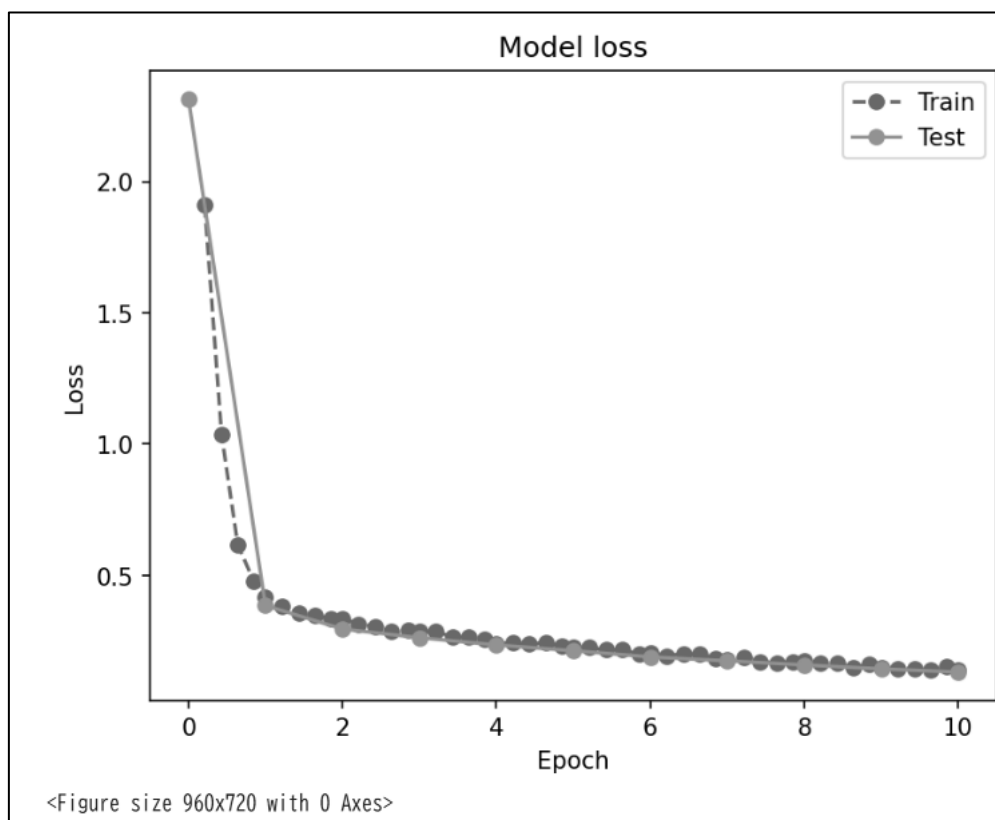


図 69 モデルのエポックと損失のグラフのプロット

Colaboratory のランタイムのタイプを GPU に変更し、CNN モデルの学習を行った。実行結果を図 70,71 に示す。

```

100% ██████████ 10/10 [01:43<00:00, 11.00s/it]
[ 0]test loss: 2.316, accuracy: 5.43%, speed:0.021s/iter
[ 0, 100] loss:1.913, accuracy:59.59%, speed:0.024s/iter
[ 0, 200] loss:1.035, accuracy:80.72%, speed:0.023s/iter
[ 0, 300] loss:0.617, accuracy:85.61%, speed:0.017s/iter
[ 0, 400] loss:0.476, accuracy:87.88%, speed:0.016s/iter
[ 0, 469] loss:0.417, accuracy:88.59%, speed:0.016s/iter
[ 1]test loss: 0.384, accuracy: 89.61%, speed:0.018s/iter
[ 1, 100] loss:0.383, accuracy:89.30%, speed:0.018s/iter
[ 1, 200] loss:0.355, accuracy:89.80%, speed:0.016s/iter
[ 1, 300] loss:0.345, accuracy:90.24%, speed:0.024s/iter
[ 1, 400] loss:0.335, accuracy:90.48%, speed:0.019s/iter
[ 1, 469] loss:0.336, accuracy:90.26%, speed:0.017s/iter
[ 2]test loss: 0.295, accuracy: 91.47%, speed:0.017s/iter
[ 2, 100] loss:0.314, accuracy:90.91%, speed:0.017s/iter
[ 2, 200] loss:0.304, accuracy:90.85%, speed:0.016s/iter
[ 2, 300] loss:0.288, accuracy:91.80%, speed:0.016s/iter
[ 2, 400] loss:0.290, accuracy:91.63%, speed:0.017s/iter
[ 2, 469] loss:0.286, accuracy:91.89%, speed:0.025s/iter
[ 3]test loss: 0.262, accuracy: 92.34%, speed:0.020s/iter
[ 3, 100] loss:0.285, accuracy:91.82%, speed:0.018s/iter
[ 3, 200] loss:0.265, accuracy:92.16%, speed:0.017s/iter
[ 3, 300] loss:0.265, accuracy:92.55%, speed:0.017s/iter
[ 3, 400] loss:0.255, accuracy:92.56%, speed:0.017s/iter
[ 3, 469] loss:0.240, accuracy:92.78%, speed:0.017s/iter
[ 4]test loss: 0.234, accuracy: 93.30%, speed:0.019s/iter
[ 4, 100] loss:0.241, accuracy:92.88%, speed:0.026s/iter
[ 4, 200] loss:0.237, accuracy:93.19%, speed:0.016s/iter
[ 4, 300] loss:0.243, accuracy:92.94%, speed:0.016s/iter
[ 4, 400] loss:0.230, accuracy:93.55%, speed:0.016s/iter
[ 4, 469] loss:0.224, accuracy:93.33%, speed:0.016s/iter
[ 5]test loss: 0.212, accuracy: 93.88%, speed:0.016s/iter
[ 5, 100] loss:0.224, accuracy:93.55%, speed:0.017s/iter
[ 5, 200] loss:0.216, accuracy:93.78%, speed:0.020s/iter
[ 5, 300] loss:0.215, accuracy:93.97%, speed:0.022s/iter
[ 5, 400] loss:0.200, accuracy:94.19%, speed:0.016s/iter
[ 5, 469] loss:0.204, accuracy:94.24%, speed:0.016s/iter

```

図 70 GPU の学習の出力結果前半部分

```

[ 6]test loss: 0.188, accuracy: 94.46%, speed:0.016s/iter
[ 6, 100] loss:0.193, accuracy:94.41%, speed:0.017s/iter
[ 6, 200] loss:0.200, accuracy:94.22%, speed:0.016s/iter
[ 6, 300] loss:0.198, accuracy:94.34%, speed:0.016s/iter
[ 6, 400] loss:0.184, accuracy:94.55%, speed:0.022s/iter
[ 6, 469] loss:0.178, accuracy:94.95%, speed:0.023s/iter
[ 7]test loss: 0.173, accuracy: 94.84%, speed:0.016s/iter
[ 7, 100] loss:0.189, accuracy:94.52%, speed:0.018s/iter
[ 7, 200] loss:0.171, accuracy:95.29%, speed:0.016s/iter
[ 7, 300] loss:0.165, accuracy:95.16%, speed:0.016s/iter
[ 7, 400] loss:0.170, accuracy:94.96%, speed:0.016s/iter
[ 7, 469] loss:0.172, accuracy:94.97%, speed:0.016s/iter
[ 8]test loss: 0.158, accuracy: 95.50%, speed:0.019s/iter
[ 8, 100] loss:0.165, accuracy:95.05%, speed:0.024s/iter
[ 8, 200] loss:0.164, accuracy:95.44%, speed:0.016s/iter
[ 8, 300] loss:0.148, accuracy:95.89%, speed:0.020s/iter
[ 8, 400] loss:0.159, accuracy:95.32%, speed:0.016s/iter
[ 8, 469] loss:0.149, accuracy:95.71%, speed:0.016s/iter
[ 9]test loss: 0.143, accuracy: 95.85%, speed:0.016s/iter
[ 9, 100] loss:0.143, accuracy:95.87%, speed:0.018s/iter
[ 9, 200] loss:0.145, accuracy:95.73%, speed:0.028s/iter
[ 9, 300] loss:0.140, accuracy:96.14%, speed:0.031s/iter
[ 9, 400] loss:0.153, accuracy:95.64%, speed:0.029s/iter
[ 9, 469] loss:0.139, accuracy:96.17%, speed:0.024s/iter
[10]test loss: 0.131, accuracy: 96.32%, speed:0.018s/iter
training total 105.00128388404846 sec.

```

図 71 GPU の学習の出力結果後半部分

GPU の種類を調べた。コードおよび実行結果を図 72 に示す。

```
1 !nvidia-smi
```

Sun May 24 10:17:33 2026

NVIDIA-SMI 580.82.07			Driver Version: 580.82.07		CUDA Version: 13.0		
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile Uncorr. ECC	
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M. MIG M.
0	Tesla T4		Off	00000000:00:04.0	Off		0
N/A	60C	P0	29W / 70W	251MiB / 15360MiB		0%	Default N/A

Processes:							
GPU	GI	CI	PID	Type	Process name	GPU Memory Usage	
ID	ID	ID					
0	N/A	N/A	1317	C	/usr/bin/python3	248MiB	

図 72 GPU の種類を調べるコードおよび実行結果

最適化方法を SGD から Adam に変更した。コードおよび実行結果を図 73,74 に示す。

1 learning_rate = 0.01	
2 init_seed(seed) #seedの値は任意に設定する	
3 device = 'cuda' if torch.cuda.is_available() else 'cpu'	
4 print('use device:', device)	
5 model = MyCNN().to(device)	
6 # image_size変数をtransform作成時に予め定義しておく	
7 summary(model, (1, image_size, image_size),device=device)	
8 criterion = nn.CrossEntropyLoss()#交差エントロピー誤差の計算	
9 #optimizer = optim.SGD(model.parameters(), lr=learning_rate) #最適化手法SDG 3.2 ⑬の時はコメントアウトしておく	
10 optimizer = optim.Adam(model.parameters(), lr=learning_rate) #最適化手法Adam 3.2 ⑬の時だけ使用する	
11 print('optimizer',optimizer)	

図 73 最適化手法 Adam の設定

```
use device: cuda
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 50, 24, 24]	1,300
ReLU-2	[-1, 50, 24, 24]	0
MaxPool2d-3	[-1, 50, 12, 12]	0
Flatten-4	[-1, 7200]	0
Linear-5	[-1, 100]	720,100
ReLU-6	[-1, 100]	0
Linear-7	[-1, 10]	1,010

Total params: 722,410
Trainable params: 722,410
Non-trainable params: 0

Input size (MB): 0.00
Forward/backward pass size (MB): 0.55
Params size (MB): 2.76
Estimated Total Size (MB): 3.31

```
optimizer Adam (
Parameter Group 0
  amsgrad: False
  betas: (0.9, 0.999)
  capturable: False
  decoupled_weight_decay: False
  differentiable: False
  eps: 1e-08
  foreach: None
  fused: None
  lr: 0.01
  maximize: False
  weight_decay: 0
)
```

図 74 Adam 設定時のサマリの出力結果

最適化方法に Adam を使って CNN モデルの学習を行った。実行結果を図 75,76 に示す。

```

100% 10/10 [01:38<00:00, 9.80s/it]
[ 0]test loss: 2.316, accuracy: 5.43%, speed:0.018s/iter
[ 0, 100] loss:0.826, accuracy:79.56%, speed:0.025s/iter
[ 0, 200] loss:0.124, accuracy:96.16%, speed:0.020s/iter
[ 0, 300] loss:0.092, accuracy:97.05%, speed:0.016s/iter
[ 0, 400] loss:0.087, accuracy:97.51%, speed:0.016s/iter
[ 0, 469] loss:0.073, accuracy:97.81%, speed:0.015s/iter
[ 1]test loss: 0.062, accuracy: 97.97%, speed:0.016s/iter
[ 1, 100] loss:0.044, accuracy:98.70%, speed:0.018s/iter
[ 1, 200] loss:0.059, accuracy:98.34%, speed:0.017s/iter
[ 1, 300] loss:0.052, accuracy:98.35%, speed:0.026s/iter
[ 1, 400] loss:0.069, accuracy:97.91%, speed:0.019s/iter
[ 1, 469] loss:0.069, accuracy:97.86%, speed:0.017s/iter
[ 2]test loss: 0.053, accuracy: 98.32%, speed:0.016s/iter
[ 2, 100] loss:0.035, accuracy:98.88%, speed:0.017s/iter
[ 2, 200] loss:0.037, accuracy:98.79%, speed:0.016s/iter
[ 2, 300] loss:0.044, accuracy:98.66%, speed:0.017s/iter
[ 2, 400] loss:0.050, accuracy:98.46%, speed:0.017s/iter
[ 2, 469] loss:0.053, accuracy:98.38%, speed:0.023s/iter
[ 3]test loss: 0.057, accuracy: 98.25%, speed:0.019s/iter
[ 3, 100] loss:0.029, accuracy:98.93%, speed:0.017s/iter
[ 3, 200] loss:0.030, accuracy:99.00%, speed:0.016s/iter
[ 3, 300] loss:0.033, accuracy:98.91%, speed:0.019s/iter
[ 3, 400] loss:0.035, accuracy:98.73%, speed:0.017s/iter
[ 3, 469] loss:0.037, accuracy:98.95%, speed:0.015s/iter
[ 4]test loss: 0.055, accuracy: 98.33%, speed:0.016s/iter
[ 4, 100] loss:0.024, accuracy:99.27%, speed:0.026s/iter
[ 4, 200] loss:0.026, accuracy:99.15%, speed:0.017s/iter
[ 4, 300] loss:0.031, accuracy:99.07%, speed:0.015s/iter
[ 4, 400] loss:0.032, accuracy:99.04%, speed:0.015s/iter
[ 4, 469] loss:0.028, accuracy:99.10%, speed:0.015s/iter
[ 5]test loss: 0.061, accuracy: 98.42%, speed:0.016s/iter
[ 5, 100] loss:0.019, accuracy:99.37%, speed:0.017s/iter
[ 5, 200] loss:0.019, accuracy:99.36%, speed:0.018s/iter
[ 5, 300] loss:0.027, accuracy:99.00%, speed:0.026s/iter
[ 5, 400] loss:0.028, accuracy:99.01%, speed:0.017s/iter
[ 5, 469] loss:0.029, accuracy:99.08%, speed:0.017s/iter

```

図 75 Adam を用いた CNN モデルの学習の出力結果前半部分

```

[ 6]test loss: 0.063, accuracy: 98.39%, speed:0.015s/iter
[ 6, 100] loss:0.013, accuracy:99.55%, speed:0.017s/iter
[ 6, 200] loss:0.022, accuracy:99.21%, speed:0.016s/iter
[ 6, 300] loss:0.018, accuracy:99.38%, speed:0.016s/iter
[ 6, 400] loss:0.027, accuracy:99.16%, speed:0.018s/iter
[ 6, 469] loss:0.036, accuracy:99.01%, speed:0.024s/iter
[ 7]test loss: 0.075, accuracy: 98.12%, speed:0.018s/iter
[ 7, 100] loss:0.023, accuracy:99.30%, speed:0.018s/iter
[ 7, 200] loss:0.020, accuracy:99.40%, speed:0.017s/iter
[ 7, 300] loss:0.031, accuracy:99.09%, speed:0.017s/iter
[ 7, 400] loss:0.027, accuracy:99.22%, speed:0.016s/iter
[ 7, 469] loss:0.026, accuracy:99.21%, speed:0.015s/iter
[ 8]test loss: 0.072, accuracy: 98.50%, speed:0.015s/iter
[ 8, 100] loss:0.015, accuracy:99.53%, speed:0.026s/iter
[ 8, 200] loss:0.015, accuracy:99.53%, speed:0.017s/iter
[ 8, 300] loss:0.020, accuracy:99.36%, speed:0.016s/iter
[ 8, 400] loss:0.017, accuracy:99.43%, speed:0.017s/iter
[ 8, 469] loss:0.025, accuracy:99.26%, speed:0.016s/iter
[ 9]test loss: 0.109, accuracy: 98.05%, speed:0.015s/iter
[ 9, 100] loss:0.023, accuracy:99.37%, speed:0.017s/iter
[ 9, 200] loss:0.021, accuracy:99.38%, speed:0.017s/iter
[ 9, 300] loss:0.018, accuracy:99.54%, speed:0.025s/iter
[ 9, 400] loss:0.022, accuracy:99.36%, speed:0.017s/iter
[ 9, 469] loss:0.019, accuracy:99.32%, speed:0.016s/iter
[ 10]test loss: 0.088, accuracy: 98.40%, speed:0.015s/iter
training total 99.62421441078186 sec.

```

図 76 Adam を用いた CNN モデルの学習の出力結果後半部分

学習の結果を可視化した。実行結果を図 77,78 に示す。

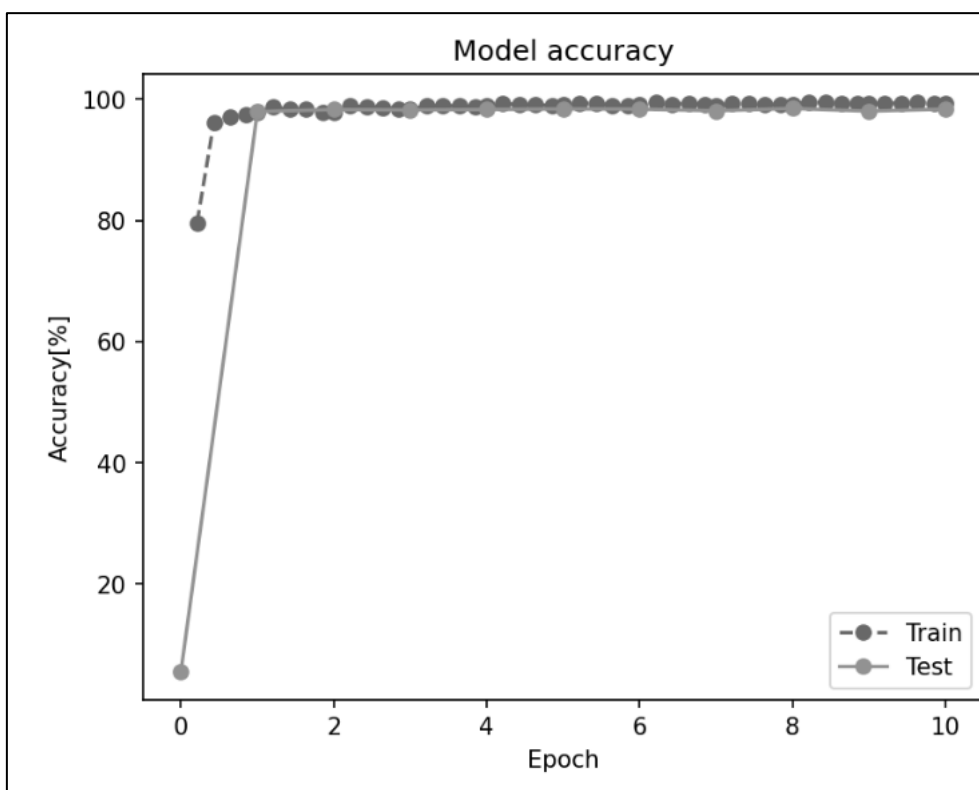


図 77 Adam を用いた場合のモデルのエポックと精度のグラフ

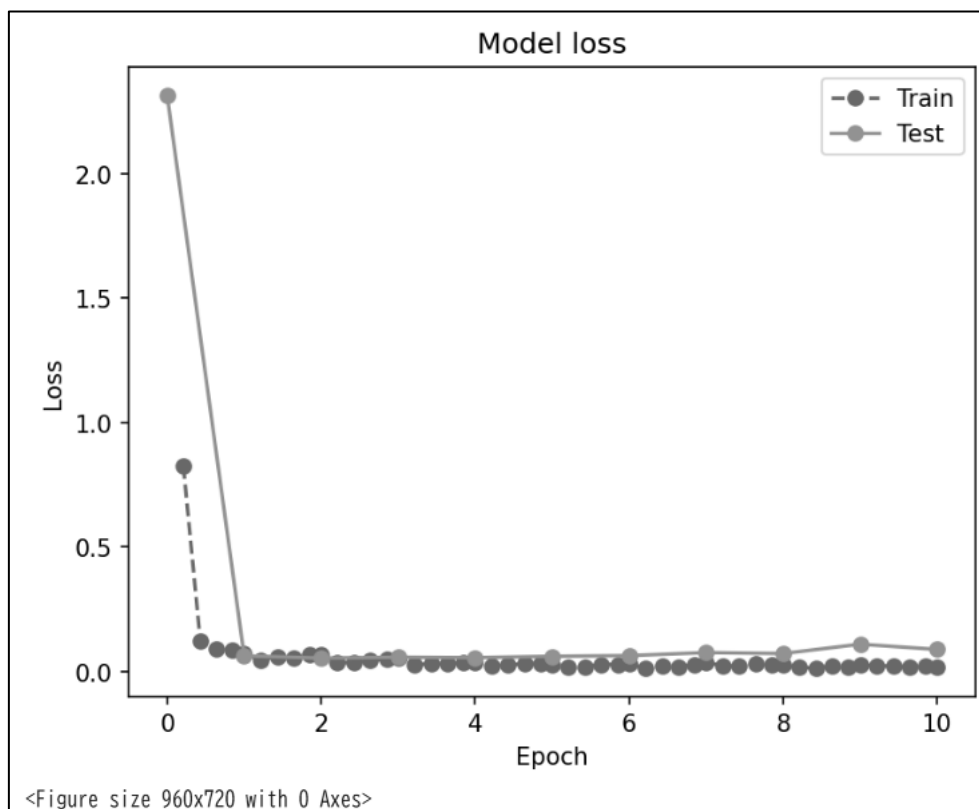


図 78 Adam を用いた場合のモデルのエポックと損失のグラフ

学習に用いるデータ数を 100 分の 1 にしたサブデータセットを作成した。コードおよび実行結果を図 79 に示す。

```
1 from sklearn.model_selection import StratifiedKFold
2 init_seed(1234) #ここでの seed の値は 1234 で固定する
3 skf = StratifiedKFold(n_splits=100)
4 split_set = skf.split(train_dataset.data, train_dataset.targets)
5 _, train_index = next(split_set)
6 sub_train_dataset = data.Subset(train_dataset, train_index)
7 sub_train_df = pd.DataFrame(train_dataset.targets[train_index])
8 sub_balance = sub_train_df.value_counts()
9 train_samples = len(sub_train_dataset)
10 display('sub train', sub_balance)
```

'sub train'	count
0	
1	68
7	63
3	61
2	60
0	59
6	59
9	59
4	58
8	58
5	55

dtype: int64

図 79 100 分の 1 の学習用データセットのバランスの可視化

サブデータセット用のデータローダを作成した。コードを図 80 に示す。

```
7 batch_size = 128
8 #sub_train_loaderにsub_train_datasetを入れる
9 sub_train_loader = data.DataLoader(sub_train_dataset,
10                                   batch_size = batch_size, shuffle=True, num_workers = 2)
```

図 80 サブセット用のデータローダ作成

サブデータローダと SGD を用いた CNN モデル 1 の学習を行った。コードおよび実行結果を図 81,82,83 に示す。

```

1 n_epoch=10#エポック数
2 print_iteration=1#学習時のロギング間隔
3 logger=Logger(print_iteration,train_samples,batch_size)
4 eval=Evaluator(test_loader,criterion,device)
5 start=time.time()
6 for epoch in tqdm(range(n_epoch)):#エポック数だけ学習を行う。
7     eval.evaluation(model,epoch)
8     model.train()
9     logger.clear_run()
10    print_time=time.time()
11    for i,(images,labels) in enumerate(sub_train_loader,0):
12        images=images.to(device)
13        labels=labels.to(device)
14        outputs=model(images)
15        optimizer.zero_grad()
16        loss=criterion(outputs,labels)
17        loss.backward()
18        optimizer.step()
19        logger.logging(outputs,loss,labels,epoch,i+1)
20 #3.2 ⑦精度向上のために追加
21 """
22 if epoch == 10:
23     learning_rate = 0.001
24 if epoch == 15:
25     learning_rate = 0.0001
26 """
27 eval.evaluation(model,n_epoch)
28 elapsed_time=time.time()-start
29 print('training total',elapsed_time,'sec.')#合計の処理時間
30
31 #データの記録
32 finalaccuracy = eval.log["acc"][-1] #accuracy
33 finalloss = eval.log["loss"][-1] # loss

```

図 81 CNN モデル 1 の学習

CNN モデル 1 の学習を行った。実行結果を図 82,83 に示す。

```

100% 10/10 [00:16<00:00, 1.79s/it]
[ 0]test loss: 1.928, accuracy: 66.57%, speed:0.023s/iter
[ 0, 1] loss:1.858, accuracy:82.03%, speed:0.161s/iter
[ 0, 2] loss:1.898, accuracy:71.09%, speed:0.025s/iter
[ 0, 3] loss:1.890, accuracy:73.44%, speed:0.029s/iter
[ 0, 4] loss:1.866, accuracy:71.09%, speed:0.013s/iter
[ 0, 5] loss:1.830, accuracy:80.68%, speed:0.012s/iter
[ 1]test loss: 1.881, accuracy: 68.22%, speed:0.021s/iter
[ 1, 1] loss:1.828, accuracy:78.12%, speed:0.111s/iter
[ 1, 2] loss:1.831, accuracy:72.66%, speed:0.017s/iter
[ 1, 3] loss:1.821, accuracy:71.09%, speed:0.024s/iter
[ 1, 4] loss:1.820, accuracy:72.66%, speed:0.004s/iter
[ 1, 5] loss:1.795, accuracy:77.27%, speed:0.011s/iter
[ 2]test loss: 1.833, accuracy: 69.08%, speed:0.017s/iter
[ 2, 1] loss:1.769, accuracy:82.03%, speed:0.094s/iter
[ 2, 2] loss:1.801, accuracy:72.66%, speed:0.012s/iter
[ 2, 3] loss:1.743, accuracy:76.56%, speed:0.018s/iter
[ 2, 4] loss:1.787, accuracy:70.31%, speed:0.005s/iter
[ 2, 5] loss:1.739, accuracy:78.41%, speed:0.009s/iter
[ 3]test loss: 1.784, accuracy: 70.01%, speed:0.016s/iter
[ 3, 1] loss:1.758, accuracy:70.31%, speed:0.091s/iter
[ 3, 2] loss:1.678, accuracy:82.03%, speed:0.010s/iter
[ 3, 3] loss:1.729, accuracy:78.12%, speed:0.009s/iter
[ 3, 4] loss:1.722, accuracy:74.22%, speed:0.007s/iter
[ 3, 5] loss:1.692, accuracy:77.27%, speed:0.012s/iter
[ 4]test loss: 1.734, accuracy: 70.81%, speed:0.017s/iter
[ 4, 1] loss:1.683, accuracy:79.69%, speed:0.097s/iter
[ 4, 2] loss:1.657, accuracy:78.12%, speed:0.010s/iter
[ 4, 3] loss:1.651, accuracy:79.69%, speed:0.015s/iter
[ 4, 4] loss:1.643, accuracy:72.66%, speed:0.006s/iter
[ 4, 5] loss:1.678, accuracy:72.73%, speed:0.009s/iter
[ 5]test loss: 1.684, accuracy: 71.16%, speed:0.016s/iter
[ 5, 1] loss:1.693, accuracy:72.66%, speed:0.100s/iter
[ 5, 2] loss:1.511, accuracy:86.72%, speed:0.007s/iter
[ 5, 3] loss:1.616, accuracy:73.44%, speed:0.016s/iter
[ 5, 4] loss:1.627, accuracy:75.00%, speed:0.004s/iter
[ 5, 5] loss:1.608, accuracy:71.59%, speed:0.011s/iter

```

図 82 CNN モデル 1 の学習の出力結果前半部分

```

[ 6]test loss: 1.632, accuracy: 72.04%, speed:0.016s/iter
[ 6,    1] loss:1.572, accuracy:80.47%, speed:0.098s/iter
[ 6,    2] loss:1.532, accuracy:82.03%, speed:0.015s/iter
[ 6,    3] loss:1.610, accuracy:70.31%, speed:0.028s/iter
[ 6,    4] loss:1.529, accuracy:81.25%, speed:0.006s/iter
[ 6,    5] loss:1.506, accuracy:77.27%, speed:0.008s/iter
[ 7]test loss: 1.582, accuracy: 71.10%, speed:0.017s/iter
[ 7,    1] loss:1.458, accuracy:79.69%, speed:0.099s/iter
[ 7,    2] loss:1.497, accuracy:82.03%, speed:0.010s/iter
[ 7,    3] loss:1.533, accuracy:76.56%, speed:0.016s/iter
[ 7,    4] loss:1.517, accuracy:71.88%, speed:0.004s/iter
[ 7,    5] loss:1.476, accuracy:75.00%, speed:0.010s/iter
[ 8]test loss: 1.529, accuracy: 72.37%, speed:0.025s/iter
[ 8,    1] loss:1.524, accuracy:78.12%, speed:0.130s/iter
[ 8,    2] loss:1.458, accuracy:79.69%, speed:0.065s/iter
[ 8,    3] loss:1.403, accuracy:81.25%, speed:0.009s/iter
[ 8,    4] loss:1.429, accuracy:77.34%, speed:0.022s/iter
[ 8,    5] loss:1.385, accuracy:80.68%, speed:0.004s/iter
[ 9]test loss: 1.476, accuracy: 72.53%, speed:0.022s/iter
[ 9,    1] loss:1.404, accuracy:83.59%, speed:0.106s/iter
[ 9,    2] loss:1.407, accuracy:73.44%, speed:0.010s/iter
[ 9,    3] loss:1.467, accuracy:70.31%, speed:0.014s/iter
[ 9,    4] loss:1.316, accuracy:83.59%, speed:0.005s/iter
[ 9,    5] loss:1.316, accuracy:80.68%, speed:0.009s/iter
[10]test loss: 1.426, accuracy: 73.34%, speed:0.017s/iter
training total 18.30208158493042 sec.

```

図 83 CNN モデル 1 の学習の出力結果後半部分

図 81 より、100 分の 1 のデータセットを用いているため、`print_iteration` を 100 分の 1 にすることでロギング間隔を維持している。

学習の結果を可視化した。実行結果を図 84,85 に示す。

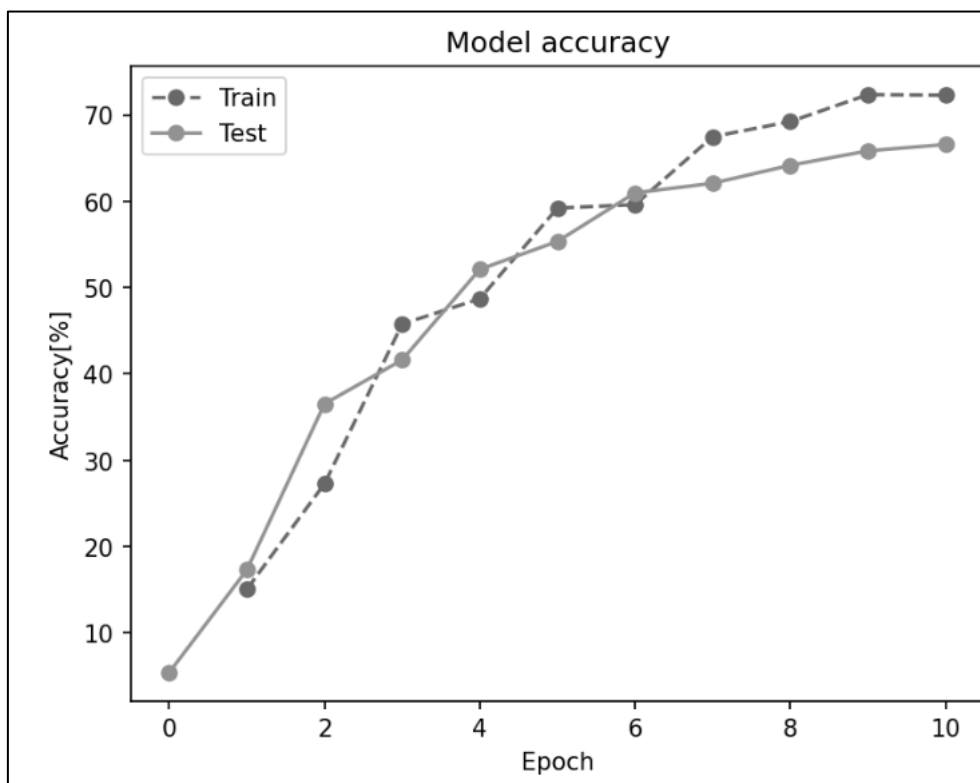


図 84 CNN モデル 1 のエポックと精度のグラフ

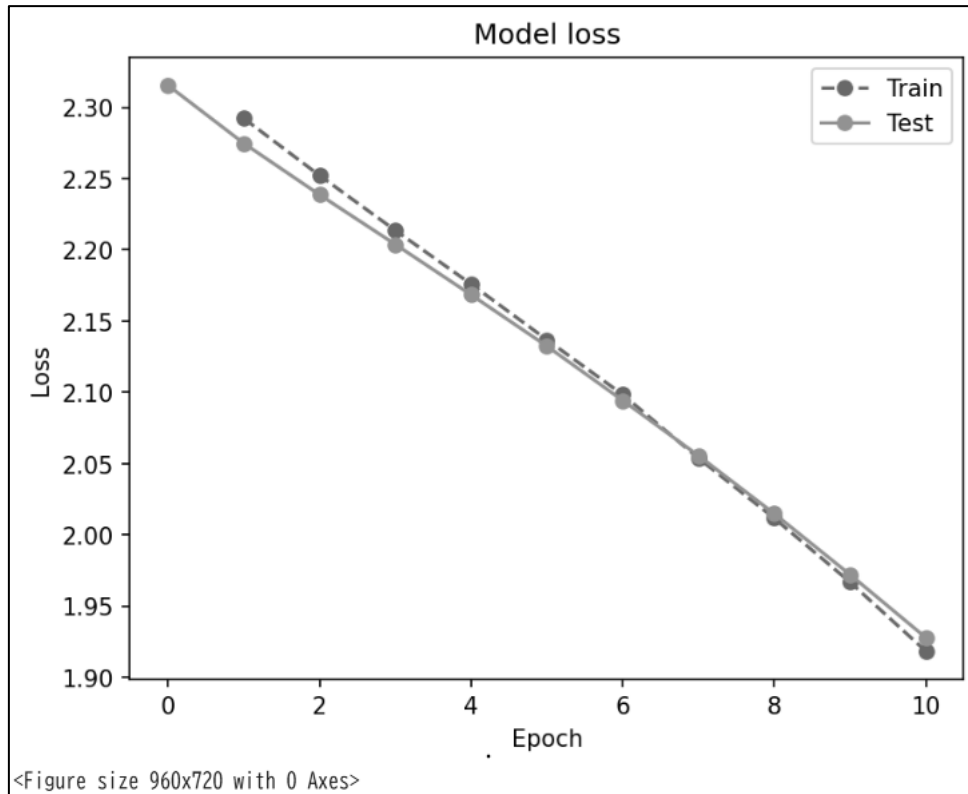


図 85 CNN モデル 1 のエポックと損失のグラフ

CNN モデル 1 の最適化手法を Adam に変更した CNN モデル 2 を図 73 のコードを用いて定義し、学習を行った。実行結果を図 86,87 に示す。

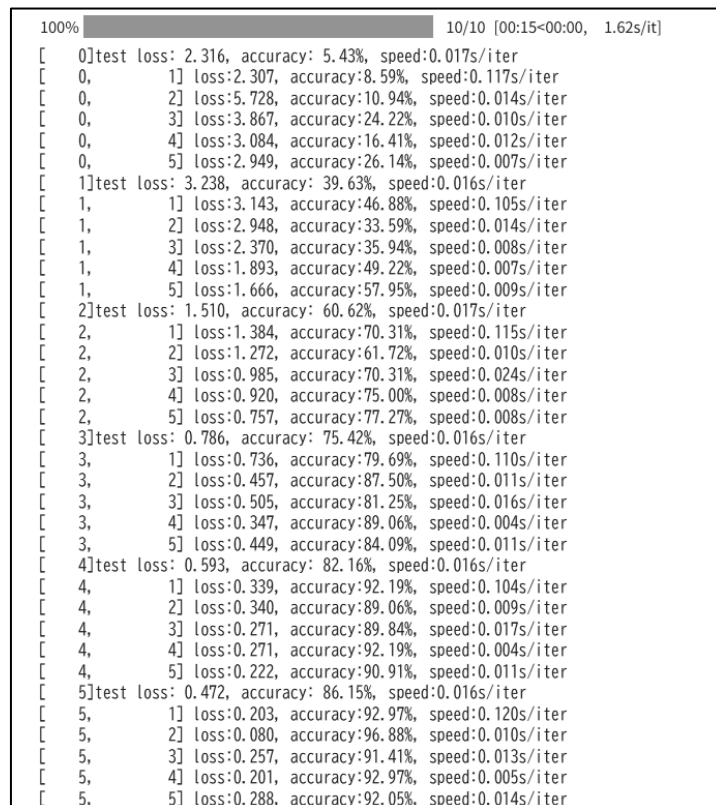


図 86 CNN モデル 2 の学習の出力結果前半部分

```

[ 6]test loss: 0.433, accuracy: 87.68%, speed:0.019s/iter
[ 6,      1] loss:0.073, accuracy:98.44%, speed:0.127s/iter
[ 6,      2] loss:0.089, accuracy:96.88%, speed:0.061s/iter
[ 6,      3] loss:0.150, accuracy:96.88%, speed:0.010s/iter
[ 6,      4] loss:0.122, accuracy:95.31%, speed:0.018s/iter
[ 6,      5] loss:0.042, accuracy:100.00%, speed:0.005s/iter
[ 7]test loss: 0.382, accuracy: 89.38%, speed:0.027s/iter
[ 7,      1] loss:0.039, accuracy:100.00%, speed:0.123s/iter
[ 7,      2] loss:0.032, accuracy:100.00%, speed:0.012s/iter
[ 7,      3] loss:0.095, accuracy:96.88%, speed:0.013s/iter
[ 7,      4] loss:0.069, accuracy:98.44%, speed:0.004s/iter
[ 7,      5] loss:0.030, accuracy:100.00%, speed:0.011s/iter
[ 8]test loss: 0.372, accuracy: 89.61%, speed:0.016s/iter
[ 8,      1] loss:0.022, accuracy:99.22%, speed:0.097s/iter
[ 8,      2] loss:0.031, accuracy:99.22%, speed:0.011s/iter
[ 8,      3] loss:0.018, accuracy:100.00%, speed:0.008s/iter
[ 8,      4] loss:0.026, accuracy:100.00%, speed:0.010s/iter
[ 8,      5] loss:0.041, accuracy:98.86%, speed:0.006s/iter
[ 9]test loss: 0.398, accuracy: 89.63%, speed:0.016s/iter
[ 9,      1] loss:0.018, accuracy:99.22%, speed:0.102s/iter
[ 9,      2] loss:0.015, accuracy:100.00%, speed:0.009s/iter
[ 9,      3] loss:0.021, accuracy:100.00%, speed:0.014s/iter
[ 9,      4] loss:0.011, accuracy:100.00%, speed:0.004s/iter
[ 9,      5] loss:0.006, accuracy:100.00%, speed:0.010s/iter
[10]test loss: 0.389, accuracy: 90.67%, speed:0.018s/iter
training total 17.28978991508484 sec.

```

図 87 CNN モデル 2 の学習の出力結果後半部分

学習の結果を可視化した。実行結果を図 88,89 に示す。

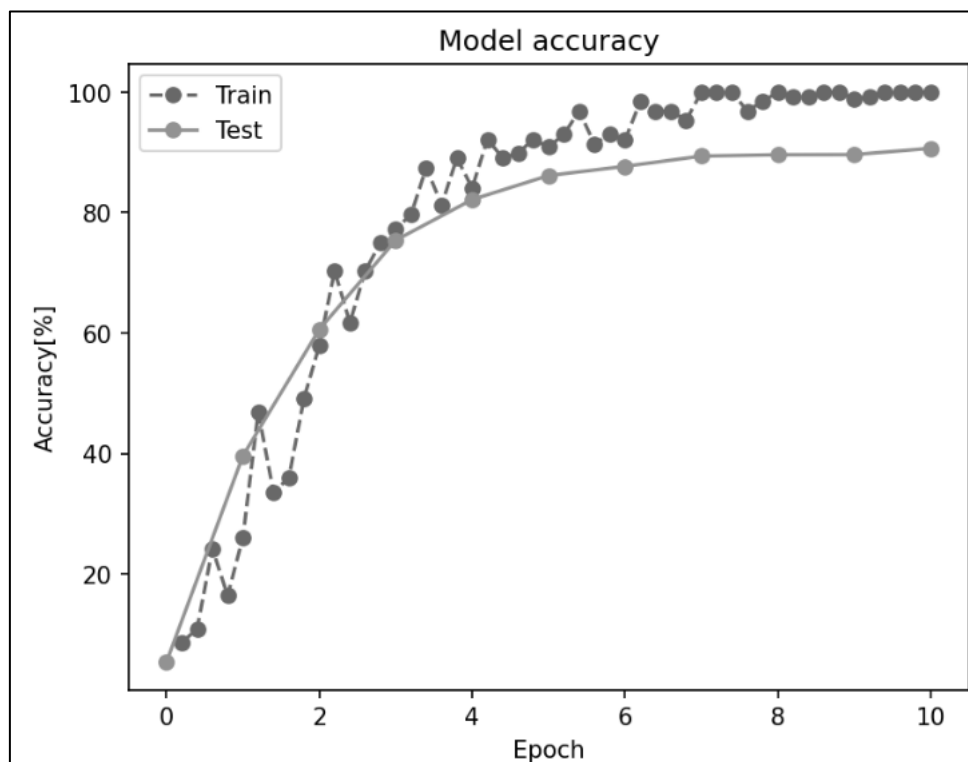


図 88 CNN モデル 2 のエポックと精度のグラフ

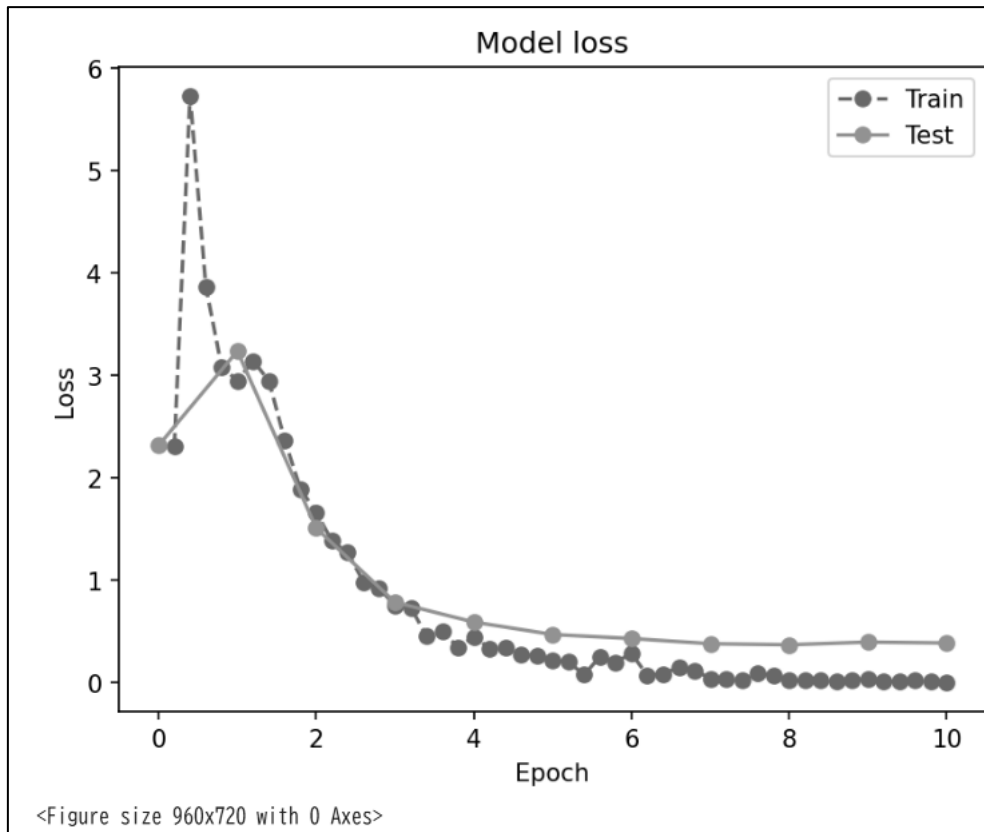


図 89 CNN モデル 2 のエポックと損失のグラフ

CNN モデル 2 にドロップアウト層を追加した CNN モデル 3 を定義した。コードを図 90 に示す。

```

1 class MyCNN(nn.Module):
2     def __init__(self):
3         super(MyCNN, self).__init__()
4         self.conv1 = nn.Conv2d(1, 50, (5, 5))
5         self.conv2 = nn.Conv2d(50, 100, (3, 3), padding=1)
6         #self.fc1 = nn.Linear(6*6*100, 100)
7         self.fc1 = nn.Linear(12*12*50, 100)
8         self.fc2 = nn.Linear(100, 10)
9         self.pool = nn.MaxPool2d((2, 2))
10        self.flat = nn.Flatten()
11        self.drop = nn.Dropout(0.5)
12        self.relu = nn.ReLU()
13    def forward(self, x):
14        h = self.relu(self.conv1(x))#畳み込み層の後ReLU関数により活性化
15        h = self.pool(h)#Maxプーリング層
16        #h = self.relu(self.conv2(h))畳み込み層の後ReLU関数により活性化 3.2 ⑦精度向上のために追加
17        #h = self.pool(h)#Maxプーリング層 3.2 ⑦精度向上のために追加
18        h = self.flat(h)#平坦化
19        h = self.relu(self.fc1(h))#全結合層の後ReLU関数により活性化
20        h = self.drop(h)#ドロップアウト 3.2 ⑦精度向上のために追加
21        y = self.fc2(h)#全結合層の後ReLU関数により活性化
22        return y
23    def predict(self, x):
24        y = self.forward(x)
25        return F.softmax(y, dim=1)#softmax関数を通して推論結果を出力

```

図 90 CNN モデル 3 の定義

CNN モデル 3 の学習を行った。実行結果を図 91,92 に示す。

```

100% ██████████ 10/10 [00:16<00:00, 1.58s/it]
[ 0]test loss: 2.316, accuracy: 5.43%, speed:0.017s/iter
[ 0, 1] loss:2.311, accuracy:8.59%, speed:0.114s/iter
[ 0, 2] loss:4.837, accuracy:17.97%, speed:0.012s/iter
[ 0, 3] loss:5.425, accuracy:11.72%, speed:0.012s/iter
[ 0, 4] loss:3.703, accuracy:18.75%, speed:0.005s/iter
[ 0, 5] loss:2.389, accuracy:20.45%, speed:0.010s/iter
[ 1]test loss: 1.962, accuracy: 38.01%, speed:0.016s/iter
[ 1, 1] loss:2.117, accuracy:28.12%, speed:0.116s/iter
[ 1, 2] loss:1.948, accuracy:39.06%, speed:0.012s/iter
[ 1, 3] loss:1.889, accuracy:32.81%, speed:0.015s/iter
[ 1, 4] loss:1.859, accuracy:40.62%, speed:0.010s/iter
[ 1, 5] loss:1.884, accuracy:36.36%, speed:0.008s/iter
[ 2]test loss: 1.647, accuracy: 50.40%, speed:0.017s/iter
[ 2, 1] loss:1.566, accuracy:47.66%, speed:0.112s/iter
[ 2, 2] loss:1.604, accuracy:46.88%, speed:0.012s/iter
[ 2, 3] loss:1.463, accuracy:45.31%, speed:0.011s/iter
[ 2, 4] loss:1.307, accuracy:46.88%, speed:0.009s/iter
[ 2, 5] loss:1.233, accuracy:54.55%, speed:0.005s/iter
[ 3]test loss: 0.953, accuracy: 72.75%, speed:0.017s/iter
[ 3, 1] loss:1.306, accuracy:56.25%, speed:0.114s/iter
[ 3, 2] loss:1.067, accuracy:62.50%, speed:0.015s/iter
[ 3, 3] loss:1.069, accuracy:62.50%, speed:0.009s/iter
[ 3, 4] loss:0.980, accuracy:67.19%, speed:0.006s/iter
[ 3, 5] loss:0.966, accuracy:63.64%, speed:0.011s/iter
[ 4]test loss: 0.779, accuracy: 77.07%, speed:0.018s/iter
[ 4, 1] loss:0.823, accuracy:73.44%, speed:0.111s/iter
[ 4, 2] loss:0.778, accuracy:78.91%, speed:0.014s/iter
[ 4, 3] loss:0.712, accuracy:73.44%, speed:0.021s/iter
[ 4, 4] loss:0.811, accuracy:78.12%, speed:0.005s/iter
[ 4, 5] loss:0.796, accuracy:73.86%, speed:0.011s/iter
[ 5]test loss: 0.556, accuracy: 82.73%, speed:0.023s/iter
[ 5, 1] loss:0.668, accuracy:75.78%, speed:0.119s/iter
[ 5, 2] loss:0.381, accuracy:86.72%, speed:0.062s/iter
[ 5, 3] loss:0.712, accuracy:78.91%, speed:0.011s/iter
[ 5, 4] loss:0.450, accuracy:83.59%, speed:0.018s/iter
[ 5, 5] loss:0.935, accuracy:68.18%, speed:0.004s/iter

```

図 91 CNN モデル 3 の学習の出力結果前半部分

```

[ 6]test loss: 0.452, accuracy: 87.07%, speed:0.023s/iter
[ 6, 1] loss:0.459, accuracy:80.47%, speed:0.093s/iter
[ 6, 2] loss:0.575, accuracy:78.91%, speed:0.013s/iter
[ 6, 3] loss:0.444, accuracy:89.06%, speed:0.019s/iter
[ 6, 4] loss:0.528, accuracy:82.03%, speed:0.011s/iter
[ 6, 5] loss:0.464, accuracy:81.82%, speed:0.005s/iter
[ 7]test loss: 0.394, accuracy: 88.85%, speed:0.017s/iter
[ 7, 1] loss:0.354, accuracy:89.06%, speed:0.101s/iter
[ 7, 2] loss:0.428, accuracy:89.84%, speed:0.012s/iter
[ 7, 3] loss:0.481, accuracy:84.38%, speed:0.010s/iter
[ 7, 4] loss:0.572, accuracy:85.16%, speed:0.004s/iter
[ 7, 5] loss:0.285, accuracy:92.05%, speed:0.010s/iter
[ 8]test loss: 0.380, accuracy: 88.92%, speed:0.016s/iter
[ 8, 1] loss:0.329, accuracy:86.72%, speed:0.106s/iter
[ 8, 2] loss:0.333, accuracy:87.50%, speed:0.013s/iter
[ 8, 3] loss:0.326, accuracy:87.50%, speed:0.006s/iter
[ 8, 4] loss:0.333, accuracy:89.06%, speed:0.006s/iter
[ 8, 5] loss:0.427, accuracy:85.23%, speed:0.011s/iter
[ 9]test loss: 0.362, accuracy: 89.56%, speed:0.016s/iter
[ 9, 1] loss:0.225, accuracy:92.97%, speed:0.111s/iter
[ 9, 2] loss:0.319, accuracy:85.94%, speed:0.012s/iter
[ 9, 3] loss:0.365, accuracy:88.28%, speed:0.010s/iter
[ 9, 4] loss:0.362, accuracy:85.94%, speed:0.004s/iter
[ 9, 5] loss:0.291, accuracy:88.64%, speed:0.010s/iter
[ 10]test loss: 0.355, accuracy: 90.05%, speed:0.016s/iter
training total 17.514211654663086 sec.

```

図 92 CNN モデル 3 の学習の出力結果後半部分

学習の結果を可視化した。実行結果を図 93,94 に示す。

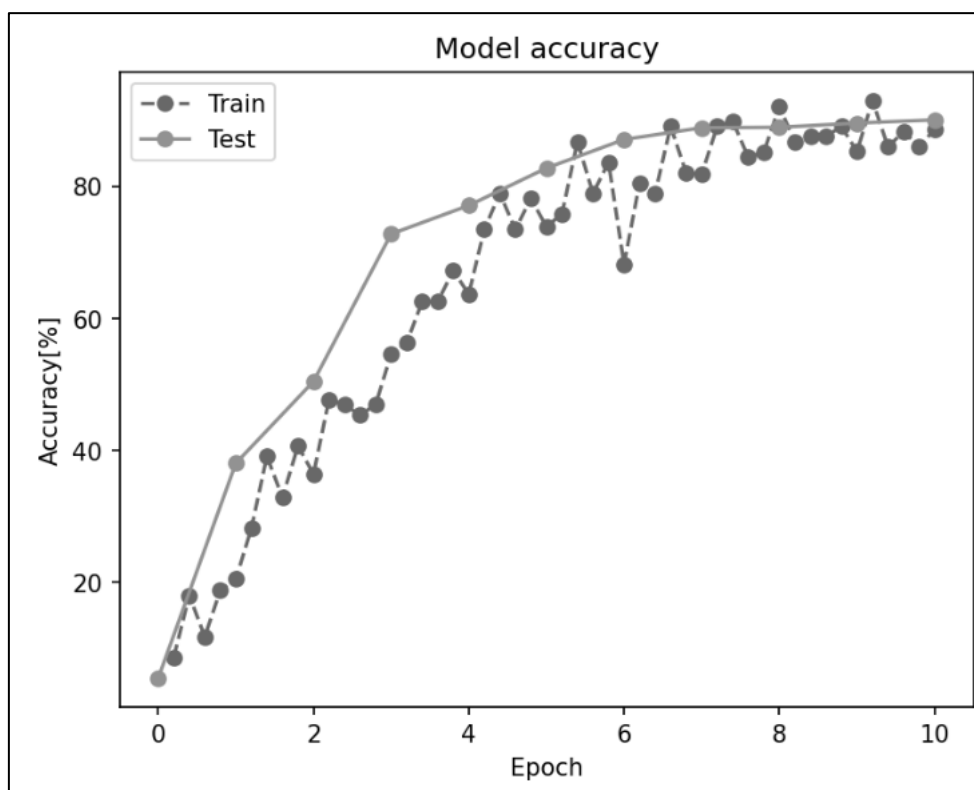


図 93 CNN モデル 3 のエポックと精度のグラフ

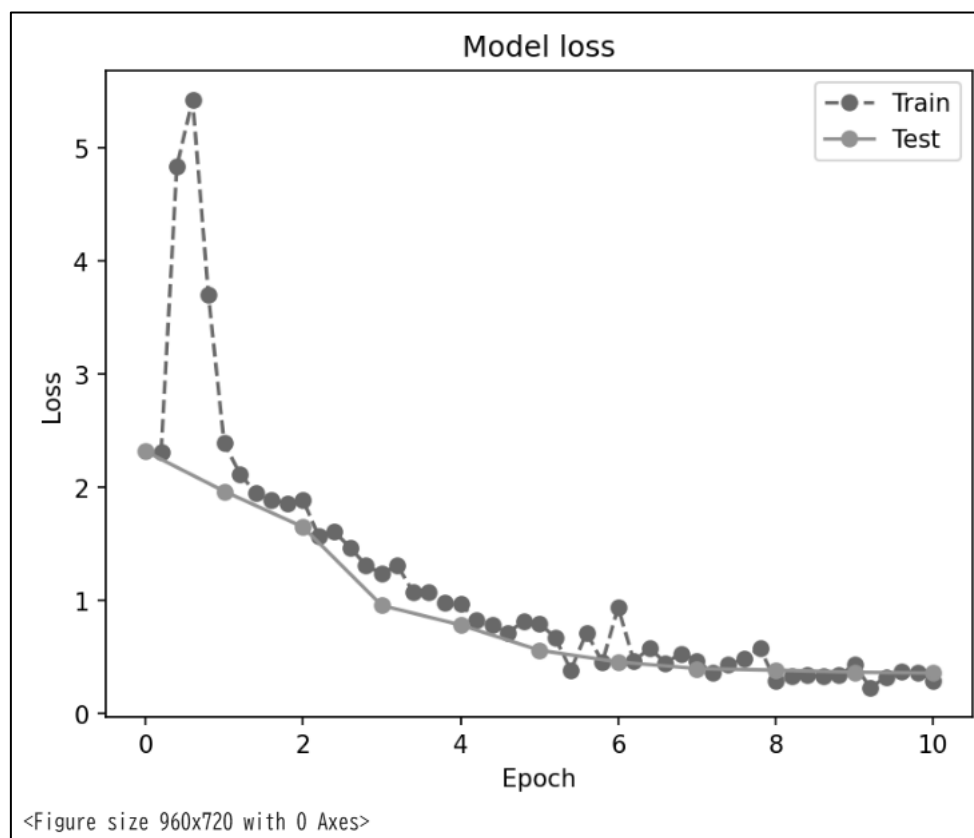


図 94 CNN モデル 3 のエポックと損失のグラフ

CNN モデル 3 に畳み込み層とプーリング層を追加した CNN モデル 4 を定義した。コードを図 95 に示す。

```

1 class MyCNN(nn.Module):
2     def __init__(self):
3         super(MyCNN, self).__init__()
4         self.conv1 = nn.Conv2d(1, 50, (5, 5))
5         self.conv2 = nn.Conv2d(50, 100, (3, 3), padding=1)
6         self.fc1 = nn.Linear(6*6*100, 100)
7         #self.fc1 = nn.Linear(12*12*50, 100)
8         self.fc2 = nn.Linear(100, 10)
9         self.pool = nn.MaxPool2d((2,2))
10        self.flat = nn.Flatten()
11        self.drop = nn.Dropout(0.5)
12        self.relu = nn.ReLU()
13    def forward(self, x):
14        h = self.relu(self.conv1(x))#畳み込み層の後ReLU関数により活性化
15        h = self.pool(h)#Maxプーリング層
16        h = self.relu(self.conv2(h))#畳み込み層の後ReLU関数により活性化3.2 ⑦精度向上のために追加
17        h = self.pool(h)#Maxプーリング層 3.2 ⑦精度向上のために追加
18        h = self.flat(h)#平坦化
19        h = self.relu(self.fc1(h))#全結合層の後ReLU関数により活性化
20        #h = self.drop(h)#ドロップアウト 3.2 ⑦精度向上のために追加
21        y = self.fc2(h)#全結合層の後ReLU関数により活性化
22        return y
23    def predict(self, x):
24        y = self.forward(x)
25        return F.softmax(y, dim=1)#softmax関数を通して推論結果を出力

```

図 95 CNN モデル 4 の定義

CNN モデル 4 を用いて学習を行った。実行結果を図 96,97 に示す。

```

100% 10/10 [00:16<00:00, 1.60s/it]
[ 0]test loss: 2.309, accuracy: 4.46%, speed:0.017s/iter
[ 0, 1] loss:2.296, accuracy:4.69%, speed:0.129s/iter
[ 0, 2] loss:4.014, accuracy:18.75%, speed:0.010s/iter
[ 0, 3] loss:2.408, accuracy:6.25%, speed:0.014s/iter
[ 0, 4] loss:2.286, accuracy:14.06%, speed:0.006s/iter
[ 0, 5] loss:2.253, accuracy:29.55%, speed:0.010s/iter
[ 1]test loss: 2.209, accuracy: 18.87%, speed:0.017s/iter
[ 1, 1] loss:2.217, accuracy:11.72%, speed:0.103s/iter
[ 1, 2] loss:2.106, accuracy:41.41%, speed:0.010s/iter
[ 1, 3] loss:1.976, accuracy:53.12%, speed:0.014s/iter
[ 1, 4] loss:1.825, accuracy:51.56%, speed:0.006s/iter
[ 1, 5] loss:1.497, accuracy:73.86%, speed:0.008s/iter
[ 2]test loss: 1.411, accuracy: 58.24%, speed:0.016s/iter
[ 2, 1] loss:1.372, accuracy:61.72%, speed:0.111s/iter
[ 2, 2] loss:1.011, accuracy:73.44%, speed:0.013s/iter
[ 2, 3] loss:0.917, accuracy:69.53%, speed:0.013s/iter
[ 2, 4] loss:0.930, accuracy:70.31%, speed:0.007s/iter
[ 2, 5] loss:0.573, accuracy:79.55%, speed:0.006s/iter
[ 3]test loss: 0.966, accuracy: 69.46%, speed:0.016s/iter
[ 3, 1] loss:0.828, accuracy:70.31%, speed:0.107s/iter
[ 3, 2] loss:0.708, accuracy:78.91%, speed:0.012s/iter
[ 3, 3] loss:0.506, accuracy:82.81%, speed:0.011s/iter
[ 3, 4] loss:0.351, accuracy:90.62%, speed:0.006s/iter
[ 3, 5] loss:0.735, accuracy:75.00%, speed:0.008s/iter
[ 4]test loss: 0.762, accuracy: 77.90%, speed:0.018s/iter
[ 4, 1] loss:0.551, accuracy:82.81%, speed:0.108s/iter
[ 4, 2] loss:0.461, accuracy:88.28%, speed:0.013s/iter
[ 4, 3] loss:0.351, accuracy:87.50%, speed:0.013s/iter
[ 4, 4] loss:0.348, accuracy:89.84%, speed:0.006s/iter
[ 4, 5] loss:0.326, accuracy:89.77%, speed:0.009s/iter
[ 5]test loss: 0.461, accuracy: 86.61%, speed:0.020s/iter
[ 5, 1] loss:0.395, accuracy:89.06%, speed:0.132s/iter
[ 5, 2] loss:0.186, accuracy:96.09%, speed:0.060s/iter
[ 5, 3] loss:0.336, accuracy:87.50%, speed:0.016s/iter
[ 5, 4] loss:0.294, accuracy:92.19%, speed:0.009s/iter
[ 5, 5] loss:0.308, accuracy:90.91%, speed:0.005s/iter

```

図 96 CNN モデル 4 の学習の出力結果前半部分


```

[ 6]test loss: 0.380, accuracy: 88.74%, speed:0.025s/iter
[ 6,      1] loss:0.162, accuracy:95.31%, speed:0.132s/iter
[ 6,      2] loss:0.206, accuracy:93.75%, speed:0.052s/iter
[ 6,      3] loss:0.324, accuracy:89.84%, speed:0.009s/iter
[ 6,      4] loss:0.147, accuracy:94.53%, speed:0.027s/iter
[ 6,      5] loss:0.171, accuracy:95.45%, speed:0.006s/iter
[ 7]test loss: 0.352, accuracy: 90.29%, speed:0.017s/iter
[ 7,      1] loss:0.086, accuracy:96.88%, speed:0.110s/iter
[ 7,      2] loss:0.139, accuracy:94.53%, speed:0.015s/iter
[ 7,      3] loss:0.150, accuracy:93.75%, speed:0.027s/iter
[ 7,      4] loss:0.135, accuracy:96.88%, speed:0.006s/iter
[ 7,      5] loss:0.102, accuracy:98.86%, speed:0.009s/iter
[ 8]test loss: 0.347, accuracy: 90.28%, speed:0.017s/iter
[ 8,      1] loss:0.091, accuracy:96.09%, speed:0.106s/iter
[ 8,      2] loss:0.099, accuracy:98.44%, speed:0.010s/iter
[ 8,      3] loss:0.044, accuracy:99.22%, speed:0.010s/iter
[ 8,      4] loss:0.084, accuracy:98.44%, speed:0.007s/iter
[ 8,      5] loss:0.113, accuracy:95.45%, speed:0.009s/iter
[ 9]test loss: 0.370, accuracy: 90.27%, speed:0.016s/iter
[ 9,      1] loss:0.056, accuracy:97.66%, speed:0.105s/iter
[ 9,      2] loss:0.057, accuracy:98.44%, speed:0.014s/iter
[ 9,      3] loss:0.125, accuracy:94.53%, speed:0.011s/iter
[ 9,      4] loss:0.031, accuracy:99.22%, speed:0.008s/iter
[ 9,      5] loss:0.032, accuracy:98.86%, speed:0.006s/iter
[10]test loss: 0.342, accuracy: 91.11%, speed:0.017s/iter
training total 17.590608835220337 sec.

```

図 97 CNN モデル 4 の学習の出力結果後半部分

学習の結果を可視化した。実行結果を図 98,99 に示す。

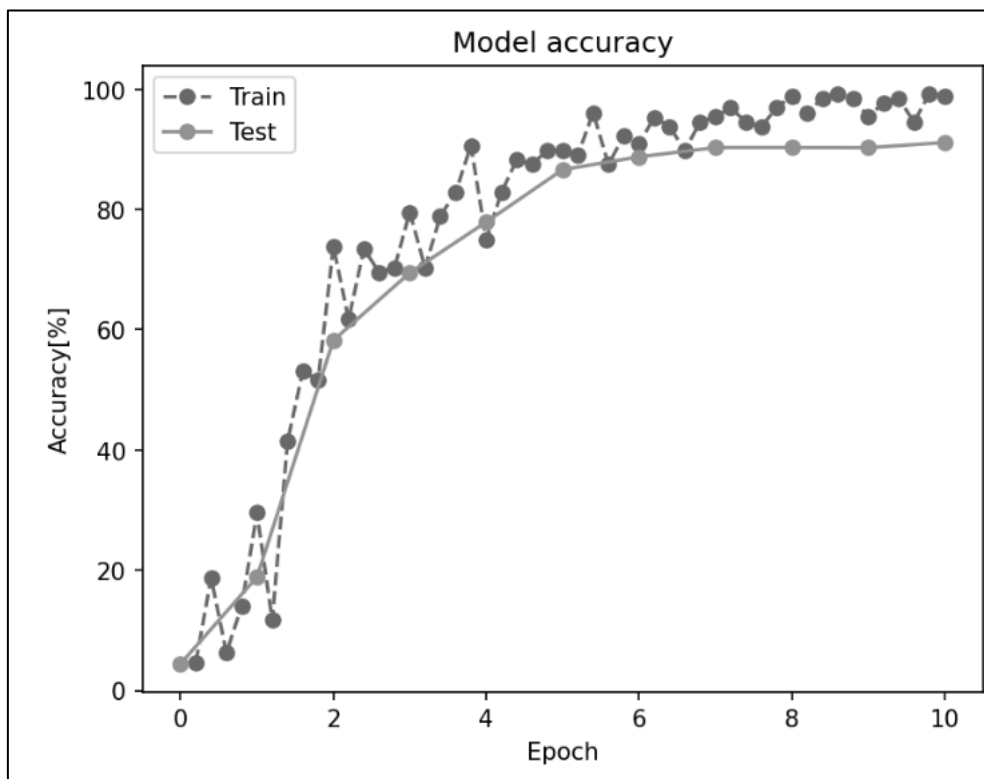


図 98 CNN モデル 4 のエポックと精度のグラフ

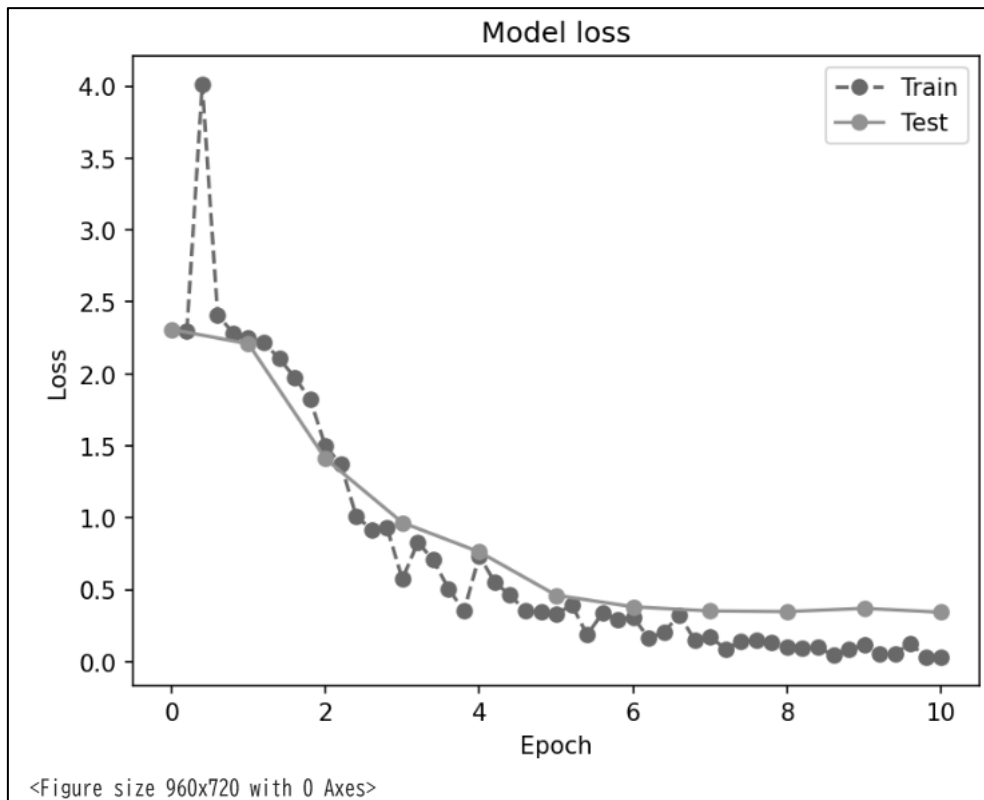


図 99 CNN モデル 4 のエポックと損失のグラフ

CNN モデル 4 のエポックを 15 に変更して学習を行った。コードを図 100 に示す。

```

1 n_epoch=15#エポック数
2 print_iteration=1#学習時のロギング間隔
3 logger=Logger(print_iteration,train_samples,batch_size)
4 eval=Evaluator(test_loader,criterion,device)
5 start=time.time()
6 for epoch in tqdm(range(n_epoch)):#エポック数だけ学習を行う。
7     eval.evaluation(model,epoch)
8     model.train()
9     logger.clear_run()
10    print_time=time.time()
11    for i,(images,labels) in enumerate(sub_train_loader,0):
12        images=images.to(device)
13        labels=labels.to(device)
14        outputs=model(images)
15        optimizer.zero_grad()
16        loss=criterion(outputs,labels)
17        loss.backward()
18        optimizer.step()
19        logger.logging(outputs,loss,labels,epoch,i+1)
20    eval.evaluation(model,n_epoch)
21    elapsed_time=time.time()-start
22    print('training total',elapsed_time,'sec.')
```

#データの記録

```

25 finalaccuracy = eval.log["acc"][-1] #accuracy
26 finalloss = eval.log["loss"][-1] # loss

```

図 100 CNN モデル 4 のエポック 15 での学習

CNN モデル 4 の学習を行った。実行結果を図 101,102,103 に示す。

```

100% 15/15 [00:24<00:00, 1.57s/it]
[ 0]test loss: 2.309, accuracy: 4.46%, speed:0.022s/iter
[ 0, 1] loss:2.296, accuracy:4.69%, speed:0.112s/iter
[ 0, 2] loss:4.014, accuracy:18.75%, speed:0.014s/iter
[ 0, 3] loss:2.408, accuracy:6.25%, speed:0.012s/iter
[ 0, 4] loss:2.286, accuracy:14.06%, speed:0.006s/iter
[ 0, 5] loss:2.253, accuracy:29.55%, speed:0.008s/iter
[ 1]test loss: 2.209, accuracy: 18.87%, speed:0.016s/iter
[ 1, 1] loss:2.217, accuracy:11.72%, speed:0.104s/iter
[ 1, 2] loss:2.106, accuracy:41.41%, speed:0.014s/iter
[ 1, 3] loss:1.976, accuracy:53.12%, speed:0.009s/iter
[ 1, 4] loss:1.825, accuracy:51.56%, speed:0.007s/iter
[ 1, 5] loss:1.497, accuracy:73.86%, speed:0.009s/iter
[ 2]test loss: 1.411, accuracy: 58.24%, speed:0.016s/iter
[ 2, 1] loss:1.372, accuracy:61.72%, speed:0.107s/iter
[ 2, 2] loss:1.011, accuracy:73.44%, speed:0.013s/iter
[ 2, 3] loss:0.917, accuracy:69.53%, speed:0.015s/iter
[ 2, 4] loss:0.930, accuracy:70.31%, speed:0.006s/iter
[ 2, 5] loss:0.573, accuracy:79.55%, speed:0.009s/iter
[ 3]test loss: 0.966, accuracy: 69.46%, speed:0.016s/iter
[ 3, 1] loss:0.828, accuracy:70.31%, speed:0.107s/iter
[ 3, 2] loss:0.708, accuracy:78.91%, speed:0.011s/iter
[ 3, 3] loss:0.506, accuracy:82.81%, speed:0.013s/iter
[ 3, 4] loss:0.351, accuracy:90.62%, speed:0.006s/iter
[ 3, 5] loss:0.735, accuracy:75.00%, speed:0.009s/iter
[ 4]test loss: 0.762, accuracy: 77.90%, speed:0.016s/iter
[ 4, 1] loss:0.551, accuracy:82.81%, speed:0.103s/iter
[ 4, 2] loss:0.461, accuracy:88.28%, speed:0.013s/iter
[ 4, 3] loss:0.351, accuracy:87.50%, speed:0.010s/iter
[ 4, 4] loss:0.348, accuracy:89.84%, speed:0.007s/iter
[ 4, 5] loss:0.326, accuracy:89.77%, speed:0.009s/iter
[ 5]test loss: 0.461, accuracy: 86.61%, speed:0.016s/iter
[ 5, 1] loss:0.395, accuracy:89.06%, speed:0.111s/iter
[ 5, 2] loss:0.186, accuracy:96.09%, speed:0.013s/iter
[ 5, 3] loss:0.336, accuracy:87.50%, speed:0.011s/iter
[ 5, 4] loss:0.294, accuracy:92.19%, speed:0.007s/iter
[ 5, 5] loss:0.308, accuracy:90.91%, speed:0.007s/iter

```

図 101 CNN モデル 4 のエポック 15 での学習の出力結果 1

```

[ 6]test loss: 0.380, accuracy: 88.74%, speed:0.016s/iter
[ 6, 1] loss:0.162, accuracy:95.31%, speed:0.111s/iter
[ 6, 2] loss:0.206, accuracy:93.75%, speed:0.013s/iter
[ 6, 3] loss:0.324, accuracy:89.84%, speed:0.009s/iter
[ 6, 4] loss:0.147, accuracy:94.53%, speed:0.012s/iter
[ 6, 5] loss:0.171, accuracy:95.45%, speed:0.006s/iter
[ 7]test loss: 0.352, accuracy: 90.29%, speed:0.022s/iter
[ 7, 1] loss:0.086, accuracy:96.88%, speed:0.149s/iter
[ 7, 2] loss:0.139, accuracy:94.53%, speed:0.038s/iter
[ 7, 3] loss:0.150, accuracy:93.75%, speed:0.011s/iter
[ 7, 4] loss:0.135, accuracy:96.88%, speed:0.021s/iter
[ 7, 5] loss:0.102, accuracy:98.86%, speed:0.005s/iter
[ 8]test loss: 0.347, accuracy: 90.28%, speed:0.024s/iter
[ 8, 1] loss:0.091, accuracy:96.09%, speed:0.106s/iter
[ 8, 2] loss:0.099, accuracy:98.44%, speed:0.013s/iter
[ 8, 3] loss:0.044, accuracy:99.22%, speed:0.013s/iter
[ 8, 4] loss:0.084, accuracy:98.44%, speed:0.006s/iter
[ 8, 5] loss:0.113, accuracy:95.45%, speed:0.009s/iter
[ 9]test loss: 0.370, accuracy: 90.27%, speed:0.017s/iter
[ 9, 1] loss:0.056, accuracy:97.66%, speed:0.112s/iter
[ 9, 2] loss:0.057, accuracy:98.44%, speed:0.013s/iter
[ 9, 3] loss:0.125, accuracy:94.53%, speed:0.010s/iter
[ 9, 4] loss:0.031, accuracy:99.22%, speed:0.008s/iter
[ 9, 5] loss:0.032, accuracy:98.86%, speed:0.010s/iter
[ 10]test loss: 0.342, accuracy: 91.11%, speed:0.017s/iter
[ 10, 1] loss:0.039, accuracy:98.44%, speed:0.120s/iter
[ 10, 2] loss:0.045, accuracy:99.22%, speed:0.017s/iter
[ 10, 3] loss:0.049, accuracy:97.66%, speed:0.012s/iter
[ 10, 4] loss:0.025, accuracy:100.00%, speed:0.006s/iter
[ 10, 5] loss:0.025, accuracy:100.00%, speed:0.010s/iter

```

図 102 CNN モデル 4 のエポック 15 での学習の出力結果 2

```

[ 11]test loss: 0.355, accuracy: 91.10%, speed:0.017s/iter
[ 11,      1] loss:0.015, accuracy:100.00%, speed:0.108s/iter
[ 11,      2] loss:0.015, accuracy:100.00%, speed:0.013s/iter
[ 11,      3] loss:0.024, accuracy:100.00%, speed:0.010s/iter
[ 11,      4] loss:0.004, accuracy:100.00%, speed:0.010s/iter
[ 11,      5] loss:0.016, accuracy:100.00%, speed:0.006s/iter
[ 12]test loss: 0.407, accuracy: 91.31%, speed:0.017s/iter
[ 12,      1] loss:0.017, accuracy:100.00%, speed:0.121s/iter
[ 12,      2] loss:0.006, accuracy:100.00%, speed:0.014s/iter
[ 12,      3] loss:0.005, accuracy:100.00%, speed:0.017s/iter
[ 12,      4] loss:0.009, accuracy:100.00%, speed:0.008s/iter
[ 12,      5] loss:0.003, accuracy:100.00%, speed:0.009s/iter
[ 13]test loss: 0.382, accuracy: 91.81%, speed:0.017s/iter
[ 13,      1] loss:0.005, accuracy:100.00%, speed:0.108s/iter
[ 13,      2] loss:0.004, accuracy:100.00%, speed:0.012s/iter
[ 13,      3] loss:0.005, accuracy:100.00%, speed:0.015s/iter
[ 13,      4] loss:0.005, accuracy:100.00%, speed:0.006s/iter
[ 13,      5] loss:0.004, accuracy:100.00%, speed:0.006s/iter
[ 14]test loss: 0.389, accuracy: 91.73%, speed:0.017s/iter
[ 14,      1] loss:0.005, accuracy:100.00%, speed:0.111s/iter
[ 14,      2] loss:0.002, accuracy:100.00%, speed:0.015s/iter
[ 14,      3] loss:0.001, accuracy:100.00%, speed:0.009s/iter
[ 14,      4] loss:0.002, accuracy:100.00%, speed:0.006s/iter
[ 14,      5] loss:0.003, accuracy:100.00%, speed:0.017s/iter
[ 15]test loss: 0.393, accuracy: 92.22%, speed:0.024s/iter
training total 25.991215705871582 sec.

```

図 103 CNN モデル 4 のエポック 15 での学習の出力結果 3

学習の結果を可視化した。実行結果を図 104,105 に示す。

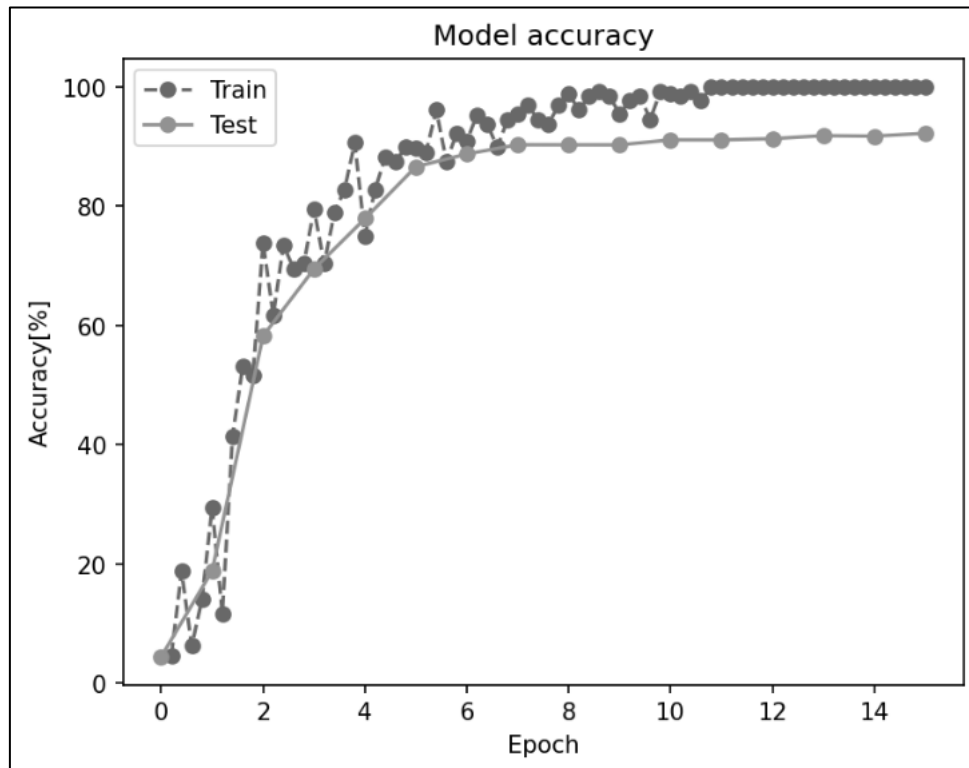


図 104 CNN モデル 4 のエポック 15 でのエポックと精度のグラフ

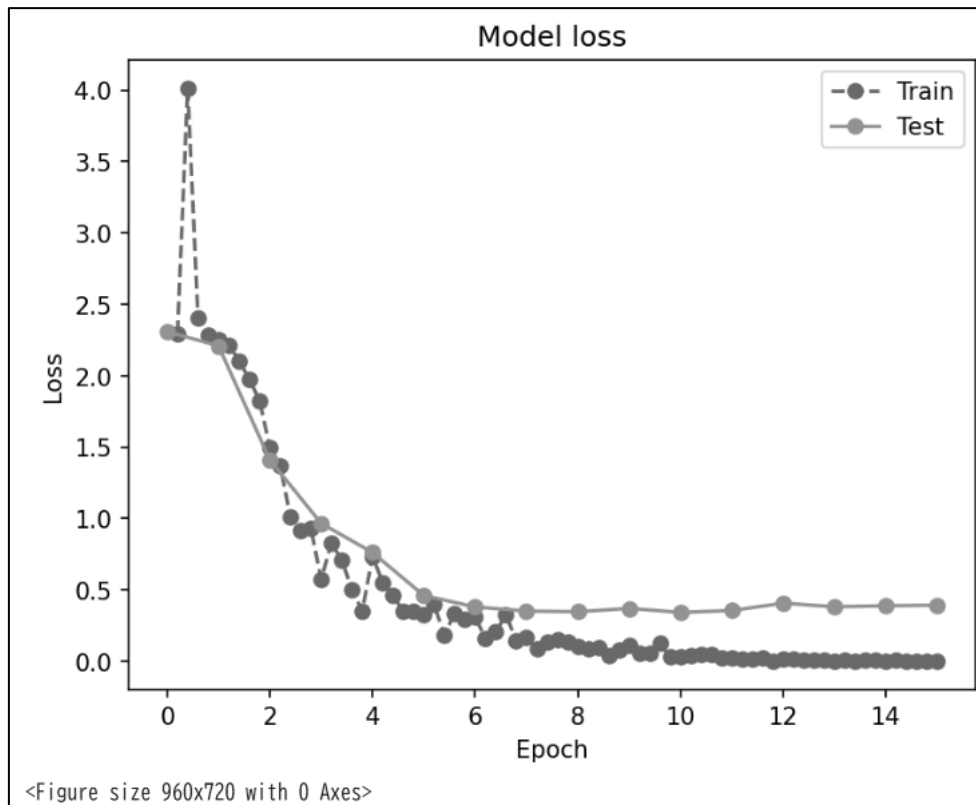


図 105 CNN モデル 4 のエポック 15 でのエポックと損失のグラフ

学習データの画像を回転させ、入力データの拡張を行った。コードを図 106 に示す。

```

1 image_size = 28
2 transform = T.Compose(
3     [T.Resize((image_size, image_size)), #画像サイズを28x28にリサイズ
4     T.ToTensor()] #Pytorchのテンソルに変換し、値を0.0~1.0にスケーリング
5 )
6 train_transform = T.Compose([
7     T.Resize((image_size, image_size)),
8     T.RandomRotation(5, fill=0), #回転を利用した学習用のデータ拡張
9     T.ToTensor()
10 ])
11 #MNISTから学習データ、テストデータを取得
12 train_dataset = datasets.MNIST(root='./data', train=True,
13                                transform=train_transform, download = True)
14 test_dataset = datasets.MNIST(root='./data', train=False,
15                               transform=transform, download = True)
16 train_samples = len(train_dataset) #学習データのサンプル数
17 classes = [key for key in test_dataset.class_to_idx] #テストデータセットから正解ラベルとクラス名を取り出す
18 print("train dataset: %n", train_dataset, "%n")
19 print("test dataset: %n", test_dataset, "%n")
20 print("dataset classes: %n", classes)

```

図 106 回転を用いた MNIST データの前処理

実行結果を図 107 に示す。

```
train dataset: Dataset MNIST
  Number of datapoints: 60000
  Root location: ./data
  Split: Train
  StandardTransform
Transform: Compose(
  Resize(size=(28, 28), interpolation=bilinear, max_size=None, antialias=True)
  RandomRotation(degrees=[-5.0, 5.0], interpolation=nearest, expand=False, fill=0)
  ToTensor()
)

test dataset:
Dataset MNIST
  Number of datapoints: 10000
  Root location: ./data
  Split: Test
  StandardTransform
Transform: Compose(
  Resize(size=(28, 28), interpolation=bilinear, max_size=None, antialias=True)
  ToTensor()
)

dataset classes:
['0 - zero', '1 - one', '2 - two', '3 - three', '4 - four', '5 - five', '6 - six', '7 - seven', '8 - eight', '9 - nine']
```

図 107 MNIST データの前処理の出力結果

train_dataset に RandomRotation が含まれていることが確認できる。

拡張した学習データを用いて、CNN モデル 4 の学習を行った。実行結果を図 108,109 に示す。

```
100% ██████████ 10/10 [00:18<00:00, 1.88s/it]
[ 0]test loss: 2.309, accuracy: 4.46%, speed:0.021s/iter
[ 0, 1] loss:2.295, accuracy:4.69%, speed:0.129s/iter
[ 0, 2] loss:3.807, accuracy:12.50%, speed:0.016s/iter
[ 0, 3] loss:2.378, accuracy:7.03%, speed:0.040s/iter
[ 0, 4] loss:2.302, accuracy:10.16%, speed:0.013s/iter
[ 0, 5] loss:2.304, accuracy:9.09%, speed:0.024s/iter
[ 1]test loss: 2.303, accuracy: 10.09%, speed:0.027s/iter
[ 1, 1] loss:2.307, accuracy:4.69%, speed:0.170s/iter
[ 1, 2] loss:2.300, accuracy:11.72%, speed:0.052s/iter
[ 1, 3] loss:2.298, accuracy:12.50%, speed:0.030s/iter
[ 1, 4] loss:2.300, accuracy:7.81%, speed:0.041s/iter
[ 1, 5] loss:2.282, accuracy:13.64%, speed:0.006s/iter
[ 2]test loss: 2.276, accuracy: 10.09%, speed:0.017s/iter
[ 2, 1] loss:2.298, accuracy:5.47%, speed:0.131s/iter
[ 2, 2] loss:2.250, accuracy:9.38%, speed:0.012s/iter
[ 2, 3] loss:2.214, accuracy:14.84%, speed:0.026s/iter
[ 2, 4] loss:2.191, accuracy:31.25%, speed:0.006s/iter
[ 2, 5] loss:2.170, accuracy:14.77%, speed:0.014s/iter
[ 3]test loss: 2.089, accuracy: 39.55%, speed:0.018s/iter
[ 3, 1] loss:2.058, accuracy:38.28%, speed:0.126s/iter
[ 3, 2] loss:1.959, accuracy:55.47%, speed:0.015s/iter
[ 3, 3] loss:1.916, accuracy:40.62%, speed:0.019s/iter
[ 3, 4] loss:1.731, accuracy:61.72%, speed:0.008s/iter
[ 3, 5] loss:1.565, accuracy:56.82%, speed:0.016s/iter
[ 4]test loss: 1.435, accuracy: 58.38%, speed:0.016s/iter
[ 4, 1] loss:1.372, accuracy:63.28%, speed:0.127s/iter
[ 4, 2] loss:1.178, accuracy:71.09%, speed:0.013s/iter
[ 4, 3] loss:1.044, accuracy:75.00%, speed:0.029s/iter
[ 4, 4] loss:0.914, accuracy:73.44%, speed:0.006s/iter
[ 4, 5] loss:1.063, accuracy:65.91%, speed:0.017s/iter
[ 5]test loss: 1.057, accuracy: 64.99%, speed:0.017s/iter
[ 5, 1] loss:0.861, accuracy:68.75%, speed:0.126s/iter
[ 5, 2] loss:0.564, accuracy:79.69%, speed:0.010s/iter
[ 5, 3] loss:0.853, accuracy:74.22%, speed:0.031s/iter
[ 5, 4] loss:0.890, accuracy:73.44%, speed:0.008s/iter
[ 5, 5] loss:1.103, accuracy:68.18%, speed:0.015s/iter
```

図 108 拡張した学習データを用いた CNN モデル 4 の学習の出力結果前半部分

```

[ 6]test loss: 0.828, accuracy: 74.08%, speed:0.018s/iter
[ 6, 1] loss:0.486, accuracy:83.59%, speed:0.129s/iter
[ 6, 2] loss:0.764, accuracy:72.66%, speed:0.015s/iter
[ 6, 3] loss:0.880, accuracy:74.22%, speed:0.019s/iter
[ 6, 4] loss:0.689, accuracy:71.88%, speed:0.007s/iter
[ 6, 5] loss:0.684, accuracy:77.27%, speed:0.016s/iter
[ 7]test loss: 0.734, accuracy: 76.06%, speed:0.017s/iter
[ 7, 1] loss:0.416, accuracy:83.59%, speed:0.129s/iter
[ 7, 2] loss:0.589, accuracy:83.59%, speed:0.013s/iter
[ 7, 3] loss:0.643, accuracy:81.25%, speed:0.023s/iter
[ 7, 4] loss:0.811, accuracy:75.78%, speed:0.008s/iter
[ 7, 5] loss:0.479, accuracy:87.50%, speed:0.016s/iter
[ 8]test loss: 0.601, accuracy: 81.55%, speed:0.023s/iter
[ 8, 1] loss:0.455, accuracy:88.28%, speed:0.213s/iter
[ 8, 2] loss:0.436, accuracy:85.16%, speed:0.026s/iter
[ 8, 3] loss:0.526, accuracy:79.69%, speed:0.039s/iter
[ 8, 4] loss:0.594, accuracy:82.03%, speed:0.011s/iter
[ 8, 5] loss:0.405, accuracy:82.95%, speed:0.020s/iter
[ 9]test loss: 0.609, accuracy: 81.00%, speed:0.024s/iter
[ 9, 1] loss:0.397, accuracy:85.16%, speed:0.123s/iter
[ 9, 2] loss:0.342, accuracy:89.84%, speed:0.012s/iter
[ 9, 3] loss:0.501, accuracy:84.38%, speed:0.021s/iter
[ 9, 4] loss:0.323, accuracy:90.62%, speed:0.011s/iter
[ 9, 5] loss:0.276, accuracy:90.91%, speed:0.012s/iter
[10]test loss: 0.608, accuracy: 81.35%, speed:0.017s/iter
training total 19.569236993789673 sec.

```

図 109 拡張した学習データを用いた CNN モデル 4 の学習の出力結果後半部分

学習の結果を可視化した。実行結果を図 110,111 に示す。

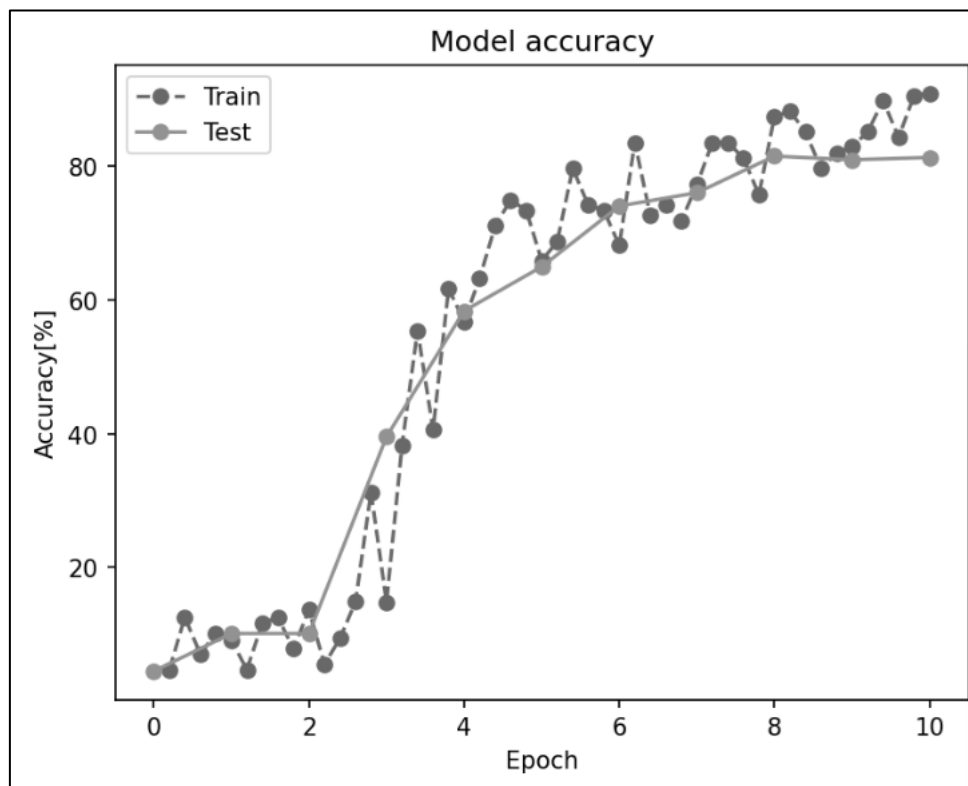


図 110 拡張した学習データを用いた CNN モデル 4 のエポックと精度のグラフ

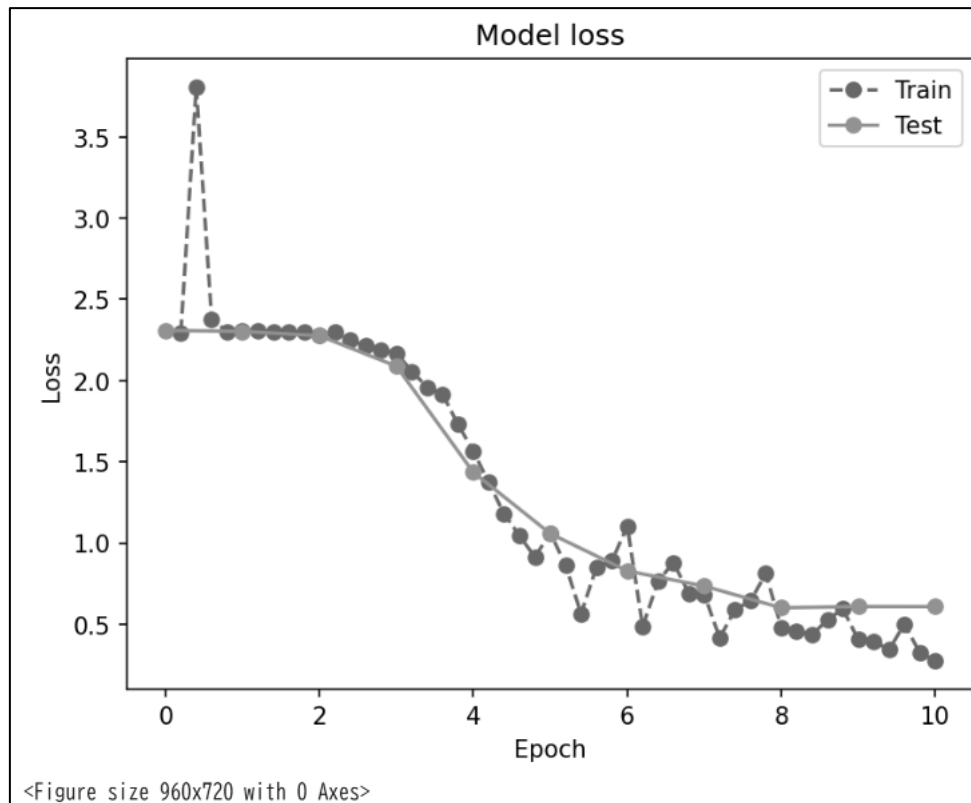


図 111 拡張した学習データを用いた CNN モデル 4 のエポックと損失のグラフ

エポックを 40、プリントイテレーションを 5 に変更した。コードおよび実行結果を図 112,113,114,115 に示す。

```

1 n_epoch=40#エポック数
2 print_iteration=5#学習時のロギング間隔
3 logger=Logger(print_iteration,train_samples,batch_size)
4 eval=Evaluator(test_loader,criterion,device)
5 start=time.time()
6 for epoch in tqdm(range(n_epoch)):#エポック数だけ学習を行う。
7     eval.evaluation(model,epoch)
8     model.train()
9     logger.clear_run()
10    print_time=time.time()
11    for i,(images,labels) in enumerate(sub_train_loader,0):
12        images=images.to(device)
13        labels=labels.to(device)
14        outputs=model(images)
15        optimizer.zero_grad()
16        loss=criterion(outputs,labels)
17        loss.backward()
18        optimizer.step()
19        logger.logging(outputs,loss,labels,epoch,i+1)
20    eval.evaluation(model,n_epoch)
21    elapsed_time=time.time()-start
22    print('training total',elapsed_time,'sec.')#合計の処理時間
23
24 #データの記録
25 finalaccuracy = eval.log["acc"][-1] #accuracy
26 finalloss = eval.log["loss"][-1] # loss

```

図 112 エポックを 40、プリントイテレーションを 5 にした学習のコード

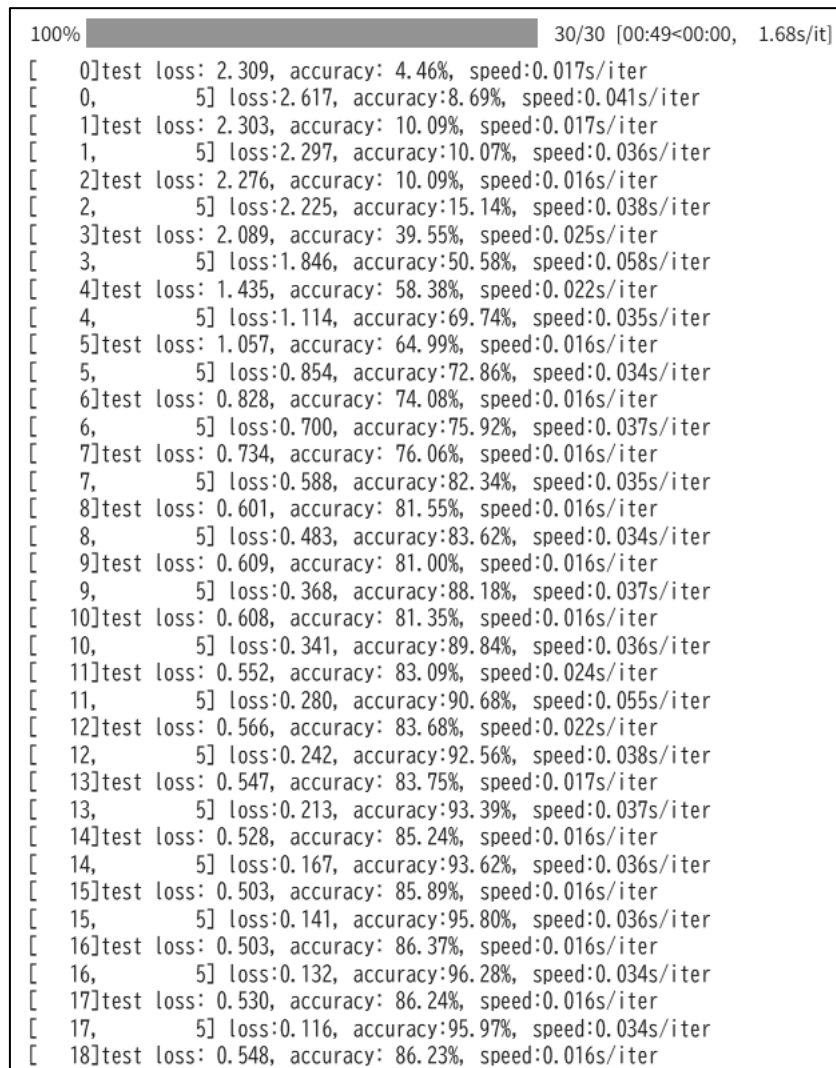


図 113 拡張した学習データを用いた CNN モデル 4 の図 112 の学習 1

```

[ 18,      5] loss:0.090, accuracy:97.81%, speed:0.034s/iter
[ 19]test loss: 0.548, accuracy: 86.33%, speed:0.016s/iter
[ 19,      5] loss:0.085, accuracy:97.22%, speed:0.034s/iter
[ 20]test loss: 0.548, accuracy: 86.89%, speed:0.024s/iter
[ 20,      5] loss:0.078, accuracy:97.97%, speed:0.053s/iter
[ 21]test loss: 0.577, accuracy: 86.77%, speed:0.018s/iter
[ 21,      5] loss:0.042, accuracy:98.84%, speed:0.034s/iter
[ 22]test loss: 0.569, accuracy: 87.12%, speed:0.016s/iter
[ 22,      5] loss:0.043, accuracy:98.75%, speed:0.034s/iter
[ 23]test loss: 0.595, accuracy: 87.40%, speed:0.016s/iter
[ 23,      5] loss:0.040, accuracy:98.68%, speed:0.034s/iter
[ 24]test loss: 0.642, accuracy: 86.52%, speed:0.021s/iter
[ 24,      5] loss:0.035, accuracy:98.91%, speed:0.057s/iter
[ 25]test loss: 0.640, accuracy: 87.16%, speed:0.019s/iter
[ 25,      5] loss:0.040, accuracy:98.91%, speed:0.036s/iter
[ 26]test loss: 0.627, accuracy: 87.39%, speed:0.016s/iter
[ 26,      5] loss:0.023, accuracy:99.22%, speed:0.034s/iter
[ 27]test loss: 0.640, accuracy: 87.01%, speed:0.020s/iter
[ 27,      5] loss:0.035, accuracy:98.91%, speed:0.058s/iter
[ 28]test loss: 0.652, accuracy: 86.86%, speed:0.023s/iter
[ 28,      5] loss:0.030, accuracy:99.08%, speed:0.036s/iter
[ 29]test loss: 0.674, accuracy: 87.33%, speed:0.016s/iter
[ 29,      5] loss:0.032, accuracy:99.08%, speed:0.033s/iter
[ 30]test loss: 0.719, accuracy: 87.26%, speed:0.016s/iter
[ 30,      5] loss:0.030, accuracy:99.39%, speed:0.036s/iter
[ 31]test loss: 0.732, accuracy: 86.63%, speed:0.017s/iter
[ 31,      5] loss:0.076, accuracy:98.54%, speed:0.035s/iter
[ 32]test loss: 0.723, accuracy: 86.48%, speed:0.016s/iter
[ 32,      5] loss:0.063, accuracy:98.12%, speed:0.035s/iter
[ 33]test loss: 0.729, accuracy: 86.95%, speed:0.018s/iter
[ 33,      5] loss:0.066, accuracy:97.81%, speed:0.035s/iter
[ 34]test loss: 0.711, accuracy: 86.29%, speed:0.016s/iter
[ 34,      5] loss:0.038, accuracy:98.99%, speed:0.034s/iter
[ 35]test loss: 0.663, accuracy: 87.05%, speed:0.022s/iter
[ 35,      5] loss:0.031, accuracy:98.99%, speed:0.054s/iter
[ 36]test loss: 0.666, accuracy: 86.77%, speed:0.022s/iter
[ 36,      5] loss:0.023, accuracy:99.46%, speed:0.035s/iter
[ 37]test loss: 0.669, accuracy: 87.32%, speed:0.016s/iter

```

図 114 拡張した学習データを用いた CNN モデル 4 の図 112 の学習 2

```

[ 37,      5] loss:0.015, accuracy:99.53%, speed:0.033s/iter
[ 38]test loss: 0.609, accuracy: 88.14%, speed:0.016s/iter
[ 38,      5] loss:0.008, accuracy:100.00%, speed:0.035s/iter
[ 39]test loss: 0.623, accuracy: 88.57%, speed:0.016s/iter
[ 39,      5] loss:0.005, accuracy:100.00%, speed:0.035s/iter
[ 40]test loss: 0.702, accuracy: 88.10%, speed:0.016s/iter
training total 66.60954999923706 sec.

```

図 115 拡張した学習データを用いた CNN モデル 4 の図 112 の学習 3

学習の結果を可視化した。実行結果を図 116,117 に示す

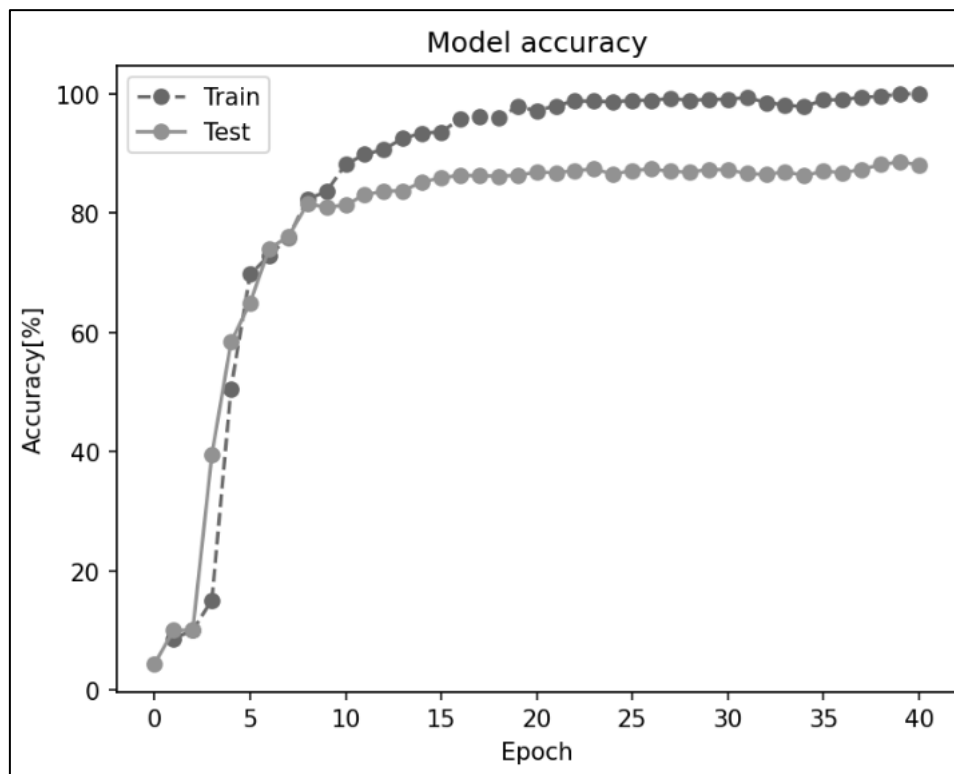


図 116 図 112 のコードと拡張した学習データを用いた CNN モデル 4 のエポックと精度のグラフ

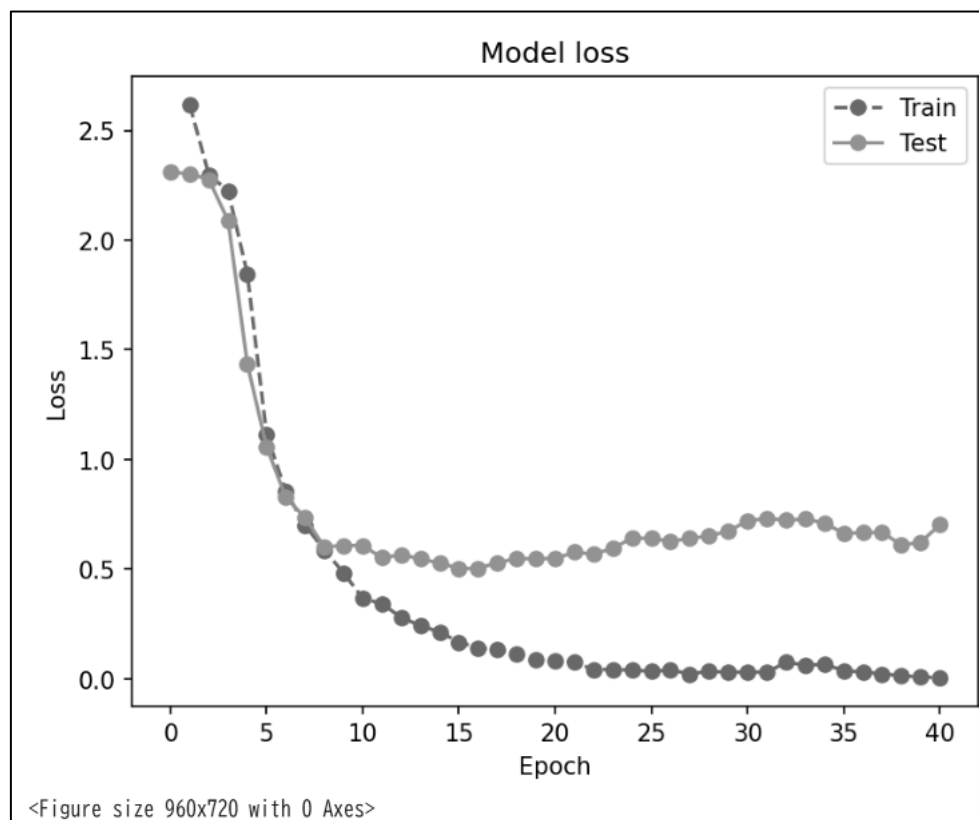


図 117 図 112 のコードと拡張した学習データを用いた CNN モデル 4 のエポックと損失のグラフ

CNN モデル 4 にドロップアウト層を追加した CNN モデル 5 を定義した。コードを図 118 に示す。

```
1 class MyCNN(nn.Module):
2     def __init__(self):
3         super(MyCNN, self).__init__()
4         self.conv1 = nn.Conv2d(1, 50, (5, 5))
5         self.conv2 = nn.Conv2d(50, 100, (3, 3), padding=1)
6         self.fc1 = nn.Linear(6*6*100, 100)
7         #self.fc1 = nn.Linear(12*12*50, 100)
8         self.fc2 = nn.Linear(100, 10)
9         self.pool = nn.MaxPool2d((2,2))
10        self.flat = nn.Flatten()
11        self.drop = nn.Dropout(0.5)
12        self.relu = nn.ReLU()
13    def forward(self, x):
14        h = self.relu(self.conv1(x))#畳み込み層の後ReLU関数により活性化
15        h = self.pool(h)#Maxプーリング層
16        h = self.relu(self.conv2(h))#畳み込み層の後ReLU関数により活性化3.2 ⑦精度向上のために追加
17        h = self.pool(h)#Maxプーリング層 3.2 ⑦精度向上のために追加
18        h = self.flat(h)#平坦化
19        h = self.relu(self.fc1(h))#全結合層の後ReLU関数により活性化
20        h = self.drop(h)#ドロップアウト 3.2 ⑦精度向上のために追加
21        y = self.fc2(h)#全結合層の後ReLU関数により活性化
22        return y
23    def predict(self, x):
24        y = self.forward(x)
25        return F.softmax(y, dim=1)#softmax関数を通して推論結果を出力
```

図 118 CNN モデル 5 の定義

図 112 を用いて拡張した学習データを用いた CNN モデル 5 の学習を行った。実行結果を図 119,120,121 に示す。

```
100% ██████████ 40/40 [01:07<00:00, 1.62s/it]
[ 0]test loss: 2.309, accuracy: 4.46%, speed:0.018s/iter
[ 0,      5] loss:2.939, accuracy:9.86%, speed:0.036s/iter
[ 1]test loss: 2.303, accuracy: 10.09%, speed:0.016s/iter
[ 1,      5] loss:2.295, accuracy:13.12%, speed:0.038s/iter
[ 2]test loss: 2.274, accuracy: 11.35%, speed:0.016s/iter
[ 2,      5] loss:2.232, accuracy:15.84%, speed:0.038s/iter
[ 3]test loss: 2.137, accuracy: 23.93%, speed:0.023s/iter
[ 3,      5] loss:2.017, accuracy:31.63%, speed:0.056s/iter
[ 4]test loss: 1.700, accuracy: 44.24%, speed:0.022s/iter
[ 4,      5] loss:1.659, accuracy:41.31%, speed:0.037s/iter
[ 5]test loss: 1.258, accuracy: 56.86%, speed:0.016s/iter
[ 5,      5] loss:1.376, accuracy:53.49%, speed:0.039s/iter
[ 6]test loss: 1.071, accuracy: 67.93%, speed:0.018s/iter
[ 6,      5] loss:1.082, accuracy:65.36%, speed:0.038s/iter
[ 7]test loss: 0.897, accuracy: 72.49%, speed:0.018s/iter
[ 7,      5] loss:0.975, accuracy:68.03%, speed:0.038s/iter
[ 8]test loss: 0.740, accuracy: 77.23%, speed:0.017s/iter
[ 8,      5] loss:0.825, accuracy:72.19%, speed:0.037s/iter
[ 9]test loss: 0.633, accuracy: 79.69%, speed:0.017s/iter
[ 9,      5] loss:0.730, accuracy:75.03%, speed:0.039s/iter
[10]test loss: 0.537, accuracy: 82.95%, speed:0.017s/iter
[10,      5] loss:0.636, accuracy:77.50%, speed:0.055s/iter
[11]test loss: 0.536, accuracy: 82.85%, speed:0.027s/iter
[11,      5] loss:0.608, accuracy:79.33%, speed:0.042s/iter
[12]test loss: 0.454, accuracy: 85.55%, speed:0.016s/iter
[12,      5] loss:0.554, accuracy:80.53%, speed:0.034s/iter
[13]test loss: 0.458, accuracy: 85.66%, speed:0.018s/iter
[13,      5] loss:0.535, accuracy:80.97%, speed:0.035s/iter
[14]test loss: 0.421, accuracy: 86.71%, speed:0.017s/iter
[14,      5] loss:0.440, accuracy:84.56%, speed:0.037s/iter
[15]test loss: 0.376, accuracy: 88.82%, speed:0.017s/iter
[15,      5] loss:0.418, accuracy:84.33%, speed:0.035s/iter
[16]test loss: 0.389, accuracy: 88.16%, speed:0.018s/iter
[16,      5] loss:0.391, accuracy:86.46%, speed:0.036s/iter
[17]test loss: 0.366, accuracy: 88.86%, speed:0.017s/iter
```

図 119 拡張した学習データを用いた CNN モデル 5 の図 112 の学習 1

```

[ 17,      5] loss:0.457, accuracy:84.89%, speed:0.038s/iter
[ 18]test loss: 0.361, accuracy: 89.15%, speed:0.024s/iter
[ 18,      5] loss:0.380, accuracy:88.51%, speed:0.057s/iter
[ 19]test loss: 0.380, accuracy: 88.18%, speed:0.022s/iter
[ 19,      5] loss:0.352, accuracy:87.95%, speed:0.035s/iter
[ 20]test loss: 0.344, accuracy: 89.91%, speed:0.016s/iter
[ 20,      5] loss:0.327, accuracy:88.37%, speed:0.037s/iter
[ 21]test loss: 0.332, accuracy: 90.22%, speed:0.017s/iter
[ 21,      5] loss:0.285, accuracy:90.53%, speed:0.034s/iter
[ 22]test loss: 0.372, accuracy: 89.90%, speed:0.017s/iter
[ 22,      5] loss:0.297, accuracy:90.77%, speed:0.035s/iter
[ 23]test loss: 0.324, accuracy: 90.60%, speed:0.017s/iter
[ 23,      5] loss:0.256, accuracy:90.70%, speed:0.036s/iter
[ 24]test loss: 0.357, accuracy: 89.93%, speed:0.017s/iter
[ 24,      5] loss:0.281, accuracy:89.76%, speed:0.034s/iter
[ 25]test loss: 0.317, accuracy: 90.50%, speed:0.017s/iter
[ 25,      5] loss:0.234, accuracy:93.00%, speed:0.033s/iter
[ 26]test loss: 0.326, accuracy: 91.00%, speed:0.026s/iter
[ 26,      5] loss:0.264, accuracy:89.30%, speed:0.058s/iter
[ 27]test loss: 0.339, accuracy: 90.84%, speed:0.019s/iter
[ 27,      5] loss:0.225, accuracy:92.23%, speed:0.035s/iter
[ 28]test loss: 0.356, accuracy: 90.57%, speed:0.016s/iter
[ 28,      5] loss:0.215, accuracy:92.02%, speed:0.037s/iter
[ 29]test loss: 0.369, accuracy: 90.21%, speed:0.017s/iter
[ 29,      5] loss:0.218, accuracy:92.24%, speed:0.037s/iter
[ 30]test loss: 0.316, accuracy: 91.44%, speed:0.016s/iter
[ 30,      5] loss:0.236, accuracy:90.36%, speed:0.036s/iter
[ 31]test loss: 0.337, accuracy: 91.29%, speed:0.018s/iter
[ 31,      5] loss:0.200, accuracy:91.93%, speed:0.035s/iter
[ 32]test loss: 0.334, accuracy: 91.22%, speed:0.016s/iter
[ 32,      5] loss:0.214, accuracy:92.46%, speed:0.037s/iter
[ 33]test loss: 0.333, accuracy: 90.95%, speed:0.018s/iter
[ 33,      5] loss:0.206, accuracy:91.95%, speed:0.056s/iter
[ 34]test loss: 0.308, accuracy: 91.88%, speed:0.026s/iter
[ 34,      5] loss:0.185, accuracy:92.26%, speed:0.053s/iter
[ 35]test loss: 0.360, accuracy: 91.60%, speed:0.017s/iter
[ 35,      5] loss:0.206, accuracy:93.17%, speed:0.041s/iter

```

図 120 拡張した学習データを用いた CNN モデル 5 の図 112 の学習 2

```

[ 36]test loss: 0.332, accuracy: 91.80%, speed:0.016s/iter
[ 36,      5] loss:0.202, accuracy:92.77%, speed:0.035s/iter
[ 37]test loss: 0.352, accuracy: 91.58%, speed:0.017s/iter
[ 37,      5] loss:0.175, accuracy:93.42%, speed:0.040s/iter
[ 38]test loss: 0.332, accuracy: 91.63%, speed:0.017s/iter
[ 38,      5] loss:0.146, accuracy:94.18%, speed:0.036s/iter
[ 39]test loss: 0.335, accuracy: 91.54%, speed:0.017s/iter
[ 39,      5] loss:0.159, accuracy:95.04%, speed:0.039s/iter
[ 40]test loss: 0.337, accuracy: 91.87%, speed:0.016s/iter
training total 68.59521269798279 sec.

```

図 121 拡張した学習データを用いた CNN モデル 5 の図 112 の学習 3

学習の結果を可視化した。実行結果を図 122,123 に示す。

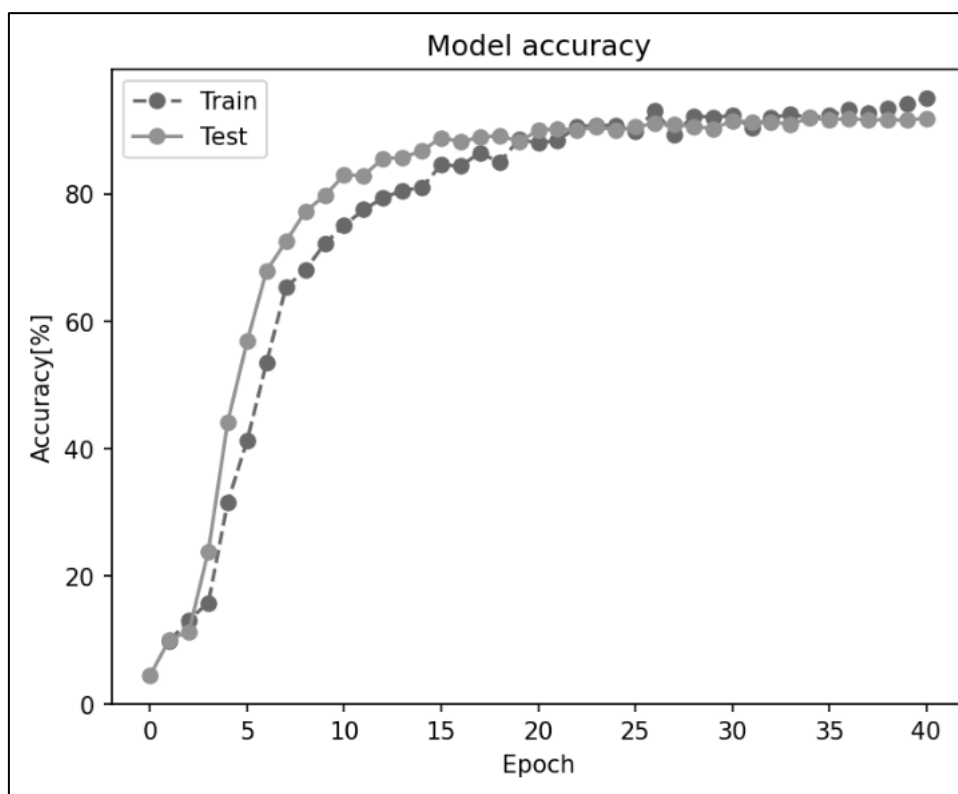


図 122 図 112 の学習の拡張した学習データを用いた CNN モデル 5 のエポックと精度のグラフ

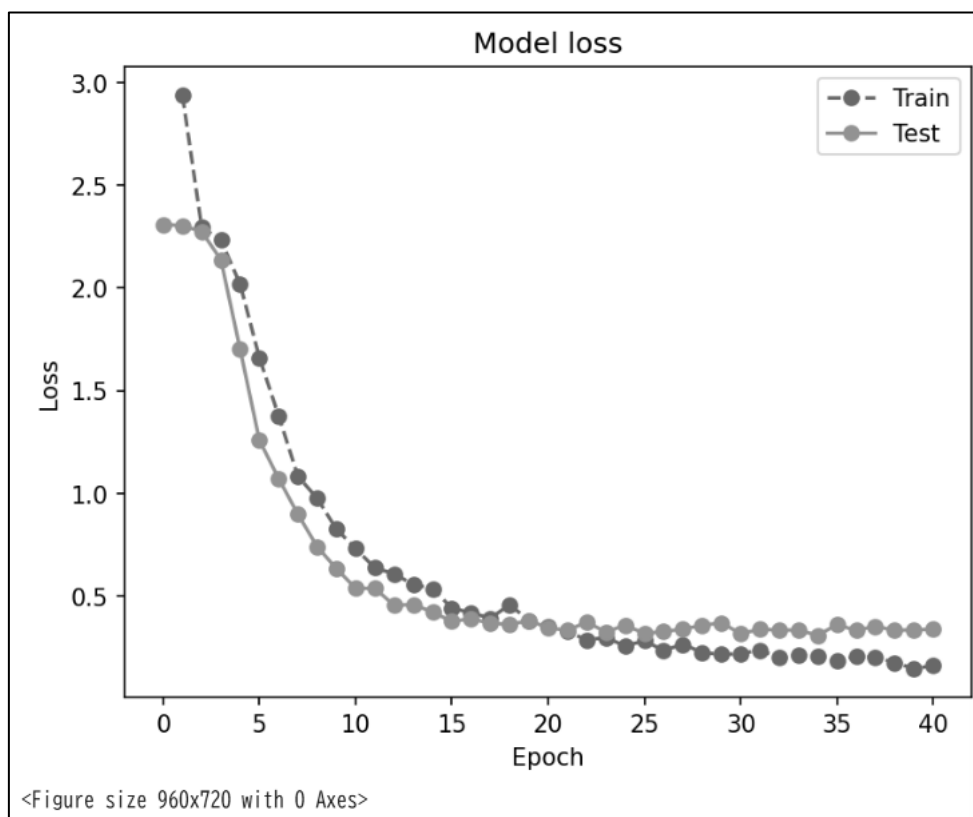


図 123 図 112 の学習の拡張した学習データを用いた CNN モデル 5 のエポックと損失のグラフ

学習率を 0.01 から 0.001 に変更し、同じ条件で学習を行った。コードを図 124 に示す。

```
1 learning_rate = 0.001
2 init_seed(seed) #seedの値は任意に設定する
3 device = 'cuda' if torch.cuda.is_available() else 'cpu'
4 print('use device:', device)
5 model = MyCNN().to(device)
6 # image_size変数をtransform作成時に予め定義しておく
7 summary(model, (1, image_size, image_size), device=device)
8 criterion = nn.CrossEntropyLoss() #交差エントロピー誤差の計算
9 #optimizer = optim.SGD(model.parameters(), lr=learning_rate) #最適化手法SDG 3.2 ③の時はコメントアウトしておく
10 optimizer = optim.Adam(model.parameters(), lr=learning_rate) #最適化手法Adam 3.2 ③の時だけ使用する
11 print('optimizer', optimizer)
```

図 124 学習率 0.001 の場合のコード

実行結果を図 125,126,127 に示す。

```
100% ██████████ 40/40 [01:10<00:00, 1.85s/it]
[ 0]test loss: 2.309, accuracy: 4.46%, speed:0.038s/iter
[ 0,      5] loss:2.255, accuracy:17.70%, speed:0.059s/iter
[ 1]test loss: 2.085, accuracy: 53.39%, speed:0.026s/iter
[ 1,      5] loss:1.913, accuracy:41.89%, speed:0.038s/iter
[ 2]test loss: 1.574, accuracy: 66.86%, speed:0.017s/iter
[ 2,      5] loss:1.446, accuracy:54.26%, speed:0.037s/iter
[ 3]test loss: 1.101, accuracy: 70.53%, speed:0.017s/iter
[ 3,      5] loss:1.090, accuracy:67.77%, speed:0.035s/iter
[ 4]test loss: 0.843, accuracy: 75.73%, speed:0.017s/iter
[ 4,      5] loss:0.901, accuracy:72.36%, speed:0.037s/iter
[ 5]test loss: 0.706, accuracy: 79.53%, speed:0.018s/iter
[ 5,      5] loss:0.762, accuracy:74.94%, speed:0.039s/iter
[ 6]test loss: 0.628, accuracy: 79.70%, speed:0.018s/iter
[ 6,      5] loss:0.665, accuracy:79.26%, speed:0.038s/iter
[ 7]test loss: 0.535, accuracy: 84.05%, speed:0.017s/iter
[ 7,      5] loss:0.549, accuracy:82.63%, speed:0.049s/iter
[ 8]test loss: 0.482, accuracy: 85.07%, speed:0.025s/iter
[ 8,      5] loss:0.501, accuracy:85.24%, speed:0.056s/iter
[ 9]test loss: 0.425, accuracy: 87.37%, speed:0.018s/iter
[ 9,      5] loss:0.415, accuracy:87.88%, speed:0.036s/iter
[10]test loss: 0.396, accuracy: 88.29%, speed:0.016s/iter
[10,      5] loss:0.408, accuracy:89.15%, speed:0.038s/iter
[11]test loss: 0.358, accuracy: 89.37%, speed:0.016s/iter
[11,      5] loss:0.335, accuracy:89.28%, speed:0.041s/iter
[12]test loss: 0.335, accuracy: 89.98%, speed:0.017s/iter
[12,      5] loss:0.318, accuracy:89.18%, speed:0.035s/iter
[13]test loss: 0.329, accuracy: 90.15%, speed:0.017s/iter
[13,      5] loss:0.281, accuracy:91.43%, speed:0.044s/iter
[14]test loss: 0.330, accuracy: 90.22%, speed:0.017s/iter
[14,      5] loss:0.259, accuracy:93.34%, speed:0.036s/iter
[15]test loss: 0.294, accuracy: 91.14%, speed:0.019s/iter
[15,      5] loss:0.221, accuracy:93.54%, speed:0.054s/iter
[16]test loss: 0.278, accuracy: 91.64%, speed:0.026s/iter
[16,      5] loss:0.226, accuracy:92.57%, speed:0.039s/iter
[17]test loss: 0.275, accuracy: 91.59%, speed:0.017s/iter
[17,      5] loss:0.217, accuracy:92.63%, speed:0.037s/iter
[18]test loss: 0.287, accuracy: 91.53%, speed:0.016s/iter
```

図 125 拡張した学習データを用いた CNN モデル 5 の学習率 0.001、エポック 40 の学習 1

```

[ 18,      5] loss:0.180, accuracy:94.23%, speed:0.036s/iter
[ 19]test loss: 0.275, accuracy: 91.71%, speed:0.017s/iter
[ 19,      5] loss:0.163, accuracy:94.87%, speed:0.037s/iter
[ 20]test loss: 0.289, accuracy: 91.60%, speed:0.017s/iter
[ 20,      5] loss:0.153, accuracy:94.49%, speed:0.035s/iter
[ 21]test loss: 0.270, accuracy: 91.81%, speed:0.017s/iter
[ 21,      5] loss:0.156, accuracy:95.26%, speed:0.039s/iter
[ 22]test loss: 0.250, accuracy: 92.30%, speed:0.017s/iter
[ 22,      5] loss:0.126, accuracy:96.58%, speed:0.036s/iter
[ 23]test loss: 0.259, accuracy: 92.24%, speed:0.024s/iter
[ 23,      5] loss:0.110, accuracy:97.59%, speed:0.056s/iter
[ 24]test loss: 0.262, accuracy: 92.29%, speed:0.023s/iter
[ 24,      5] loss:0.133, accuracy:95.57%, speed:0.037s/iter
[ 25]test loss: 0.266, accuracy: 92.34%, speed:0.017s/iter
[ 25,      5] loss:0.098, accuracy:97.20%, speed:0.035s/iter
[ 26]test loss: 0.265, accuracy: 92.63%, speed:0.017s/iter
[ 26,      5] loss:0.087, accuracy:97.97%, speed:0.035s/iter
[ 27]test loss: 0.282, accuracy: 92.34%, speed:0.017s/iter
[ 27,      5] loss:0.129, accuracy:95.80%, speed:0.035s/iter
[ 28]test loss: 0.273, accuracy: 92.61%, speed:0.017s/iter
[ 28,      5] loss:0.076, accuracy:97.60%, speed:0.037s/iter
[ 29]test loss: 0.267, accuracy: 92.77%, speed:0.017s/iter
[ 29,      5] loss:0.110, accuracy:95.57%, speed:0.037s/iter
[ 30]test loss: 0.277, accuracy: 92.61%, speed:0.017s/iter
[ 30,      5] loss:0.095, accuracy:97.90%, speed:0.057s/iter
[ 31]test loss: 0.259, accuracy: 92.67%, speed:0.026s/iter
[ 31,      5] loss:0.071, accuracy:98.30%, speed:0.057s/iter
[ 32]test loss: 0.268, accuracy: 92.76%, speed:0.017s/iter
[ 32,      5] loss:0.079, accuracy:96.75%, speed:0.042s/iter
[ 33]test loss: 0.252, accuracy: 93.12%, speed:0.018s/iter
[ 33,      5] loss:0.073, accuracy:97.67%, speed:0.037s/iter
[ 34]test loss: 0.252, accuracy: 93.26%, speed:0.018s/iter
[ 34,      5] loss:0.057, accuracy:98.52%, speed:0.040s/iter
[ 35]test loss: 0.263, accuracy: 93.23%, speed:0.018s/iter
[ 35,      5] loss:0.073, accuracy:97.90%, speed:0.041s/iter
[ 36]test loss: 0.259, accuracy: 93.15%, speed:0.017s/iter
[ 36,      5] loss:0.091, accuracy:96.66%, speed:0.038s/iter
[ 37]test loss: 0.256, accuracy: 93.03%, speed:0.017s/iter
[ 37,      5] loss:0.057, accuracy:98.76%, speed:0.038s/iter

```

図 126 拡張した学習データを用いた CNN モデル 5 の学習率 0.001、エポック 40 の学習 2

```

[ 38]test loss: 0.266, accuracy: 92.71%, speed:0.024s/iter
[ 38,      5] loss:0.087, accuracy:97.29%, speed:0.057s/iter
[ 39]test loss: 0.276, accuracy: 92.81%, speed:0.022s/iter
[ 39,      5] loss:0.060, accuracy:98.54%, speed:0.039s/iter
[ 40]test loss: 0.234, accuracy: 93.76%, speed:0.016s/iter
training total 71.76435136795044 sec.

```

図 127 拡張した学習データを用いた CNN モデル 5 の学習率 0.001、エポック 40 の学習 3

学習の結果を可視化した。実行結果を図 128,129 に示す。

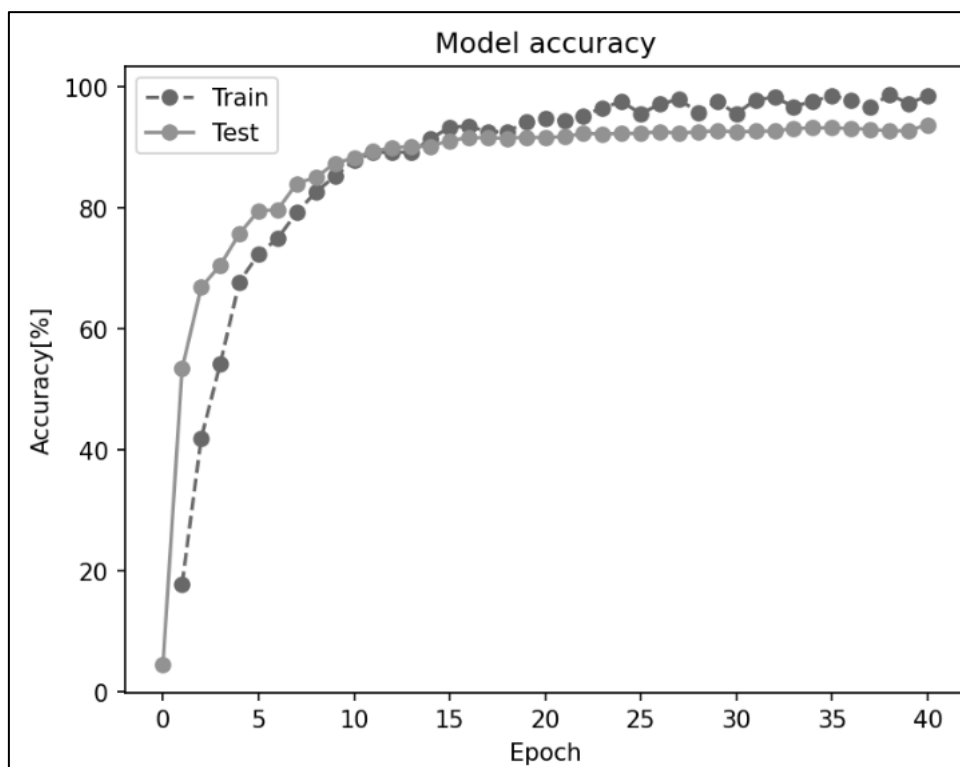


図 128 学習率 0.001、エポック 40、拡張した学習データを用いた CNN モデル 5 のエポックと精度のグラフ

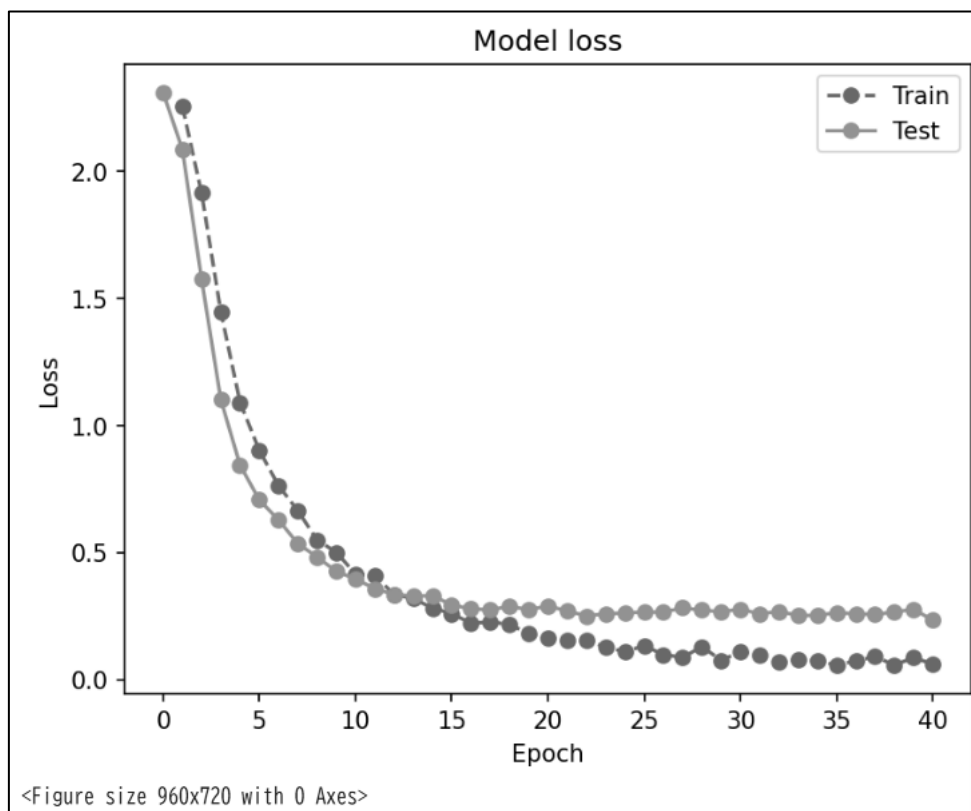


図 129 学習率 0.001、エポック 40、拡張した学習データを用いた CNN モデル 5 のエポックと精度のグラフ

6.3 C5: 画像認識コンテスト

必要なライブラリのインポート、GoogleDrive のマウント、シード値の固定、実験用ディレクトリ階層の構築を行った。

その後、TensorFlow Datasets で利用可能なデータセットである `rock_paper_scissor` をテスト用データセットとしてダウンロードした。コードおよび実行結果を図 130 に示す。

```
1 import tensorflow_datasets as tfds
2 ds_test, ds_info = tfds.load(
3     'rock_paper_scissors',
4     split='test',
5     shuffle_files = False,
6     with_info = True,
7     as_supervised = True
8 )
9 display(ds_info)

tfds.core.DatasetInfo(
  name='rock_paper_scissors',
  full_name='rock_paper_scissors/3.0.0',
  description="""
  Images of hands playing rock, paper, scissor game.
  """,
  homepage='http://laurencemoroney.com/rock-paper-scissors-dataset',
  data_dir='/root/tensorflow_datasets/rock_paper_scissors/3.0.0',
  file_format=tfrecord,
  download_size=219.53 MiB,
  dataset_size=219.23 MiB,
  features=FeaturesDict({
    'image': Image(shape=(300, 300, 3), dtype=uint8),
    'label': ClassLabel(shape=(), dtype=int64, num_classes=3),
  }),
  supervised_keys=('image', 'label'),
  disable_shuffling=False,
  nondeterministic_order=False,
  splits={
    'test': <SplitInfo num_examples=372, num_shards=1>,
    'train': <SplitInfo num_examples=2520, num_shards=2>,
  },
  citation="""@ONLINE {rps,
  author = "Laurence Moroney",
  title = "Rock, Paper, Scissors Dataset",
  month = "feb",
  year = "2019",
  url = "http://laurencemoroney.com/rock-paper-scissors-dataset"
  }""",
```

図 130 `rock_paper_scissors` データセットのダウンロード

インポートしたデータセットを PyTorch で扱うためのカスタムデータセットクラスを定義した。コードおよび実行結果を図 131 に示す。

```

1 class CustomDataset(data.Dataset):
2     def __init__(self, samples, transform=None):
3         self.samples = samples
4         self.transform = transform
5         self.targets = samples[1]
6     def __getitem__(self, index):
7         x = self.samples[0][index]
8         y = self.samples[1][index]
9         if self.transform:
10             x = self.transform(x)
11         return x, y
12     def __len__(self):
13         return len(self.samples[0])

```

図 131 カスタムデータセットクラスの定義

テスト用データセットの作成を行った。コードおよび実行結果を図 132 に示す。

```

1 from PIL import Image
2 image_size = 64
3 transform_test = T.Compose(
4     [ T.Resize((image_size, image_size)),
5       T.ToTensor()
6     ])
7 x = [Image.fromarray(data[0].numpy()) for data in ds_test]
8 y = [data[1].numpy() for data in ds_test]
9 test_dataset = CustomDataset((x, y), transform_test)
10 print('test samples', len(test_dataset))

```

test samples 372

図 132 テスト用データセットの作成

学習用データの読み込みを行った。コードおよび実行結果を図 133 に示す。

```

1 dataset_dir = '/gdrive/MyDrive/C5_水12/images/train'
2 transform = T.Compose(
3     [ T.Resize((image_size, image_size)),
4       T.ToTensor()
5     ])
6 train_dataset = datasets.ImageFolder(dataset_dir, transform)
7 print("train dataset:%n", train_dataset, "%n")
8 train_samples = len(train_dataset)
9 print("train dataset class to idx:%n", train_dataset.class_to_idx, "%n")
10 classes = [key for key in train_dataset.class_to_idx]

```

```

train dataset:
Dataset ImageFolder
  Number of datapoints: 184
  Root location: /gdrive/MyDrive/C5_水12/images/train
  StandardTransform
Transform: Compose(
  Resize(size=(64, 64), interpolation=bilinear, max_size=None, antialias=True)
  ToTensor()
)

train dataset class to idx:
{'0_グー': 0, '1_パー': 1, '2_チョキ': 2}

```

図 133 学習用データの読み込み

学習用およびテスト用データセットのバランス可視化を行った。コードおよび実行結果を図 134 に示す。

```

1 train_df = pd.DataFrame(train_dataset.targets)
2 train_df = train_df.replace({0: classes[0], 1:classes[1], 2:classes[2]})
3 balance = train_df.value_counts()
4 display('train', balance)
5 test_df = pd.DataFrame(test_dataset.targets)
6 test_df = test_df.replace({0: classes[0], 1:classes[1], 2:classes[2]})
7 balance = test_df.value_counts()
8 display('test', balance)

```

'train'

	count
0	
2_チョコ	80
1_パー	53
0_グー	51

dtype: int64

'test'

	count
0	
0_グー	124
1_パー	124
2_チョコ	124

dtype: int64

図 134 学習用およびテスト用データセットのバランス可視化

データローダを作成した。コードを図 135 に示す。

```

1 batch_size = 32
2 train_loader = data.DataLoader(train_dataset, batch_size = batch_size,
3                                shuffle=True, num_workers = 2)
4 test_loader = data.DataLoader(test_dataset, batch_size=batch_size,
5                                shuffle=False, num_workers = 2)

```

図 135 データローダ作成

データの確認を行った。コードおよび実行結果を図 136,137 に示す。

```
1 def imshow(img):
2     plt.axis('off')
3     npimg = img.numpy()
4     plt.imshow(np.transpose(npimg, (1, 2, 0)))
5     plt.show()
6
7 dataiter = iter(train_loader)
8 images, labels = next(dataiter)
9 print("train:")
10 imshow(torchvision.utils.make_grid(images))
11
12 dataiter = iter(test_loader)
13 images, labels = next(dataiter)
14 print("test:")
15 imshow(torchvision.utils.make_grid(images))
```

train:



図 136 データの確認 1

test:



図 137 データの確認 2

学習のロギング機能および精度検証機能の実装を行った。その後、ベースライン CNN の構築を行った。コードおよび実行結果を図 138 に示す。

```
1 class MyCNN(nn.Module):
2     def __init__(self):
3         super(MyCNN, self).__init__()
4         self.nets = nn.Sequential(
5             nn.Conv2d(3, 32, (3,3)),
6             nn.ReLU(),
7             nn.MaxPool2d((2,2)),
8
9             nn.Conv2d(32, 64, (3,3)),
10            nn.ReLU(),
11            nn.MaxPool2d((2,2)),
12
13            nn.Conv2d(64, 64, (3,3)),
14            nn.ReLU(),
15
16            nn.Flatten(),
17            nn.Linear(12*12*64, 64),
18            nn.ReLU(),
19
20            nn.Linear(64, 3),
21        )
22     def forward(self, x):
23         y = self.nets(x)
24         return y
25     def predict(self, x):
26         y = self.forward(x)
27         return F.softmax(y, dim=1)
```

図 138 ベースライン CNN の構築

CNN のインスタンス生成、サマリの表示および損失関数、最適化手法の設定を行った。コードおよび実行結果を図 139,140 に示す。

```

1 learning_rate = 0.01
2 init_seed(seed) #seed の値は任意に設定してよい
3 device = 'cuda' if torch.cuda.is_available() else 'cpu'
4 model = MyCNN().to(device)
5 summary(model, (3, image_size, image_size), device=device)
6 criterion = nn.CrossEntropyLoss()
7 optimizer = optim.SGD(model.parameters(), lr=learning_rate)
8 print('optimizer', optimizer)

```

図 139 CNN のインスタンス生成、サマリの表示および損失関数、最適化手法の設定のコード

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 32, 62, 62]	896
ReLU-2	[-1, 32, 62, 62]	0
MaxPool2d-3	[-1, 32, 31, 31]	0
Conv2d-4	[-1, 64, 29, 29]	18,496
ReLU-5	[-1, 64, 29, 29]	0
MaxPool2d-6	[-1, 64, 14, 14]	0
Conv2d-7	[-1, 64, 12, 12]	36,928
ReLU-8	[-1, 64, 12, 12]	0
Flatten-9	[-1, 9216]	0
Linear-10	[-1, 64]	589,888
ReLU-11	[-1, 64]	0
Linear-12	[-1, 3]	195

Total params: 646,403
 Trainable params: 646,403
 Non-trainable params: 0

Input size (MB): 0.05
 Forward/backward pass size (MB): 3.24
 Params size (MB): 2.47
 Estimated Total Size (MB): 5.75

optimizer SGD (
 Parameter Group 0
 dampening: 0
 differentiable: False
 foreach: None
 fused: None
 lr: 0.01
 maximize: False
 momentum: 0
 nesterov: False
 weight_decay: 0
)

図 140 CNN のインスタンス生成、サマリの表示および損失関数、最適化手法の設定の実行結果

CNN モデル 1 の学習を行った。コードおよび実行結果を図 141,142 に示す。


```

1 n_epoch = 10
2 print_iteration = 5 #print_iteration の値は任意に設定してよい
3 logger = Logger(print_iteration, train_samples, batch_size)
4 eval = Evaluator(test_loader, criterion, device)
5 start = time.time()
6 for epoch in tqdm(range(n_epoch)):
7     eval.evaluation(model, epoch)
8     model.train()
9     logger.clear_run()
10    print_time = time.time()
11    for i, (images, labels) in enumerate(train_loader, 0):
12        images = images.to(device)
13        labels = labels.to(device)
14        outputs = model(images)
15        optimizer.zero_grad()
16        loss = criterion(outputs, labels)
17        loss.backward()
18        optimizer.step()
19        logger.logging(outputs, loss, labels, epoch, i+1)
20    eval.evaluation(model, n_epoch)
21    elapsed_time = time.time() - start #学習に要した時間
22    print('training total', elapsed_time, 'sec.')
23
24 #データの記録
25 finalaccuracy = eval.log["acc"][-1] #accuracy
26 finalloss = eval.log["loss"][-1] # loss

```

図 141 CNN モデル 1 の学習のコード

```

100% ██████████ 10/10 [02:23<00:00, 15.09s/it]
[ 0]test loss: 1.100, accuracy: 33.33%, speed:0.144s/iter
[ 0,      5] loss:1.091, accuracy:42.50%, speed:2.536s/iter
[ 0,      6] loss:1.086, accuracy:50.00%, speed:0.153s/iter
[ 1]test loss: 1.101, accuracy: 33.33%, speed:0.127s/iter
[ 1,      5] loss:1.088, accuracy:41.88%, speed:2.471s/iter
[ 1,      6] loss:1.083, accuracy:54.17%, speed:0.129s/iter
[ 2]test loss: 1.101, accuracy: 33.33%, speed:0.125s/iter
[ 2,      5] loss:1.089, accuracy:41.25%, speed:2.369s/iter
[ 2,      6] loss:1.070, accuracy:58.33%, speed:0.127s/iter
[ 3]test loss: 1.102, accuracy: 33.33%, speed:0.125s/iter
[ 3,      5] loss:1.082, accuracy:44.38%, speed:2.545s/iter
[ 3,      6] loss:1.086, accuracy:37.50%, speed:0.135s/iter
[ 4]test loss: 1.103, accuracy: 33.33%, speed:0.129s/iter
[ 4,      5] loss:1.079, accuracy:44.38%, speed:2.280s/iter
[ 4,      6] loss:1.097, accuracy:37.50%, speed:0.223s/iter
[ 5]test loss: 1.105, accuracy: 33.33%, speed:0.136s/iter
[ 5,      5] loss:1.083, accuracy:42.50%, speed:2.026s/iter
[ 5,      6] loss:1.059, accuracy:50.00%, speed:0.206s/iter
[ 6]test loss: 1.106, accuracy: 33.33%, speed:0.185s/iter
[ 6,      5] loss:1.074, accuracy:45.62%, speed:2.136s/iter
[ 6,      6] loss:1.111, accuracy:29.17%, speed:0.161s/iter
[ 7]test loss: 1.108, accuracy: 33.33%, speed:0.159s/iter
[ 7,      5] loss:1.076, accuracy:44.38%, speed:2.165s/iter
[ 7,      6] loss:1.089, accuracy:37.50%, speed:0.146s/iter
[ 8]test loss: 1.109, accuracy: 33.33%, speed:0.139s/iter
[ 8,      5] loss:1.076, accuracy:43.75%, speed:4.228s/iter
[ 8,      6] loss:1.078, accuracy:41.67%, speed:0.238s/iter
[ 9]test loss: 1.112, accuracy: 33.33%, speed:0.177s/iter
[ 9,      5] loss:1.072, accuracy:44.38%, speed:1.999s/iter
[ 9,      6] loss:1.097, accuracy:37.50%, speed:0.222s/iter
[10]test loss: 1.114, accuracy: 33.33%, speed:0.212s/iter
training total 146.3475866317749 sec.

```

図 142 CNN モデル 1 の学習の実行結果

モデルの保存を行った。その後、学習結果の可視化を行った。実行結果を図 143,144 に示す。

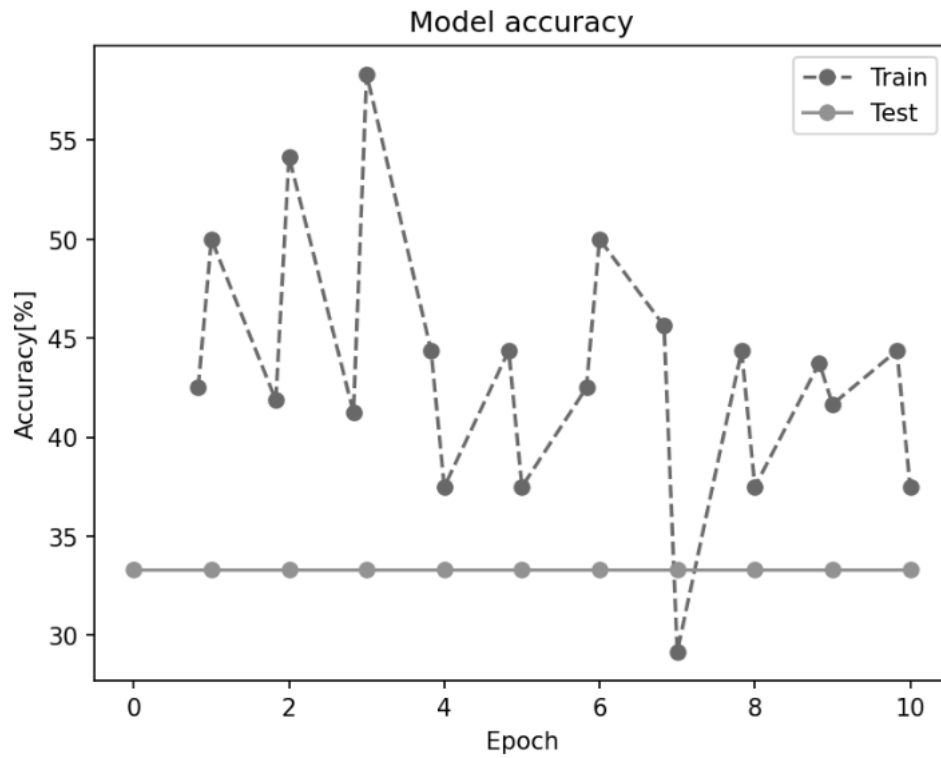


図 143 CNN モデル 1 のエポックと精度のグラフ

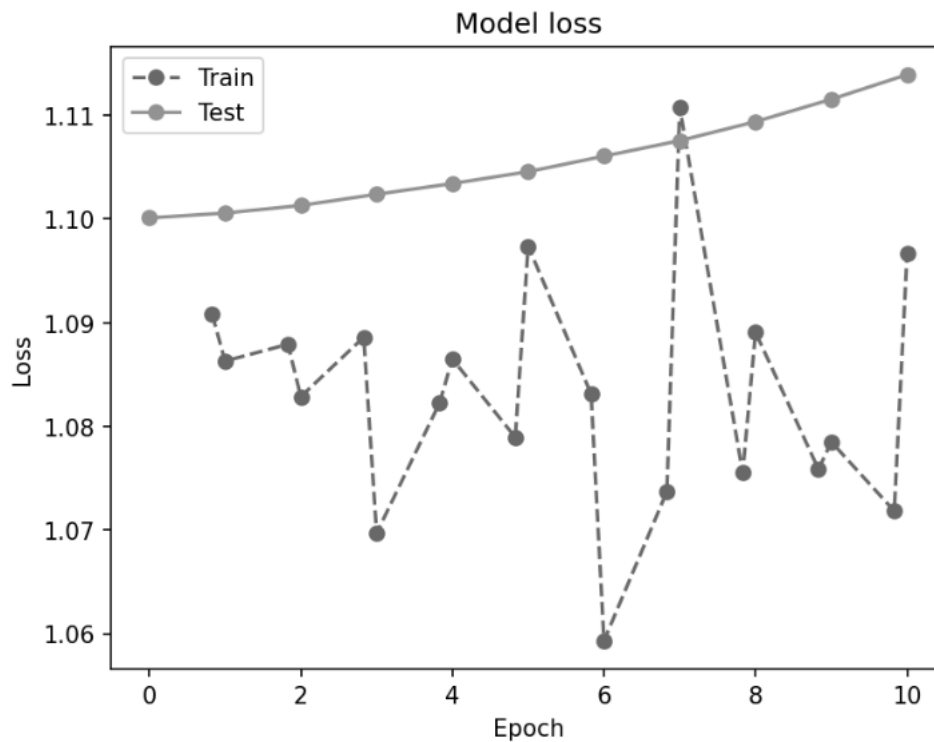


図 144 CNN モデル 1 のエポックと損失のグラフ

テストデータに対する精度検証を行った。コードおよび実行結果を図 145,146 に示す。

```

1 import torch
2 import numpy as np
3 import time
4 from tqdm.notebook import tqdm
5 from sklearn.metrics import confusion_matrix
6 def confusion(data_loader, model, classes, device):
7     model.to(device)
8     model.eval()
9     correct = 0
10    loss = 0.0
11    accuracy = 0.0
12    test_samples = 0
13    y_list=[]
14    y_hat_list=[]
15    eval_start = time.time()
16    n_iters = len(data_loader)
17    for images, labels in data_loader:
18        images = images.to(device)
19        labels = labels.to(device)
20        b_size = images.size(0)
21        test_samples += b_size
22        with torch.no_grad():
23            outputs = model(images)
24            b_loss = criterion(outputs, labels)
25            loss += b_loss.item()
26            _, predicted = torch.max(outputs.data, 1)
27            correct += (predicted == labels).sum().item()
28            y_list.append(labels.cpu().numpy())
29            y_hat_list.append(predicted.cpu().numpy())
30    eval_time = (time.time() - eval_start) / n_iters
31    loss = loss / n_iters
32    accuracy = 100 * correct / test_samples
33    print('test loss: %.3f, accuracy: %.2f%%, speed: %.3fs/iter' %
34          (loss, accuracy, eval_time))
35    y_hat = np.hstack(y_hat_list)
36    y = np.hstack(y_list)

```

図 145 精度検証のコード

```

37 print("混同行列")
38 cm = confusion_matrix(y, y_hat)
39 columns_labels = ["pred_" + str(l) for l in classes]
40 index_labels = ["act_" + str(l) for l in classes]
41 cm = pd.DataFrame(cm, columns=columns_labels, index=index_labels)
42 display(cm)
43 classes = ['グー', 'パー', 'チョキ']
44 confusion(test_loader, model, classes, device)

```

test loss: 1.114, accuracy: 33.33%, speed:0.126s/iter
混同行列

	pred_グー	pred_パー	pred_チョキ
act_グー	0	0	124
act_パー	0	0	124
act_チョキ	0	0	124

図 146 精度検証のコードおよび実行結果

図 146 より、推論精度は 33.33%であり、分類結果はすべてチョキとなっている。

CNN モデル 1 にバッチ正規化層を追加した CNN モデル 2 を定義した。コードを図 147 に示す。

```
1 class MyCNN(nn.Module):
2     def __init__(self):
3         super(MyCNN, self).__init__()
4         self.nets = nn.Sequential(
5             nn.Conv2d(3, 32, (3, 3)), #3*64*64 -> 32*62*62
6             nn.BatchNorm2d(32),
7             nn.ReLU(),
8             nn.MaxPool2d((2, 2)), # 32*31*31
9             nn.Conv2d(32, 64, (3, 3)), # 64*29*29
10            nn.BatchNorm2d(64),
11            nn.ReLU(),
12            nn.MaxPool2d((2, 2)), # 64*14*14 ?
13            nn.Conv2d(64, 64, (3, 3)), # 128*12*12
14            nn.BatchNorm2d(64),
15            nn.ReLU(),
16
17            nn.Flatten(),
18            nn.Linear(12*12*64, 12*64),
19            nn.ReLU(),
20            nn.Dropout(0.5),
21            nn.Linear(12*64, 3*64),
22            nn.ReLU(),
23            nn.Dropout(0.5),
24            nn.Linear(3*64, 64),
25            nn.ReLU(),
26            nn.Dropout(0.5),
27            nn.Linear(64, 3)
28        )
29    def forward(self, x):
30        y = self.nets(x)
31        return y
32    def predict(self, x):
33        y = self.forward(x)
34        return F.softmax(y, dim = 1)
35
```

図 147 CNN モデル 2 の定義のコード

CNN モデル 2 の学習を行った。実行結果を図 148 に示す。

```

100% ██████████ 10/10 [02:44<00:00, 16.26s/it]
[ 0]test loss: 1.100, accuracy: 33.33%, speed:0.130s/iter
[ 0,      5] loss:0.976, accuracy:56.88%, speed:2.913s/iter
[ 0,      6] loss:0.717, accuracy:75.00%, speed:0.186s/iter
[ 1]test loss: 1.106, accuracy: 33.33%, speed:0.127s/iter
[ 1,      5] loss:0.641, accuracy:80.00%, speed:3.057s/iter
[ 1,      6] loss:0.453, accuracy:87.50%, speed:0.182s/iter
[ 2]test loss: 1.134, accuracy: 33.33%, speed:0.132s/iter
[ 2,      5] loss:0.457, accuracy:85.62%, speed:3.030s/iter
[ 2,      6] loss:0.370, accuracy:91.67%, speed:0.253s/iter
[ 3]test loss: 1.201, accuracy: 33.33%, speed:0.131s/iter
[ 3,      5] loss:0.307, accuracy:91.25%, speed:2.902s/iter
[ 3,      6] loss:0.444, accuracy:79.17%, speed:0.238s/iter
[ 4]test loss: 1.145, accuracy: 33.33%, speed:0.125s/iter
[ 4,      5] loss:0.247, accuracy:93.12%, speed:2.825s/iter
[ 4,      6] loss:0.319, accuracy:79.17%, speed:0.176s/iter
[ 5]test loss: 1.144, accuracy: 34.14%, speed:0.120s/iter
[ 5,      5] loss:0.208, accuracy:95.62%, speed:2.886s/iter
[ 5,      6] loss:0.149, accuracy:100.00%, speed:0.178s/iter
[ 6]test loss: 1.305, accuracy: 33.33%, speed:0.119s/iter
[ 6,      5] loss:0.167, accuracy:97.50%, speed:2.799s/iter
[ 6,      6] loss:0.141, accuracy:100.00%, speed:0.205s/iter
[ 7]test loss: 1.227, accuracy: 41.94%, speed:0.165s/iter
[ 7,      5] loss:0.125, accuracy:100.00%, speed:2.799s/iter
[ 7,      6] loss:0.166, accuracy:100.00%, speed:0.209s/iter
[ 8]test loss: 1.418, accuracy: 38.44%, speed:0.188s/iter
[ 8,      5] loss:0.106, accuracy:100.00%, speed:2.940s/iter
[ 8,      6] loss:0.112, accuracy:100.00%, speed:0.223s/iter
[ 9]test loss: 1.619, accuracy: 38.71%, speed:0.148s/iter
[ 9,      5] loss:0.087, accuracy:100.00%, speed:2.696s/iter
[ 9,      6] loss:0.102, accuracy:100.00%, speed:0.222s/iter
[10]test loss: 1.989, accuracy: 36.56%, speed:0.131s/iter
training total 165.65352654457092 sec.

```

図 148 CNN モデル 2 の学習結果

学習結果の可視化を行った。実行結果を図 149,150 に示す。

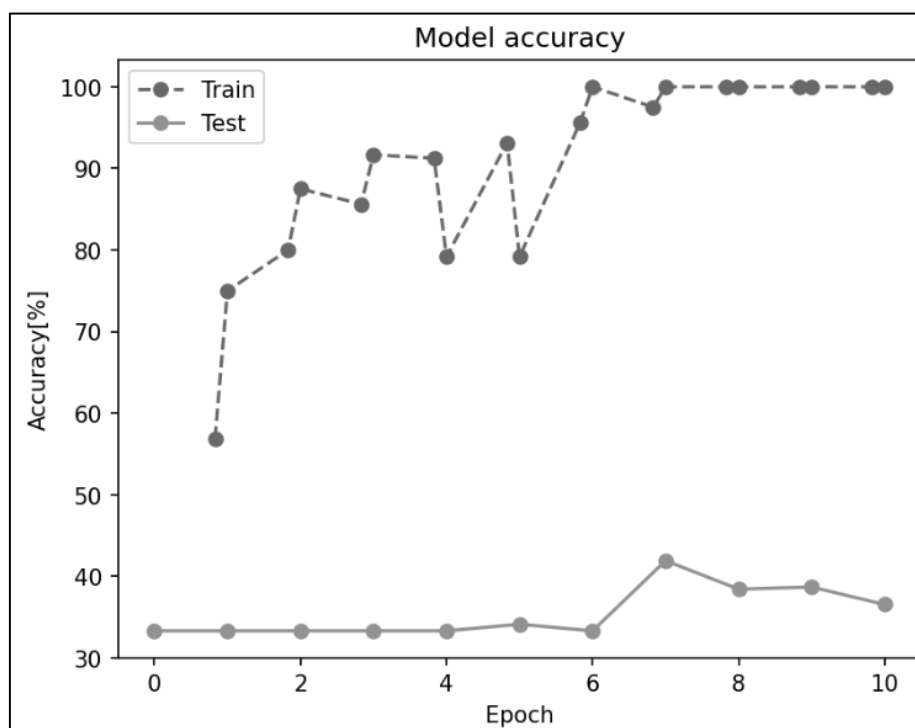


図 149 CNN モデル 2 の学習のエポックと精度のグラフ

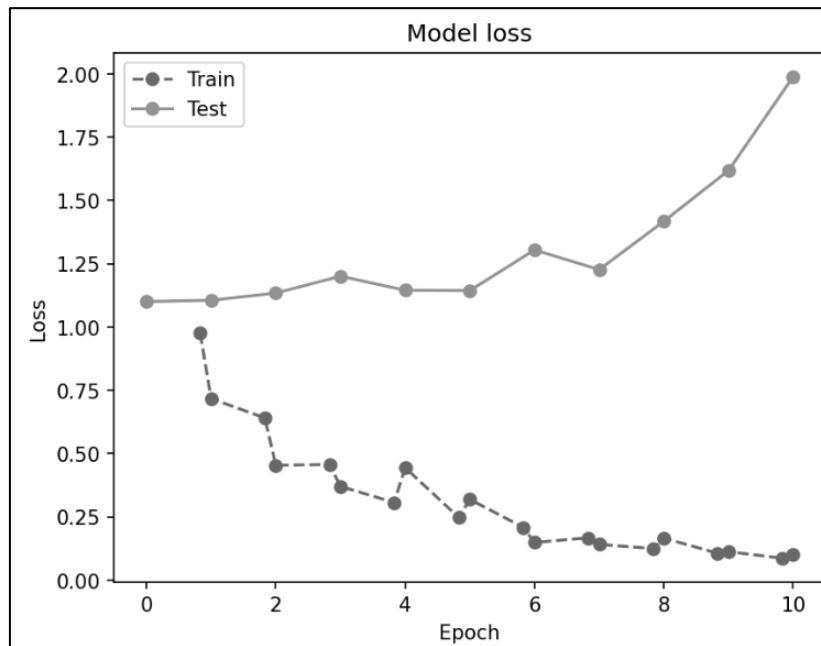


図 150 CNN モデル 2 の学習のエポックと損失のグラフ

CNN モデル 2 に全結合層とドロップアウト層を追加した CNN モデル 3 を定義した。実行結果を図 151 に示す。

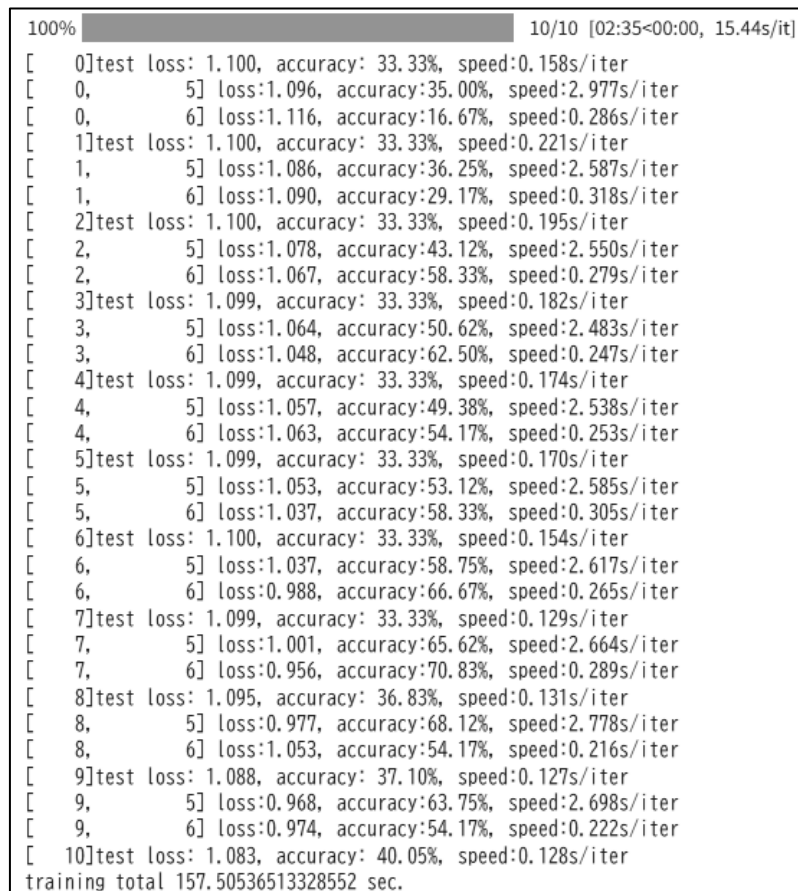


図 151 CNN モデル 3 の学習結果

学習結果の可視化を行った。実行結果を図 152,153 に示す。

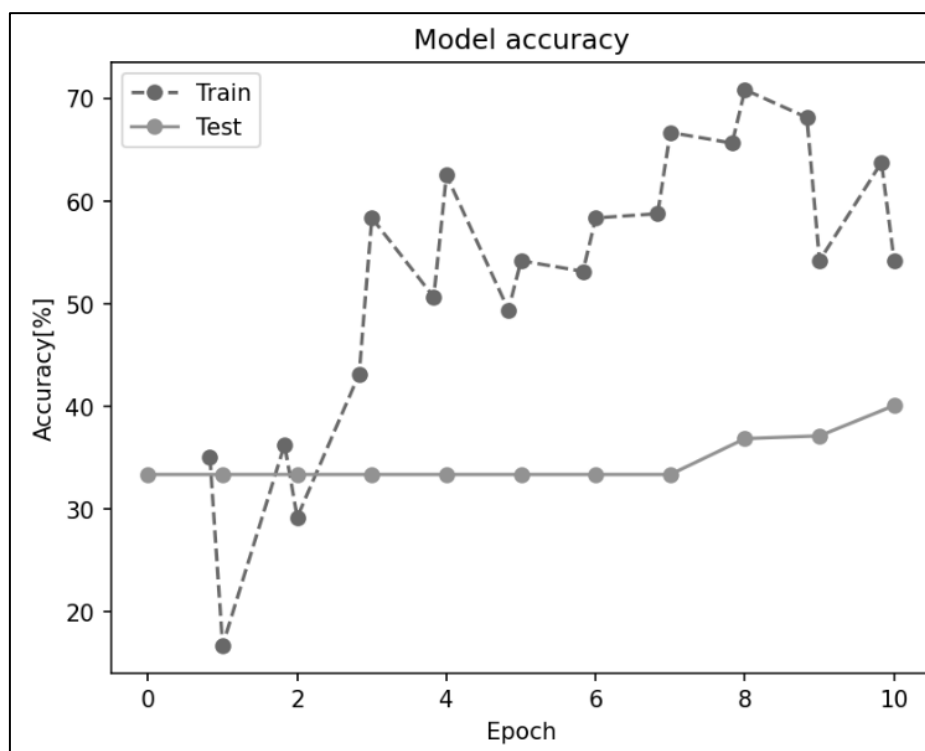


図 152 CNN モデル 3 の学習のエポックと精度のグラフ

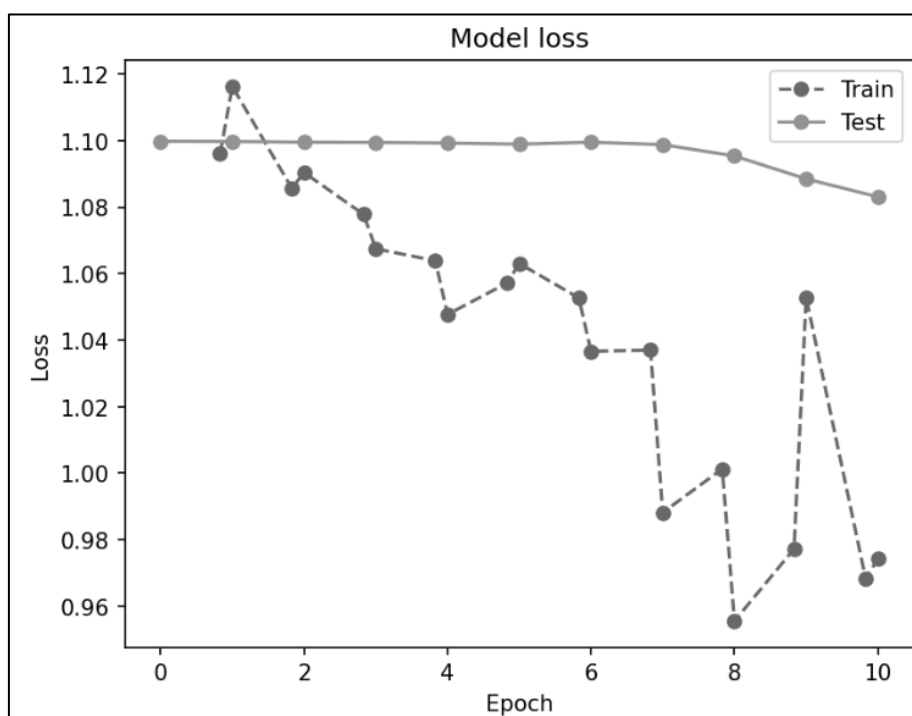


図 153 CNN モデル 3 の学習のエポックと損失のグラフ

最適化手法を Adam に変更し、CNN モデル 3 の学習を行った。実行結果を図 154 に示す。

```

100% ██████████ 10/10 [01:53<00:00, 11.42s/it]
[ 0]test loss: 1.100, accuracy: 33.33%, speed:0.048s/iter
[ 0,      5] loss:10.572, accuracy:26.88%, speed:2.130s/iter
[ 0,      6] loss:1.361, accuracy:33.33%, speed:0.011s/iter
[ 1]test loss: 1.190, accuracy: 33.33%, speed:0.033s/iter
[ 1,      5] loss:1.224, accuracy:39.38%, speed:2.235s/iter
[ 1,      6] loss:1.053, accuracy:37.50%, speed:0.011s/iter
[ 2]test loss: 1.111, accuracy: 32.53%, speed:0.032s/iter
[ 2,      5] loss:1.108, accuracy:40.00%, speed:2.280s/iter
[ 2,      6] loss:1.203, accuracy:41.67%, speed:0.126s/iter
[ 3]test loss: 1.109, accuracy: 33.33%, speed:0.032s/iter
[ 3,      5] loss:1.104, accuracy:46.88%, speed:1.861s/iter
[ 3,      6] loss:1.004, accuracy:50.00%, speed:0.011s/iter
[ 4]test loss: 1.169, accuracy: 34.95%, speed:0.050s/iter
[ 4,      5] loss:0.887, accuracy:58.12%, speed:2.149s/iter
[ 4,      6] loss:0.912, accuracy:58.33%, speed:0.011s/iter
[ 5]test loss: 1.189, accuracy: 33.87%, speed:0.032s/iter
[ 5,      5] loss:0.892, accuracy:63.12%, speed:2.281s/iter
[ 5,      6] loss:0.542, accuracy:70.83%, speed:0.011s/iter
[ 6]test loss: 2.269, accuracy: 33.33%, speed:0.034s/iter
[ 6,      5] loss:0.823, accuracy:66.88%, speed:2.232s/iter
[ 6,      6] loss:0.659, accuracy:75.00%, speed:0.011s/iter
[ 7]test loss: 2.661, accuracy: 33.33%, speed:0.034s/iter
[ 7,      5] loss:0.746, accuracy:71.88%, speed:2.000s/iter
[ 7,      6] loss:0.687, accuracy:66.67%, speed:0.012s/iter
[ 8]test loss: 4.959, accuracy: 40.59%, speed:0.056s/iter
[ 8,      5] loss:0.557, accuracy:75.62%, speed:1.958s/iter
[ 8,      6] loss:0.852, accuracy:70.83%, speed:0.823s/iter
[ 9]test loss: 8.804, accuracy: 33.33%, speed:0.033s/iter
[ 9,      5] loss:0.620, accuracy:72.50%, speed:2.282s/iter
[ 9,      6] loss:1.031, accuracy:50.00%, speed:0.011s/iter
[10]test loss: 5.967, accuracy: 36.29%, speed:0.032s/iter
training total 113.76920580863953 sec.

```

図 154 CNN モデル 3 の Adam を用いた学習結果

学習結果の可視化を行った。実行結果を図 155,156 に示す。

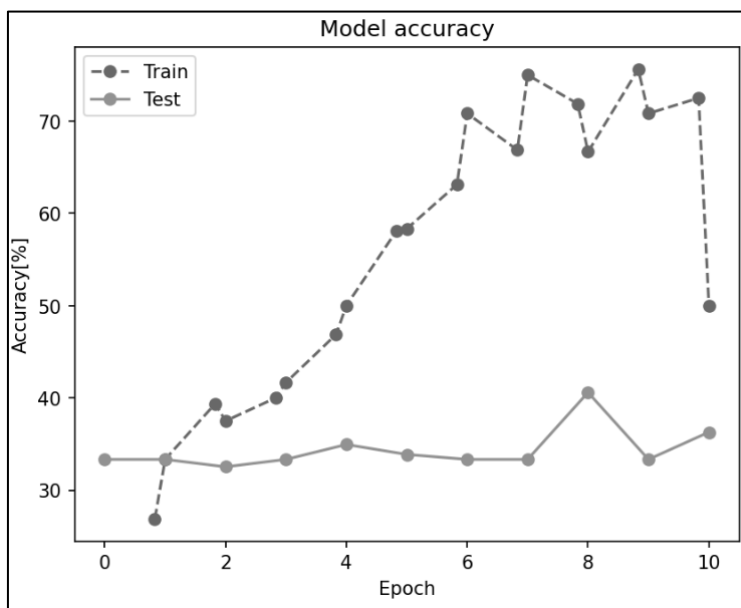


図 155 CNN モデル 3 の Adam を用いた学習のエポックと精度のグラフ

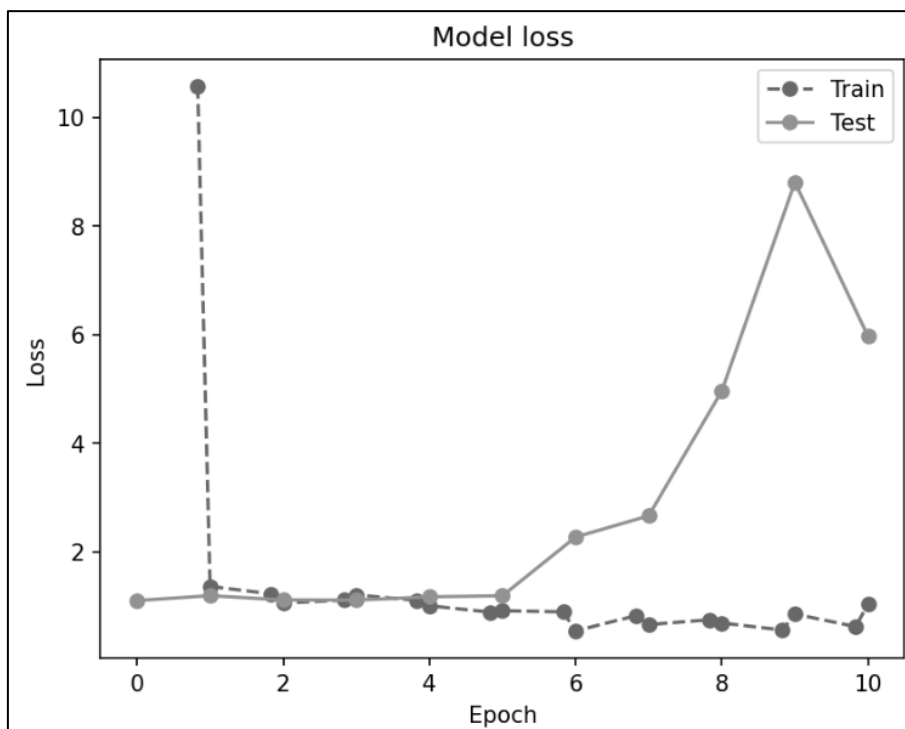


図 156 CNN モデル 3 の Adam を用いた学習のエポックと損失のグラフ

画像の前処理に回転、歪み、左右反転、明度変化を加え、学習データを拡張した。コードと実行結果を図 157 に示す。

```

1 dataset_dir = '/gdrive/MyDrive/C5_水12/images/train'
2 transform = T.Compose([
3     T.Resize((image_size, image_size)),
4     T.RandomRotation(degrees=5, fill=255), # 回転
5     T.RandomPerspective(distortion_scale=0.1, fill = 255), # 歪み
6     T.RandomHorizontalFlip(), # 左右反転
7     T.ToTensor(),
8 ])
9 train_dataset = datasets.ImageFolder(dataset_dir, transform)
10 print("train dataset:%n",train_dataset,"%n")
11 train_samples = len(train_dataset)
12 print("train dataset class to idx:%n",train_dataset.class_to_idx,"%n")
13 classes = [key for key in train_dataset.class_to_idx]

train dataset:
Dataset ImageFolder
  Number of datapoints: 184
  Root location: /gdrive/MyDrive/C5_水12/images/train
  StandardTransform
Transform: Compose(
  Resize(size=(64, 64), interpolation=bilinear, max size=None, antialias=True)
  RandomRotation(degrees=[-5.0, 5.0], interpolation=nearest, expand=False, fill=255)
  RandomPerspective(p=0.5)
  RandomHorizontalFlip(p=0.5)
  ToTensor()
)

train dataset class to idx:
{'0_グー': 0, '1_パー': 1, '2_チョコキ': 2}

```

図 157 画像の前処理による学習データの拡張

CNN モデル 3、Adam を用いて拡張した学習データのエポック 20 での学習を行った。

```
[ 0,      6] loss:6.317, accuracy:36.98%, speed:1.924s/iter
[ 1]test loss: 4.552, accuracy: 33.33%, speed:0.032s/iter
[ 1,      6] loss:1.456, accuracy:35.24%, speed:2.012s/iter
[ 2]test loss: 1.206, accuracy: 33.33%, speed:0.032s/iter
[ 2,      6] loss:1.167, accuracy:42.71%, speed:1.859s/iter
[ 3]test loss: 1.205, accuracy: 33.33%, speed:0.051s/iter
[ 3,      6] loss:1.086, accuracy:46.01%, speed:1.625s/iter
[ 4]test loss: 1.137, accuracy: 33.33%, speed:0.031s/iter
[ 4,      6] loss:1.169, accuracy:39.41%, speed:1.985s/iter
[ 5]test loss: 1.131, accuracy: 37.37%, speed:0.034s/iter
[ 5,      6] loss:1.123, accuracy:40.10%, speed:1.962s/iter
[ 6]test loss: 1.243, accuracy: 33.33%, speed:0.031s/iter
[ 6,      6] loss:1.020, accuracy:48.44%, speed:1.969s/iter
[ 7]test loss: 2.270, accuracy: 33.33%, speed:0.034s/iter
[ 7,      6] loss:1.005, accuracy:51.56%, speed:1.875s/iter
[ 8]test loss: 2.002, accuracy: 33.33%, speed:0.043s/iter
[ 8,      6] loss:1.033, accuracy:44.27%, speed:1.600s/iter
[ 9]test loss: 6.414, accuracy: 33.33%, speed:0.038s/iter
[ 9,      6] loss:0.961, accuracy:48.44%, speed:1.934s/iter
[10]test loss: 4.610, accuracy: 33.33%, speed:0.030s/iter
[10,      6] loss:0.887, accuracy:54.86%, speed:1.941s/iter
[11]test loss: 19.715, accuracy: 33.33%, speed:0.030s/iter
[11,      6] loss:0.935, accuracy:58.85%, speed:1.992s/iter
[12]test loss: 9.237, accuracy: 33.33%, speed:0.031s/iter
[12,      6] loss:0.939, accuracy:59.03%, speed:1.967s/iter
[13]test loss: 5.316, accuracy: 36.29%, speed:0.033s/iter
[13,      6] loss:0.711, accuracy:67.36%, speed:1.613s/iter
[14]test loss: 1.988, accuracy: 36.02%, speed:0.055s/iter
[14,      6] loss:0.958, accuracy:70.83%, speed:1.816s/iter
[15]test loss: 2.367, accuracy: 34.95%, speed:0.035s/iter
[15,      6] loss:0.769, accuracy:64.06%, speed:1.993s/iter
[16]test loss: 1.584, accuracy: 34.68%, speed:0.031s/iter
[16,      6] loss:0.764, accuracy:64.41%, speed:1.978s/iter
[17]test loss: 1.455, accuracy: 34.14%, speed:0.032s/iter
[17,      6] loss:0.723, accuracy:65.45%, speed:1.950s/iter
[18]test loss: 1.589, accuracy: 47.31%, speed:0.033s/iter
[18,      6] loss:1.097, accuracy:65.28%, speed:1.735s/iter
[19]test loss: 1.746, accuracy: 39.25%, speed:0.056s/iter
[19,      6] loss:0.952, accuracy:56.77%, speed:1.829s/iter
[20]test loss: 2.264, accuracy: 43.55%, speed:0.034s/iter
training total 235.94676208496094 sec.
```

図 158 拡張した学習データのエポック 20 での学習結果

学習の可視化を行った。実行結果を図 159,160 に示す。

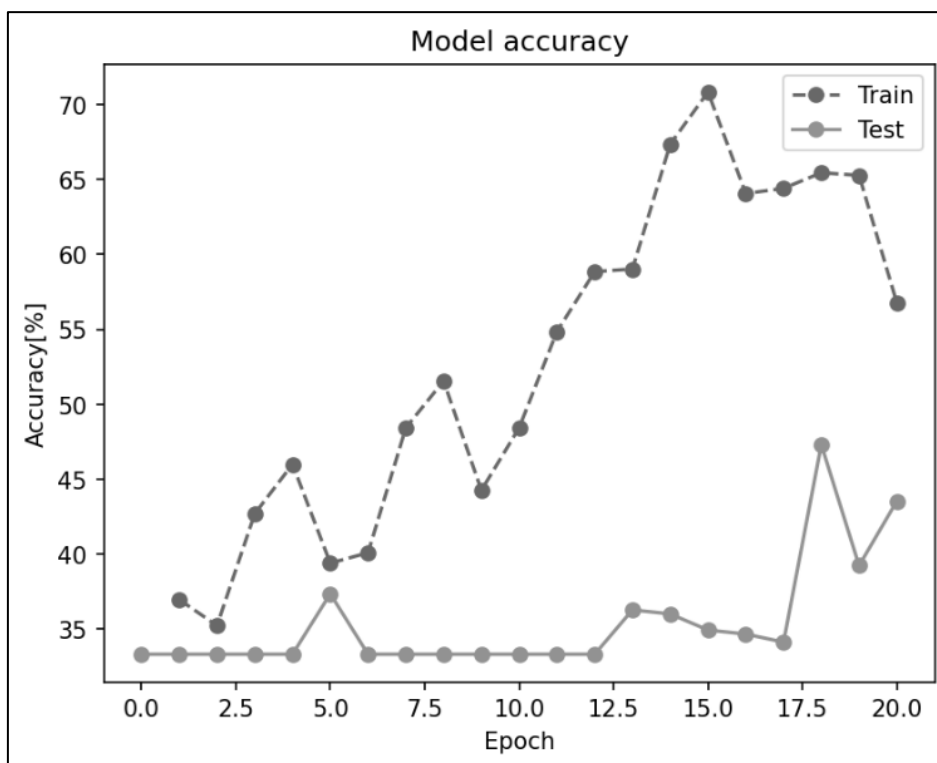


図 159 拡張した学習データのエポックと精度のグラフ

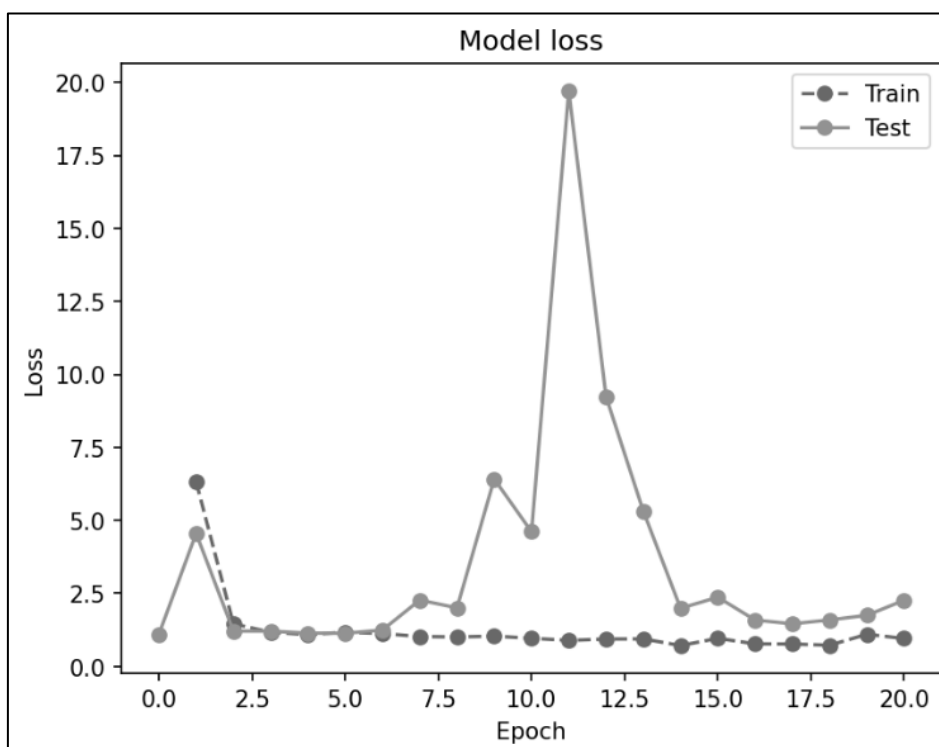


図 160 拡張した学習データのエポックと損失のグラフ

エポック 1~6 の学習率を 0.0005、エポック 7~10 の学習率を 0.0001 にして学習を行った。実行結果を図 161 に示す。

```

100% 10/10 [02:06<00:00, 12.84s/it]
[ 0]test loss: 1.108, accuracy: 33.33%, speed:0.059s/iter
[ 0,      5] loss:1.089, accuracy:30.00%, speed:2.132s/iter
[ 0,      6] loss:1.143, accuracy:20.83%, speed:0.012s/iter
[ 1]test loss: 1.105, accuracy: 33.33%, speed:0.044s/iter
[ 1,      5] loss:0.963, accuracy:42.50%, speed:2.496s/iter
[ 1,      6] loss:0.958, accuracy:41.67%, speed:0.012s/iter
[ 2]test loss: 1.117, accuracy: 33.33%, speed:0.036s/iter
[ 2,      5] loss:0.844, accuracy:50.62%, speed:2.419s/iter
[ 2,      6] loss:0.654, accuracy:54.17%, speed:0.264s/iter
[ 3]test loss: 1.143, accuracy: 33.33%, speed:0.037s/iter
[ 3,      5] loss:0.714, accuracy:63.75%, speed:2.564s/iter
[ 3,      6] loss:0.774, accuracy:45.83%, speed:0.013s/iter
[ 4]test loss: 1.310, accuracy: 33.33%, speed:0.035s/iter
[ 4,      5] loss:0.601, accuracy:61.25%, speed:2.524s/iter
[ 4,      6] loss:0.662, accuracy:54.17%, speed:0.011s/iter
[ 5]test loss: 1.445, accuracy: 33.33%, speed:0.036s/iter
[ 5,      5] loss:0.538, accuracy:64.38%, speed:2.343s/iter
[ 5,      6] loss:0.413, accuracy:79.17%, speed:0.011s/iter
[ 6]test loss: 1.424, accuracy: 33.33%, speed:0.059s/iter
[ 6,      5] loss:0.464, accuracy:71.88%, speed:2.151s/iter
[ 6,      6] loss:0.489, accuracy:70.83%, speed:0.011s/iter
[ 7]test loss: 0.982, accuracy: 61.29%, speed:0.059s/iter
[ 7,      5] loss:0.420, accuracy:74.38%, speed:2.409s/iter
[ 7,      6] loss:0.322, accuracy:87.50%, speed:0.011s/iter
[ 8]test loss: 1.476, accuracy: 43.82%, speed:0.037s/iter
[ 8,      5] loss:0.407, accuracy:80.62%, speed:2.340s/iter
[ 8,      6] loss:0.459, accuracy:75.00%, speed:0.876s/iter
[ 9]test loss: 1.180, accuracy: 61.56%, speed:0.036s/iter
[ 9,      5] loss:0.354, accuracy:79.38%, speed:2.534s/iter
[ 9,      6] loss:0.420, accuracy:70.83%, speed:0.011s/iter
[10]test loss: 1.344, accuracy: 57.53%, speed:0.036s/iter
training total 127.19214224815369 sec.

```

図 161 エポック設定変更後の学習結果

学習結果の可視化を行った。実行結果を図 162,163 に示す。

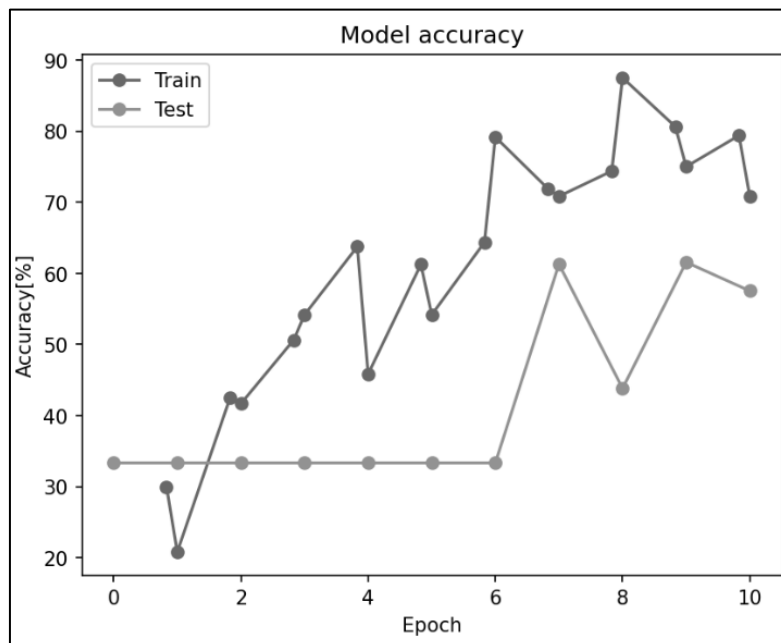


図 162 エポック設定変更後の学習のエポックと精度のグラフ

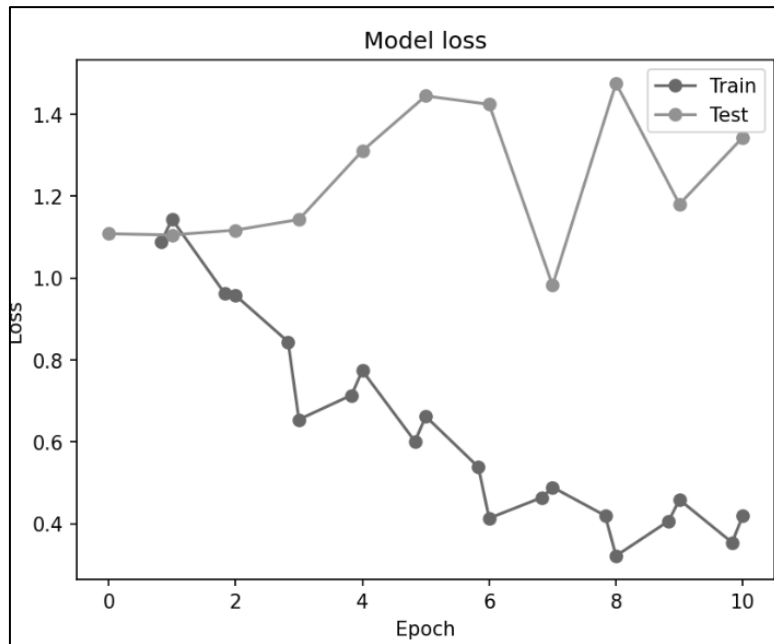


図 163 エポック設定変更後の学習のエポックと損失のグラフ

推論結果の混同行列を図 164 に示す。

test loss: 1.344, accuracy: 57.53%, speed:0.037s/iter
混同行列

	pred_グー	pred_パー	pred_チョコ
act_グー	90	0	34
act_パー	0	0	124
act_チョコ	0	0	124

図 164 エポック変更後の推論の混同行列

7. 考察

7.1 考察課題

7.1.1 C3 実験

(1) ライブラリを用いてモデルを作成することのメリット

PyTorch のようなライブラリを用いてモデルを作成することで、ニューラルネットワークの構築や学習において処理を容易に、簡潔に記述することができる。これによって、可読性の高いコードとなる。また、既成のコードを用いることで、エラーの原因となる実装ミスを抑制することができる。加えて、使用されているライブラリの知識を持っていれば、どのような構造となっているのかを容易に理解することができる。そのため、モデルの開発効率の向上と、コードの可読性、品質の向上においてメリットがあると考ええる。

(2) シード値の固定

シードを固定することで、学習結果に再現性が生まれる。シードによるランダム性がある場合、重みの初期化やデータシャッフルのパターンがランダムになることで同じモデルでも学習を実行する度に結果が変動する。また、モデルを変更する際に、精度が上がった原因がモデルの変更によるものかランダム要素によるものか判別することが困難となる。そのため、シードの固定はモデルの精度向上のための開発を行う場合、特に必要となると考える。

(3) 図 22 のコードのアーキテクチャを示すモデル図

図 165 にモデル図を示す。

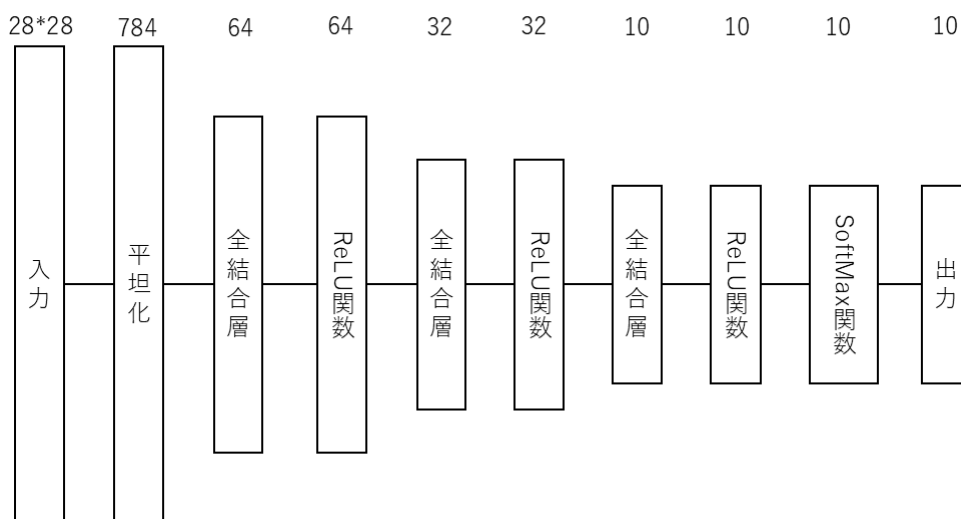


図 165 図のコードのアーキテクチャのモデル図

28*28 の画像データが入力される。入力データの平坦化を行い、全結合層と ReLU 関数を通して推論が行われる。SoftMax 関数により正規化され、各クラスに分類される確率として出力されている。

(4) 図 35 のコードによりどのようなモデルが作成されるか

図 35 より、ResNet50 の 50 層モデルが作成される。

図 37,38,39,40,41 より、このモデルは畳み込み層と ReLU 関数、ボトルネック層、MAX プーリング層、AVG プーリング層、全結合層によって構成されている。畳み込み層と ReLU 層を通して学習し、ボトルネック層で圧縮することで、計算効率を上げ、層を深くしている。また、スキップ接続により畳み込み層をスキップした入力を足し合わせることで、勾配消失問題を抑制している。

(5) ResNet50 を用いて推論可能な画像

図 48,52,54,56 より、ImageNet で事前学習された ResNet50 を用いた画像分類では、チワワおよびポメラニアンでは正しい分類が行われた。一方、空と丘の風景写真および柴犬は誤った推論結果となった。このことから、ResNet50 は ImageNet の訓練データにより事前学習されたモデルであり、未学習のクラスに対しては推論を行うことができないと考える。ImageNet により事前学習された ResNet50 を用いて推論可能な画像は、ImageNet に含まれる、学習済みの画像の種類のみである。

7.1.2 C4 実験

(1) 図 60 の CNN のアーキテクチャを示すモデル図

図 166 にモデル図を示す。

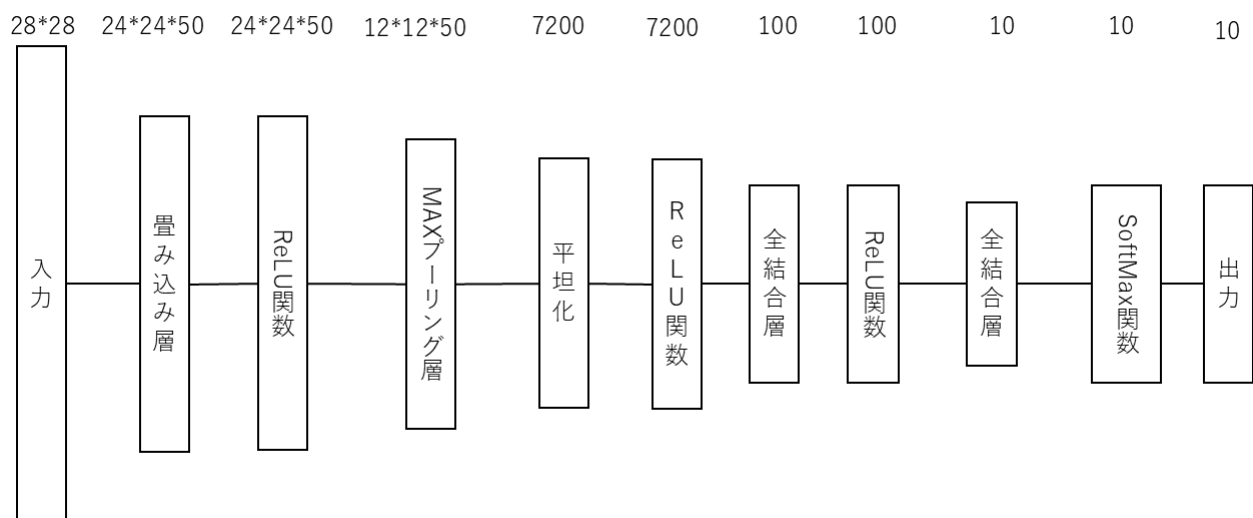


図 166 図 60 の CNN のアーキテクチャのモデル図

28*28 の画像データが入力される。入力データの畳み込みと MAX プーリングによる特徴抽出を畳み込み層と MAX プーリング層により行う。活性化関数に ReLU 関数を用いている。

(2) Optimizer に SGD を用いた場合と Adam を用いた場合の推論結果の変化

図 64,65,75,76 より SGD を用いた場合と比較して、Adam を用いた場合の最終的な推論精度がより高い。また、図 68,69,77,78 より SGD と比較して Adam の精度および損失の収束が早い。SGD はすべてのパラメータの学習率が設定した学習率で固定される。Adam は過去の勾配から学習率を動的に制御する。

(3) ランタイムのハードウェアアクセラレータに CPU を用いた場合と GPU を用いた場合の実験結果の変化

図 64,65,70,71 より CPU を用いた場合と GPU を用いた場合で、精度や損失などの実験結果に変化は見られなかった。しかし、学習時間が CPU の場合約 654 秒、GPU の場合約 105 秒であり、GPU を用いた場合の学習時間がおおよそ 0.16 倍となった。CPU が単一処理に適した高性能のコアを持つのに対して、GPU は多くのコアを利用して並列でデータ処理を行う。そのため今回のような画像データの演算に適しており、処理時間が大きく減少したと考える。よって、GPU は CPU と比較して、学習の高速化に効果がある。

(4) 学習データを 100 分の 1 にした状態に対して最も精度が高かったモデル

今回の実験では最適化手法として Adam を用いて、畳み込み層および MAX プーリング層を追加した CNN モデルをエポック 40、学習率 0.001 で学習を行ったモデルの精度が最も高かった。それぞれが精度向上に与えた効果の考察を以下に示す。

1. Adam: 学習率の制御により、学習データの少ない条件においても効率的な学習が行われたため。
2. 畳み込み層および MAX プーリング層: 畳み込み層の追加によりモデルが複雑化し、MAX プーリング層により特徴抽出を行い、特徴を捉える表現力が上がったため。
3. エポック数: 学習データを減らしたことにより不足していた学習量を、エポックを増やすことで確保したため。
4. 学習率: 0.001 に設定したことで、パラメータの更新が安定し、局所最適解に陥りにくくなった。
また、学習の早期収束により過学習となりにくかったため。

以上のことから、モデルの精度向上には十分な学習と過学習の抑制、適切な学習率が重要であると考ええる。

7.1.3 C5 実験

(1) 図 167 の CNN のアーキテクチャを示すモデル図

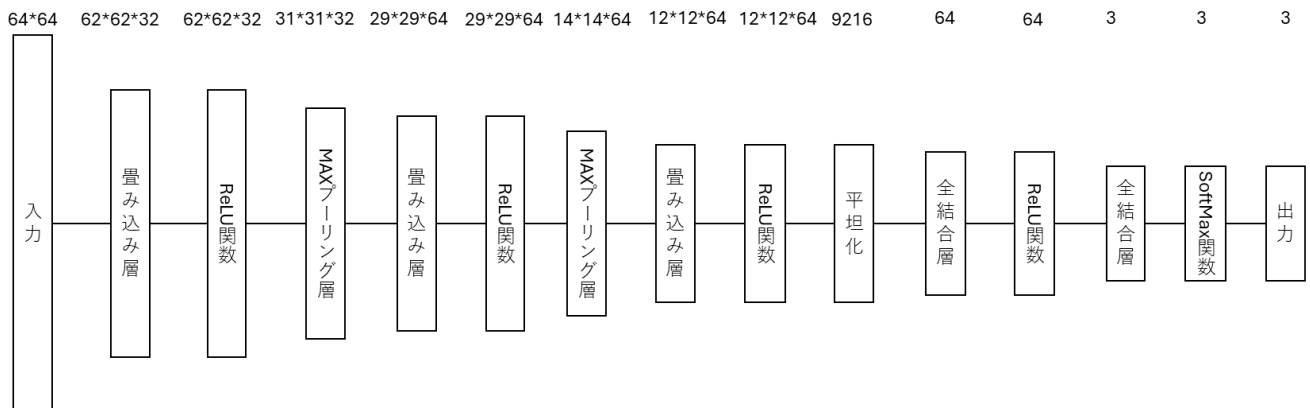


図 167 図のアーキテクチャのモデル図

64*64 の画像データの入力に対して畳み込みと MAX プーリングを繰り返し、ReLU 関数と全結合層、SoftMax 関数を通して 3 つの出力となるモデルである。

(2) 用意した学習用データとテストデータの違い

使用したテストデータは白背景で統一されており、明度も一定である。用意した学習用データは、背景や明度などの条件が統一されておらず、グー、チョキ、パーの手の形以外のノイズが多く含まれていた。そのため、ノイズをグー、チョキ、パーの特徴としてしまい、手の形の特徴を捉えられずに特定のクラスに推論が集中してしまう。

(3) テストデータに対して最も精度が高かったモデル

今回の実験では最適化手法として Adam を用いて、バッチ正規化層とドロップアウト層、全結合層を追加し、エポック 6 以下で学習率 0.0005、エポック 7 以上で学習率 0.0001 として、回転、歪み、左右反転、明度変化の画像の前処理を加えたモデルの精度が最も高かった。学習データが少ないため、バッチ正規化層やドロップアウト層など、過学習の防止による精度への影響が大きいと考えられる。バッチ正規化層は平均や分散を正規化し、データ分布の幅を抑えるため、ノイズとなる要素を削減し、学習の安定化に効果がある。

7.2 実験考察

7.2.1 C3 実験

図 29,30 のグラフより、エポック 1,2 は誤差が大きいため勾配も大きく、精度が大きく上昇し、損失が急激に減少した。損失の小さくなったエポック 3 以降では少しずつ精度が向上した。エポックが小さいほど精度が安定しておらず、損失が大きくなるためこのような結果となったと考える。

図 31,32 の推論結果より、学習済みモデルを一度保存した場合でも推論結果が変わらないことを確認した。同じモデルを使用した場合、同じデータへの推論結果は変化しない。

7.2.2 C4 実験

図 65,83 より、学習用データ数を 100 分の 1 にしたところ、精度が 96.32%から 73.34%に大きく下がった。また、図 68,69,84,85 のグラフより、100 分の 1 にした場合は精度と損失の変化がなだらかになっている。最終的な精度の低下の原因は、学習用のデータ数の減少によりそれぞれの数字の特徴抽出が不正確になったこと、エポック 1 回ごとの学習するデータ数の減少であると考ええる。損失の変化が小さい原因は、エポック 1 回ごとの学習するデータ数の減少により、精度の改善の幅が小さくなったことである。

図 87 より、Adam を用いた場合にエポック 6 以降の学習データへの推論精度が 100%、テストデータへの推論精度が 90%前後となってしまうっており、学習データに過度に適合してしまう過学習が見られた。学習用データ数が少ない場合、より過学習が起きやすくなると考える。

7.2.3 C5 実験

図 134 の混同行列より、すべての入力画像の推論結果がチョキである。また、図 164 の混同行列より、パーがすべてチョキと推論されており、グーも一部がチョキと推論されている。このことから、スプリアス相関によりチョキの学習データに含まれるノイズがテストデータに適合してしまっており、チョキと判定されやすくなっていると考ええる。特にチョキに適合した原因としては、指が 2 本出ているチョキの手の形の複雑さや、背景色などがノイズの特徴と偶然一致しやすかったという可能性がある。

推論精度を向上させるためには、前処理においてクロッピングによる背景の除去を行い、学習データの画像から対象物以外のノイズを削減すること、モデルの層において、プーリング層や畳み込み層などの特徴抽出のための層を増やすことなどが有効であると考えられる。

8. 参考文献

- [1] ディープラーニングがわかる数学入門, 涌井 良幸, 涌井 貞美, 技術評論社, 2017
- [2] ゼロから作る Deep Learning, 斎藤 康毅, オライリー・ジャパン, 2016
- [3] PyTorch ニューラルネットワーク実装ハンドブック, 宮本圭一郎, 大川洋平, 毛利拓地, 秀和システム, 2019